Angewandte KI & KI-Engineering (AKE) · Block B – Spezialisierte Anwendungsgebiete

Zeitreihen & Reinforcement Learning

Vorlesung · Woche 5

Stefan Gnaase · PHWT · Wintersemester 2026/27

Organisatorisches – Woche 5

Heute: Vorlesung mit sechs Lab-Einschüben, am Ende Capstone-Themenwahl

Termin	KW 45 (05.11.) · 270 Min.: Vorlesung mit Lab-Einschüben, Pausen zusätzlich
Vorher	Kursordner aktualisieren: git pull · uv sync (neu: pytorch-forecasting, gymnasium)
Lab 05	Sechs Aufgaben A1–A6 im Notebook lab05, je direkt nach dem Abschnitt · 80 Min.
Optional	lab05_rl_optional: DQN auf CartPole zu Hause (≈ 5 Min. Rechenzeit)
Capstone	Heute: Deadline Themenwahl, Genehmigung Thema 8 · Kickoff Woche 6 (12.11.)
Online	otexts.com/fpp3 · incompleteideas.net/book · artificialintelligenceact.eu

Roter Faden und Ablauf heute

Block B, Teil 2: Signale über die Zeit vorhersagen – und Entscheidungen über die Zeit lernen

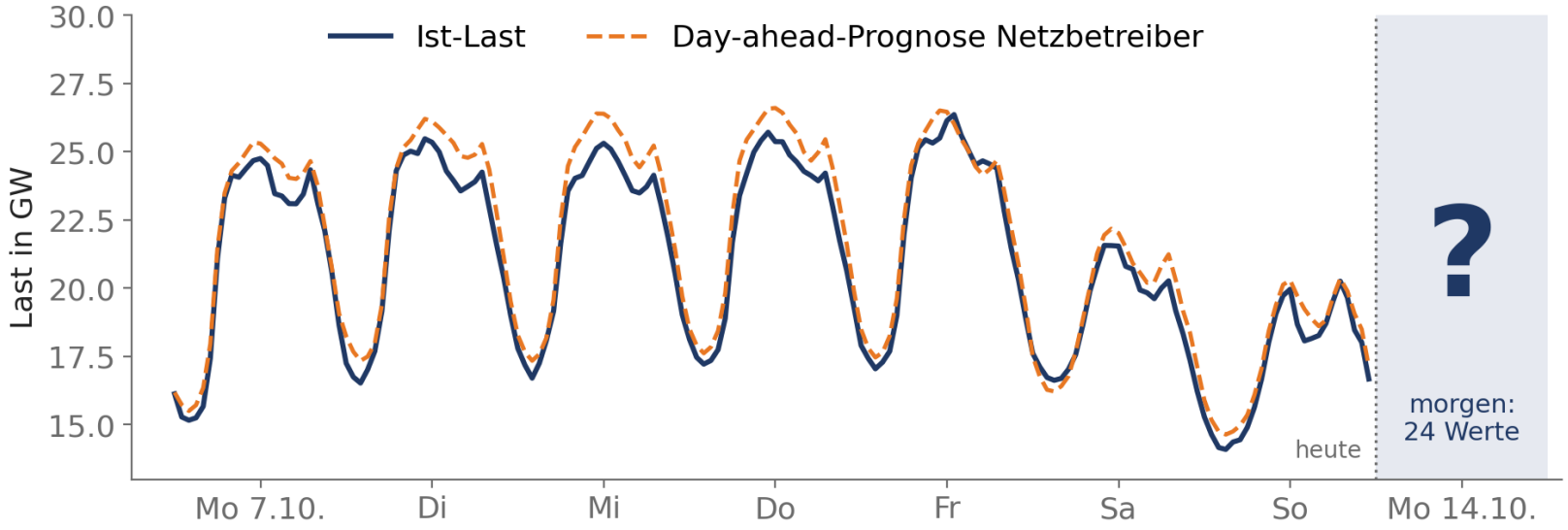
Modelle bauen → **Modelle produktiv machen** → **Modelle verantworten** → **Capstone**

heute: Ende von Block B

#	Abschnitt	Leitfrage	Lab
1	Zeitreihen verstehen	Was steckt in einer Lastkurve?	A1
2	Prognosen bewerten	Wann ist eine Prognose gut?	A2
3	Klassische Modelle	Wie weit kommt man mit Statistik?	A3
4	ML & Deep Learning	Was bringen Lags, LSTM und Transformer?	A4, A5
5	Anomalien & Drift	Was sagt das Residuum?	A6
6	RL kompakt	Wie lernt ein Agent aus Belohnung?	–
C	Capstone	Welches Thema – und welcher Compliance-Aufwand?	–

Wozu brauchen wir das?

Ein Netzbetreiber muss die Last von morgen kennen, Stunde für Stunde



513 MW — mittlerer Fehler der Netzbetreiber-Prognose 2019

2,5 % — derselbe Fehler relativ zur Last

8.760 — Stundenwerte pro Jahr – jede einzeln vorherzusagen

Andere Beispiele: Absatz im Handel, Ersatzteilbedarf, Auslastung einer Linie, Sensorwerte für Predictive Maintenance

Daten: Open Power System Data (2020), Time series, Version 2020-10-06, doi:10.25832/time_series/2020-10-06 (Rohdaten ENTSO-E)

Unser Leitbeispiel: **Day-ahead-Lastprognose** der Regelzone Amprion ➔ Messlatte ist die Prognose des Netzbetreibers

Die Geschichte des Tages

Erst verstehen, dann messen, dann modellieren, am Ende ein Blick auf Entscheidungen

1 Zeitreihen

Was steckt in einer Lastkurve?

Trend, zwei Saisons, Rest

2 Bewerten

Wann ist eine Prognose gut?

Baseline + Backtest, kein Zufallssplit

3 Statistik

Wie weit kommt man mit Statistik?

ETS, ARIMA ➔ MSTL für zwei Saisons

4 ML & DL

Was bringen Lags, LSTM, TFT?

Merkmale schlagen Architektur

5 Anomalien

Was sagt das Residuum?

Lockdown 2020: echter Drift

6 RL

Wie lernt ein Agent aus Belohnung?

MDP → Return → Bellman → PPO

Roter Faden: **ohne Baseline und Backtest ist keine Prognose etwas wert**

1

Zeitreihen verstehen

Leitfrage Was steckt in einer Lastkurve?



Was ist eine Zeitreihe?

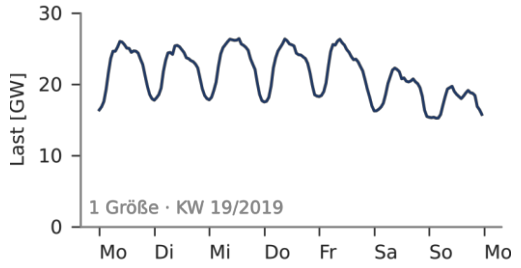
Werte in regelmäßigen Zeitabständen ➔ die Reihenfolge ist Teil der Information

Definition Eine Zeitreihe ist eine zeitlich geordnete Folge statistischer Maßzahlen

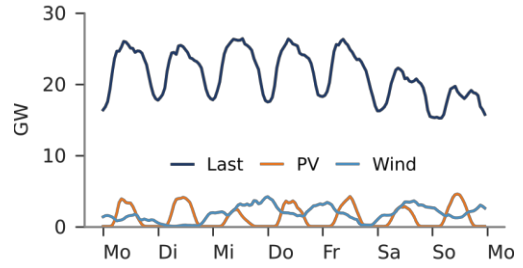
Begriffe

- **univariat:** eine Größe über die Zeit, z. B. nur die Last
- **multivariat:** mehrere (zusammenhängende) Größen gleichzeitig (Last, PV, Wind)
- **Aufgaben:**
  - Prognose
  - Anomalieerkennung
  - Imputation
  - Klassifikation
- **Horizont h:** wie weit voraus ➔ heute 1 bis 24 Stunden

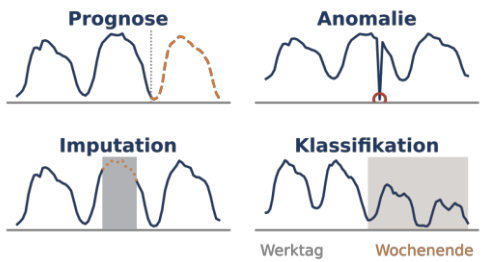
univariat



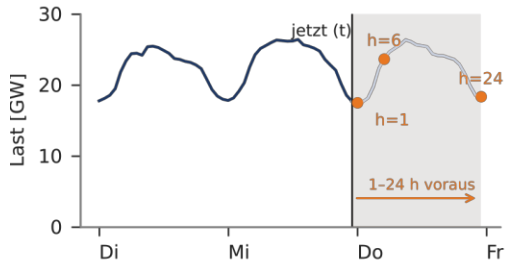
multivariat



Aufgaben



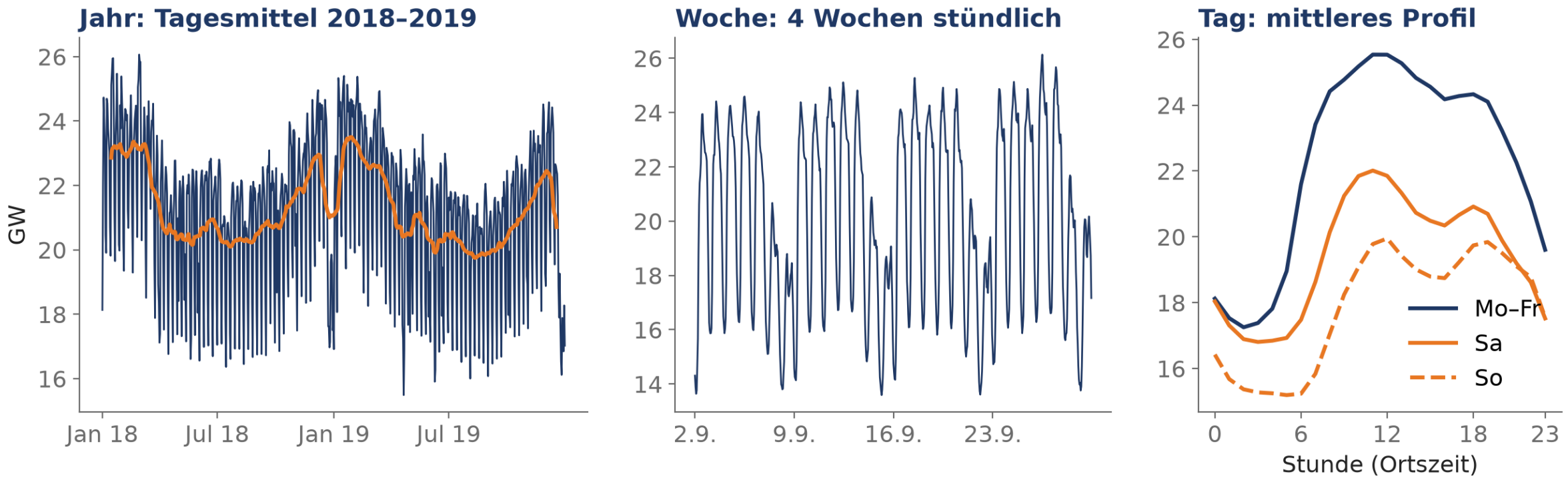
Horizont h



Anders als bei Bildern und Texten gilt: **Die Zukunft darf nie in das Training sickern**

Drei Zeitskalen in einer Kurve

Die Last schwingt gleichzeitig im Tages-, Wochen- und Jahresrhythmus



Quelle: eigene Auswertung, OPSD-Daten Regelzone Amprion · rechts: Mittel über das Training (2018-01 bis 2019-09)

Mehrfache Saisonalität: Perioden 24 h und 168 h (plus Jahresgang) ➔ für die Modellwahl entscheidend

Trend, Saison und Rest

Eine additive Zerlegung trennt die Kurve in Bausteine, die man einzeln versteht

$$y_t = T_t + S_t^{(24)} + S_t^{(168)} + R_t$$

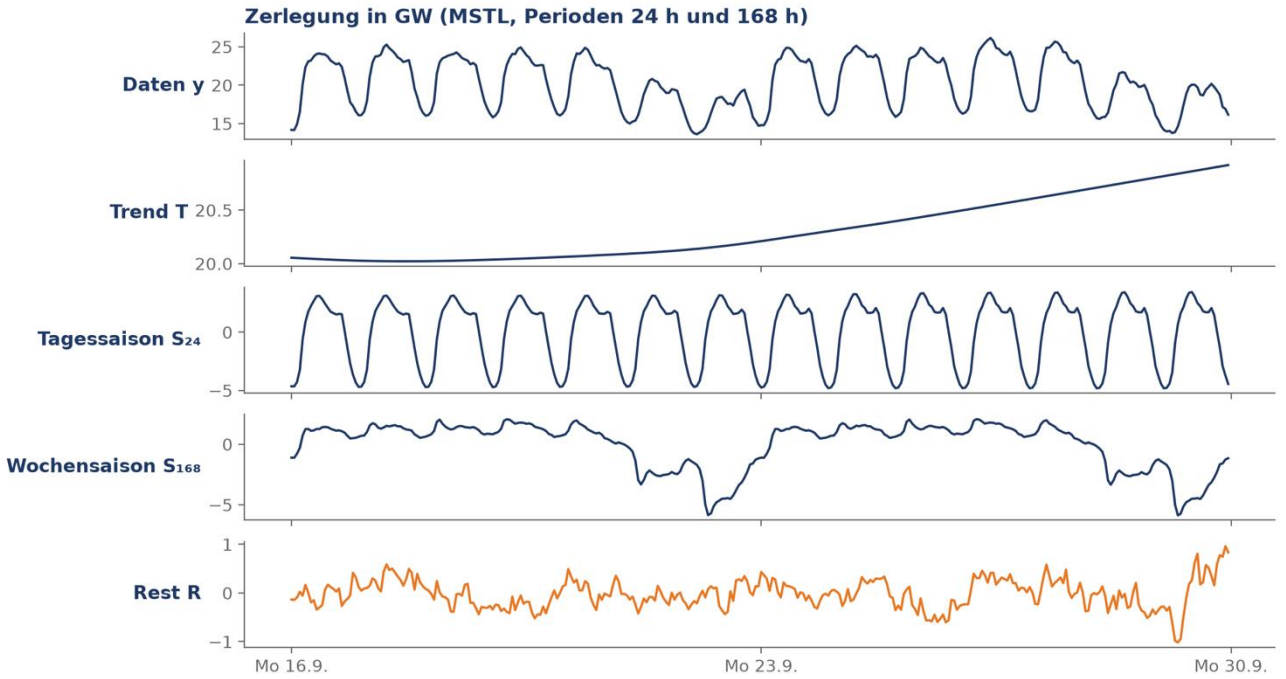
So liest man die Formel

y_t: gemessene Last zur Stunde t

T_t: Trend ➔ das langsame Niveau

$S^{(24)}$, $S^{(168)}$: Tages- und Wochenmuster, wiederholen sich

R_t: Rest ➔ was übrig bleibt, idealerweise Rauschen



Varianzanteile (8 Wochen):

Tagessaison 66 %

Wochensaison 34 %

Trend und Rest je ≈ 1 %

Beispiel: Zerlegung von Hand

Zwei Tage mit je vier Zeitblöcken: Trend als Tagesmittel, Saison als Abweichung davon

	Tag 1				Tag 2			
	Nacht	**Morgen**	**Mittag**	**Abend**	**Nacht**	**Morgen**	**Mittag**	**Abend**
y (GW)	16	22	24	20	18	24	26	22
T = Tagesmittel	20,5	20,5	20,5	20,5	22,5	22,5	22,5	22,5
S = y − T	−4,5	+1,5	+3,5	−0,5	−4,5	+1,5	+3,5	−0,5
R = y − T − S	0	0	0	0	0	0	0	0

Beobachtung

- Saison an beiden Tagen gleich: (−4,5; +1,5; +3,5; −0,5)
- Summe über eine Periode = 0 – Konvention, sonst zählt das Niveau doppelt

Selbst rechnen

Tag 3 hat das Tagesmittel **T = 21 GW**.

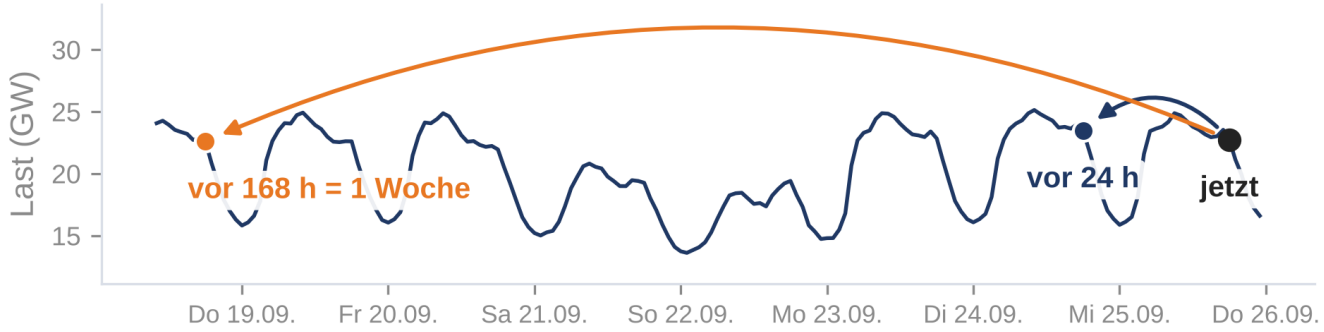
Mittags werden **26 GW** gemessen.

Wie groß ist der Rest R – und was könnte er bedeuten?

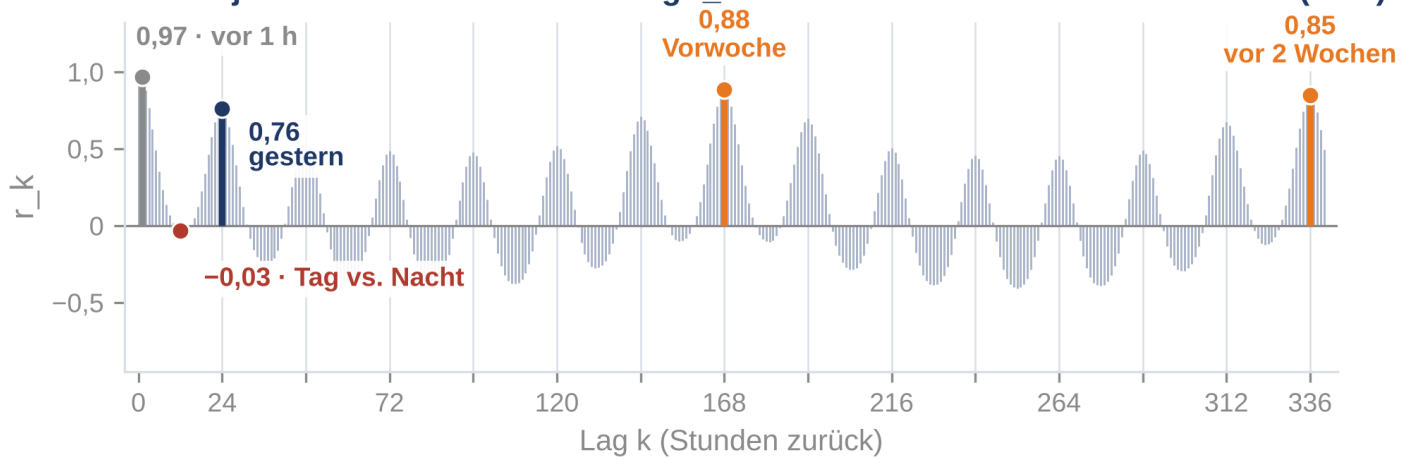
Autokorrelation

Wozu? Die ACF zeigt auf einen Blick, welche Vergangenheit sich für die Prognose lohnt

1 · Reihe gegen sich selbst verschieben: Ähnelt der Wert von früher dem von jetzt?



2 · Für jedes k die Übereinstimmung r_k messen = Autokorrelationsfunktion (ACF)



Amprion-Last, Training 2018-01 bis 2019-09 (Lab A1); Ausschnitt oben: 17.–25.09.2019

Wozu das gut ist

1 Prädiktoren finden

Hohe r_k bei 1, 24 und 168 h → y_{t-1}, y_{t-24}, y_{t-168} werden Lag-Merkmale (Abschnitt 4, LightGBM)

2 Saisonperioden ablesen

Spitzen alle 24 h, am höchsten bei 168 h → Tages- und Wochensaison für MSTL

3 Modell prüfen

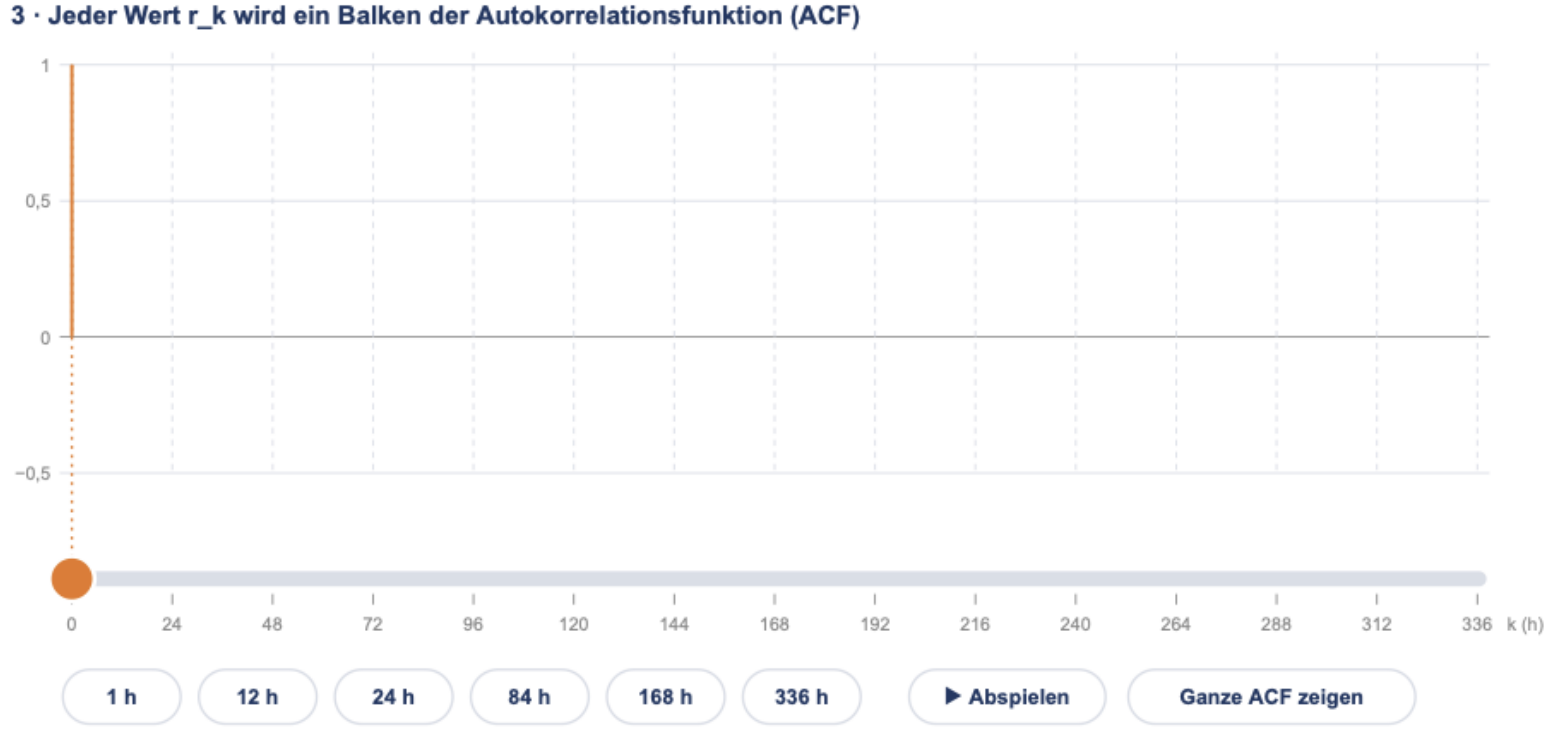
Zeigt die ACF des Rests keine Spitzen mehr, hat das Modell die Muster erfasst

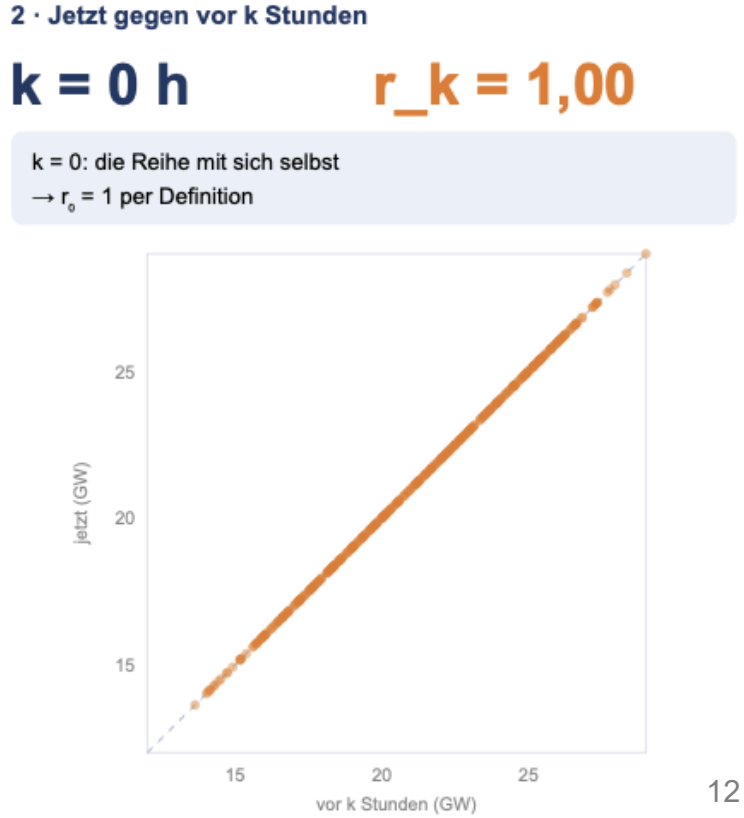
Berechnung

$$r_k = \frac{\sum_{t=k+1}^{T} (y_t - \bar{y})\,(y_{t-k} - \bar{y})}{\sum_{t=1}^{T} (y_t - \bar{y})^2}$$

Autokorrelation







Beispiel: Autokorrelation von Hand

Eine Reihe mit Periode 4 – wo liegt die Spitze der ACF?

t	1	2	3	4	5	6	7	8
y_t	2	4	6	4	2	4	6	4
d_t = y_t − 4	−2	0	2	0	−2	0	2	0

Nenner: $\Sigma d^2 = 4 + 4 + 4 + 4 = \mathbf{16}$

k = 1: jedes Produkt enthält eine 0 → Summe 0 → $r_1 = \mathbf{0}$

k = 2: $(2)(-2) + (-2)(2) + (2)(-2) = -12 \rightarrow r_2 = -12/16 = \mathbf{-0{,}75}$

gegenläufig: Berg trifft Tal – wie k = 12 bei der Last

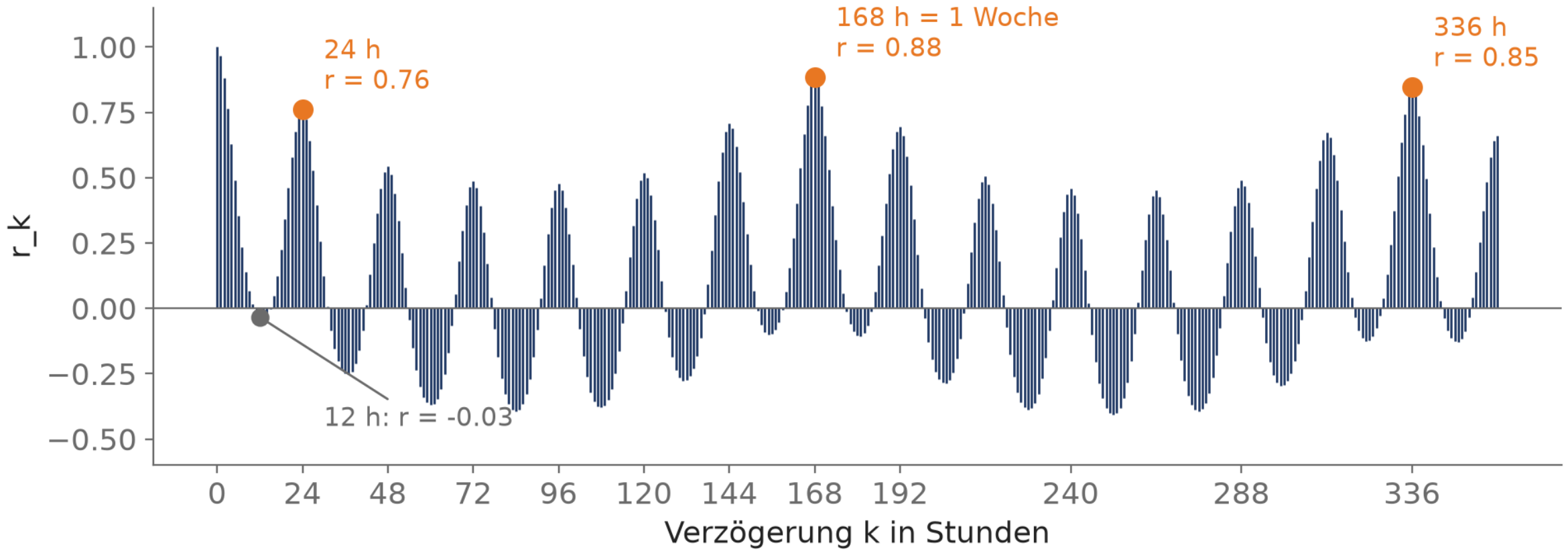
Selbst rechnen

Berechnen Sie $\mathbf{r_4}$.

Die Reihe ist exakt periodisch – warum ist r_4 trotzdem nicht 1?

Die ACF der Lastkurve

Spitzen bei 24 h und noch höher bei 168 h – die Vorwoche schlägt den Vortag



Die ACF zeigt, **welche Lags als Merkmale taugen**: 24, 48, 168, 336 – genau diese nutzt LightGBM in Abschnitt 4

Stationarität und Differenzieren

Wer Trend und Saison abzieht, erhält eine Reihe, deren Statistik sich nicht mehr ändert

$$y'_t = y_t - y_{t-1}$$

$$y_t^{(m)} = y_t - y_{t-m}$$

So liest man die Formeln

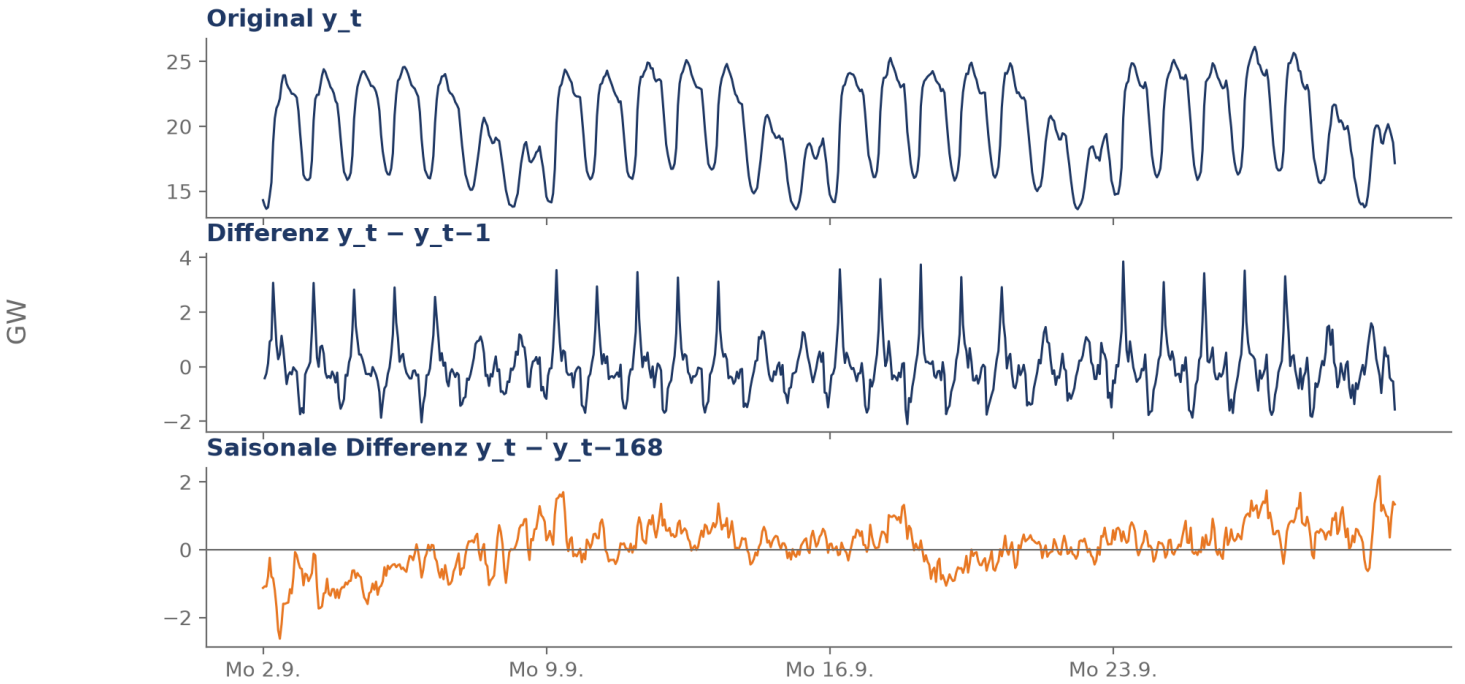
stationär: Mittel, Varianz, Autokorrelation ändern sich nicht über die Zeit

1. Differenz: entfernt linearen Trend

saisonale Differenz: entfernt Saison der Periode m (bei uns m = 168)

Rückweg: Prognose der Differenz + abgezogener Wert

saisonal-naiv = Differenz mit Prognose 0



Beispiel: Differenzieren

Zweimal differenzieren entfernt einen quadratischen Trend

linearer Trend	**3**	**5**	**7**	**9**	**11**	
1. Differenz		**2**	**2**	**2**	**2**	
quadratischer Trend	**1**	**4**	**9**	**16**	**25**	**36**
1. Differenz		**3**	**5**	**7**	**9**	**11**
2. Differenz			**2**	**2**	**2**	**2**

Rückweg: Prognose der 1. Differenz = 13 → Prognose der Reihe = 36 + 13 = **49**

Selbst rechnen

Saisonale Differenz mit **m = 4** auf die Reihe des Zerlegungsbeispiels:

16, 22, 24, 20, 18, 24, 26, 22

Was bleibt übrig – und wie viele Werte?

d-mal differenzieren entfernt polynomiale Trends bis Grad d – in der Praxis ist d fast immer 0 oder 1

Zwischenstand: Zeitreihen verstehen

Was steckt in einer Lastkurve?

1

Reihenfolge ist Information

Kalender in Ortszeit · Zukunft nie ins Training

2

Trend + zwei Saisons + Rest

MSTL: Tagessaison 66 %, Wochensaison 34 % der Varianz

3

Die ACF zeigt nützliche Vergangenheit

$r_{168} = 0{,}88 > r_{24} = 0{,}76 \rightarrow$ Vorwoche schlägt Vortag

?

Welche Perioden hätten Schwingungsdaten einer Werkzeugmaschine?

Weiter: Bevor wir modellieren, brauchen wir einen Maßstab → **Prognosen bewerten**

2

Prognosen bewerten

Leitfrage Wann ist eine Prognose gut?



Die Prognoseaufgabe präzisieren

Ursprung, Horizont und Informationsstand ➔ erst dann ist eine Prognose vergleichbar

$$\hat{y}_{T+h \mid T}$$

So liest man die Notation

T: Ursprung – letzte bekannte Stunde

h: Horizont – Stunden voraus

|T: nur Information bis T verwendet

➔ $\hat{y}_{T+h|T}$ **ist die Prognose für den Zeitpunkt T+h mit Informationen bis einschließlich T.**



- **Erlaubt:** Werte bis T · Kalender des Zieltags (Wochentag, Feiertag)
- **Verboten:** jeder Wert nach T – also kein Lag kürzer als 24 h
- Netzbetreiber gibt schon am Vormittag ab – unsere Modelle sind leicht im Vorteil

Einfache Baselines

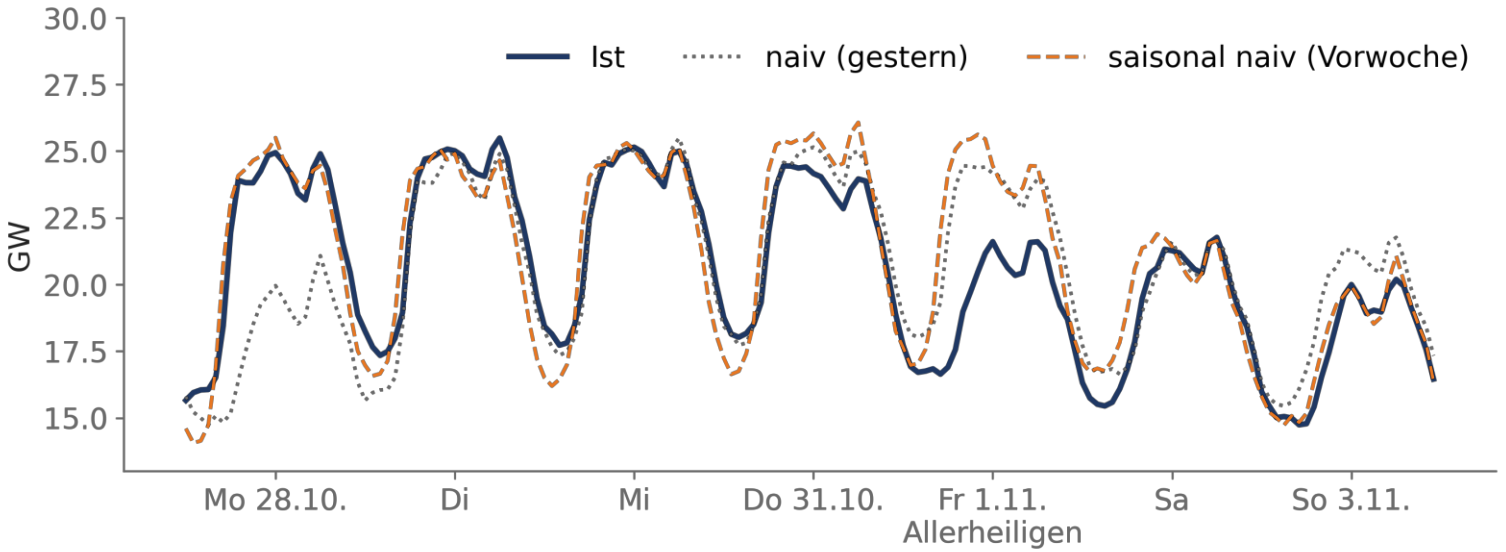
Wer eine Baseline nicht schlägt, hat nichts gelernt

naiv: $\hat{y}_t = y_{t-24}$

saisonal naiv: $\hat{y}_t = y_{t-168}$

„Naive forecasting is often a great baseline, and it can even be tricky to beat.“

Géron (2022), Kap. 15



Fr 1.11. Allerheiligen: beide Baselines liegen 3–4 GW zu hoch – ohne Kalender keine Chance

Fehlermaße

Absolut, quadratisch oder relativ ➔ jedes Maß beantwortet eine andere Frage

$$\mathrm{MAE} = \frac{1}{n}\sum_t |e_t| \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_t e_t^2} \qquad \mathrm{MAPE} = \frac{100\,\%}{n}\sum_t \left|\frac{e_t}{y_t}\right|$$

So liest man die Formel

$e_t = y_t - \hat{y}_t$: Ist minus Prognose, positiv = Prognose zu niedrig

MAE (Mean Absolute Error): typischer Fehler in MW

RMSE (Root Mean Squared Error): bestraft große Fehler stärker

MAPE (Mean Absolute Percentage Error): Fehler in % der Last

MAE

robust, in MW, leicht zu erklären
➔ unser Hauptmaß

RMSE

wenn große Fehler besonders teuer sind

MAPE

vergleichbar über Reihen
➔ Unendlich oder instabil bei y ≈ 0 (z.B. PV nachts!)

RMSE >> MAE ➔ einzelne große Ausreißer

Beispiel: Fehlermaße von Hand

Vier Stunden, zwei Prognosen – welche ist besser?

Stunde	1	2	3	4
Ist y (GW)	20	22	25	18
Prognose A	21	22	23	20
Fehler e = y − ŷ	−1	0	+2	−2

MAE = (1 + 0 + 2 + 2) / 4 = **1,25 GW**

RMSE = √((1 + 0 + 4 + 4) / 4) = √2,25 = **1,5 GW**

MAPE = (1/20 + 0 + 2/25 + 2/18) / 4 = **6,03 %**

Selbst rechnen

Prognose B: **20, 20, 25, 18**

Berechnen Sie MAE, RMSE und MAPE.

Vergleichen Sie RMSE/MAE für A und B. Was verrät das Verhältnis?

MASE: Fehler im Verhältnis zur Baseline

Unter 1 heißt: besser als die saisonal-naive Prognose im Training

$$\mathrm{MASE} = \frac{\mathrm{MAE}}{q}, \qquad q = \frac{1}{T-m}\sum_{t=m+1}^{T} |y_t - y_{t-m}|$$

So liest man die Formel

q: MAE der saisonal-naiven Prognose **im Training**

m = 168: Vorwoche, gleiche Stunde

MASE < 1: besser als dieser Maßstab · einheitenfrei, robust bei y ≈ 0

Beispiel: unsere Daten

q = **1.063 MW** (Training)

Netzbetreiber: 466 / 1.063 = **0,44**

saisonal naiv im Test: 927 / 1.063 = **0,87**

Selbst rechnen

Ein Modell erreicht MAE = 800 MW. MASE? Besser als der Netzbetreiber?

Kein Zufallssplit bei Zeitreihen

Wer zufällig mischt, prüft das Modell mit Wissen aus der Zukunft

Zufallssplit



✗

zeitlicher Split



✓

navy = Training · orange = Test · jede Kachel eine Stunde

Zeitraum	Daten	Zweck
Training	1.1.2018 – 30.9.2019 · 15.312 h	Modelle anpassen (TFT: Sep. 2019 als Validierung)
Backtest	7.10. – 3.11.2019 · 28 Tage	Modelle vergleichen – Day-ahead, 28 Ursprünge
Drift	1.1. – 30.6.2020	Abschnitt 5: Residuen im Lockdown

Training **immer vor** dem Test ➔ Test wird erst ganz am Ende **genau einmal** angefasst

Backtesting mit rollierendem Ursprung

Ein Test reicht nicht ➔ jeder Tag wird einmal mit dem damaligen Wissen vorhergesagt



navy = bekannt bis Vorabend · orange = prognostizierter Tag (24 h)

Unser Backtest

- 28 Ursprünge = 672 Prognosestunden
- Statistik: an jedem Ursprung neu geschätzt, Fenster 4 Wochen
- ML/DL: einmal trainiert, an jedem Ursprung angewandt
- Kennzahl = Mittel über alle 672 Stunden

Géron: SARIMA an **einem** Tag 12,9 % daneben (schlechter als naiv) ➔ über drei Monate MAE 32.041 statt 42.143 (naiv)

Die Messlatte

Backtest 7.10.–3.11.2019: Die Vorwoche halbiert fast den Fehler, der Netzbetreiber halbiert ihn noch einmal

Prognose	MAE (MW)	RMSE (MW)	MAPE	MASE
Naiv (Vortag)	1.562	2.350	7,64 %	1,47
Saisonal naiv (Vorwoche)	927	1.467	4,43 %	0,87
Netzbetreiber (Day-ahead)	466	574	2,29 %	0,44

Beobachtungen

- Vorwoche schlägt Vortag deutlich ➔ wie die ACF es vorhergesagt hat
- RMSE/MAE ≈ 1,5 bei den Baselines: einzelne große Fehler (Montage, Allerheiligen)
- Netzbetreiber: Wetter, Kalender, Erfahrung ➔ und gleichmäßige Fehler (RMSE/MAE = 1,23)

Ziel für alle Modelle heute: **unter 927 MW** (saisonal naiv) ➔ der Netzbetreiber mit **466 MW** ist die Kür

Zwischenstand: Prognosen bewerten

1

Gut = besser als die Baseline

unter gleichen Bedingungen: Ursprung, Horizont, Informationsstand

2

Fehlermaß passend wählen

MAE in MW · RMSE für große Fehler · MAPE und MASE zum Vergleichen

3

Zeitlich splitten, rollierend testen

28 Ursprünge · Test genau einmal · keine Lags < 24 h

?

Ein Team meldet MAPE 1 %. Welche drei Fragen stellen Sie zuerst?

Weiter: Wie weit kommt man mit klassischer Statistik? → **ETS und ARIMA**

3

Klassische Modelle: ETS und ARIMA

Leitfrage Wie weit kommt man mit Statistik?



Einfache exponentielle Glättung

Die Prognose ist ein gewichteter Mittelwert der Vergangenheit mit exponentiell fallenden Gewichten

$$\ell_t = \alpha\, y_t + (1-\alpha)\,\ell_{t-1}, \qquad \hat{y}_{T+h\,|\,T} = \ell_T$$

So liest man die Formel

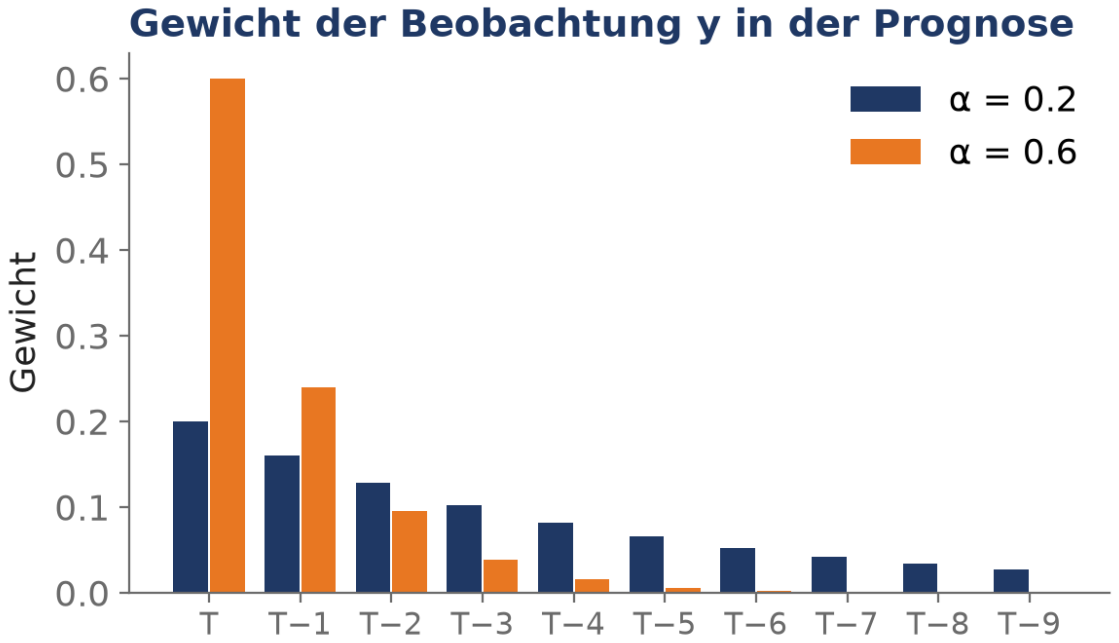
ℓ_t: geglättetes Niveau zum Zeitpunkt t

$\alpha \in (0, 1]$: wie stark die neue Beobachtung zählt

$(1-\alpha)\,\ell_{t-1}$: was das Modell bisher glaubte

ℓ_T: Prognose für alle Horizonte h – flach!

α groß → vergisst schnell · α klein → glättet stark



FPP3 Abschnitt 8.1 · Gewichte α(1 − α)^j, Summe = 1

Beispiel: Glätten von Hand

Vier Stunden, $\alpha = 0{,}5$, Startniveau $\ell_0 = 20$ GW

t	1	2	3	4
y_t (GW)	20	22	25	18
$0{,}5 \cdot y_t$	10,0	11,0	12,5	9,0
$0{,}5 \cdot \ell_{t-1}$	10,0	10,0	10,5	11,5
ℓ_t	20,0	21,0	23,0	20,5

Prognose für alle folgenden Stunden: $\hat{y} = \ell_4 = 20{,}5$ **GW**

- Das Niveau folgt mit Verzögerung: Spitze 25 → 23, Einbruch 18 → 20,5
- Eine Tagesform kann dieses Modell nicht abbilden

Selbst rechnen

Wiederholen Sie die Rechnung mit **$\alpha = 0{,}8$**.

Wie lautet die Prognose?

Liegt sie näher an $y_4 = 18$ oder am Mittelwert 21,25 – warum?

Die ETS-Familie

Error, Trend, Season: Jede Komponente kann fehlen, additiv oder multiplikativ sein

Modell	ETS-Kürzel	Komponenten	Prognoseform
Einfache Glättung	ETS(A,N,N)	Niveau	flach
Holt	ETS(A,A,N)	Niveau + Trend	Gerade
Holt, gedämpft	ETS(A,Ad,N)	Niveau + gedämpfter Trend	flacht ab
Holt-Winters additiv	ETS(A,A,A)	Niveau + Trend + Saison m	Gerade + Muster
nur Saison	ETS(A,N,A)	Niveau + Saison m	Muster, Periode m

ETS(E,T,S) = Error, Trend, Season · **N** = keine (None) · **A** = additiv · **Ad** = additiv gedämpft (damped) · **M** = multiplikativ

Vorsicht Automatik

AutoETS wählt per AICc (Corrected Akaike Information Criterion)

➔ bei m = 24 und vier Wochen Daten oft **ohne Saison** → flache Prognose, **MAE 4.778 MW**

Das Kriterium ist erfüllt, das Ziel verfehlt: **Saison fest vorgeben** und das Ergebnis immer plotten

Autoregression und gleitender Mittelwert

AR (Autoregressives Modell) rechnet mit vergangenen Werten, MA (Moving Average) mit vergangenen Fehlern

$$\mathrm{AR}(p): \quad y_t = c + \phi_1 y_{t-1} + \cdots + \phi_p\, y_{t-p} + \varepsilon_t$$

$$\mathrm{MA}(q): \quad y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q\, \varepsilon_{t-q}$$

So liest man die Formel

φ_i: Gewicht des Werts vor i Stunden (Regressionskoeffizient)

ε_t: Fehler (weißes Rauschen), nicht vorhersagbar

θ_i: Gewicht des Fehlers vor i Stunden (Korrektur)

p, q: Ordnungen = wie weit zurück

Voraussetzung: stationäre Reihe (AR(1): $|\varphi_1| < 1$)

AR

„Morgen ähnelt heute“ ➔ Regression auf Lags

MA

„Gestern lag ich daneben ➔ korrigieren“

Beispiel: Eine AR(1)-Prognose

c = 2, φ_1 = 0,8, letzter Wert y_T = 20 ➔ wohin läuft die Prognose?

Schritt h	1	2	3	4	∞
$\hat{y}_{T+h}$	18,0	16,4	15,12	14,10	10
Abstand zu μ	8,0	6,4	5,12	4,10	0

$$\mu = \frac{c}{1-\phi_1} = \frac{2}{0{,}2} = 10$$

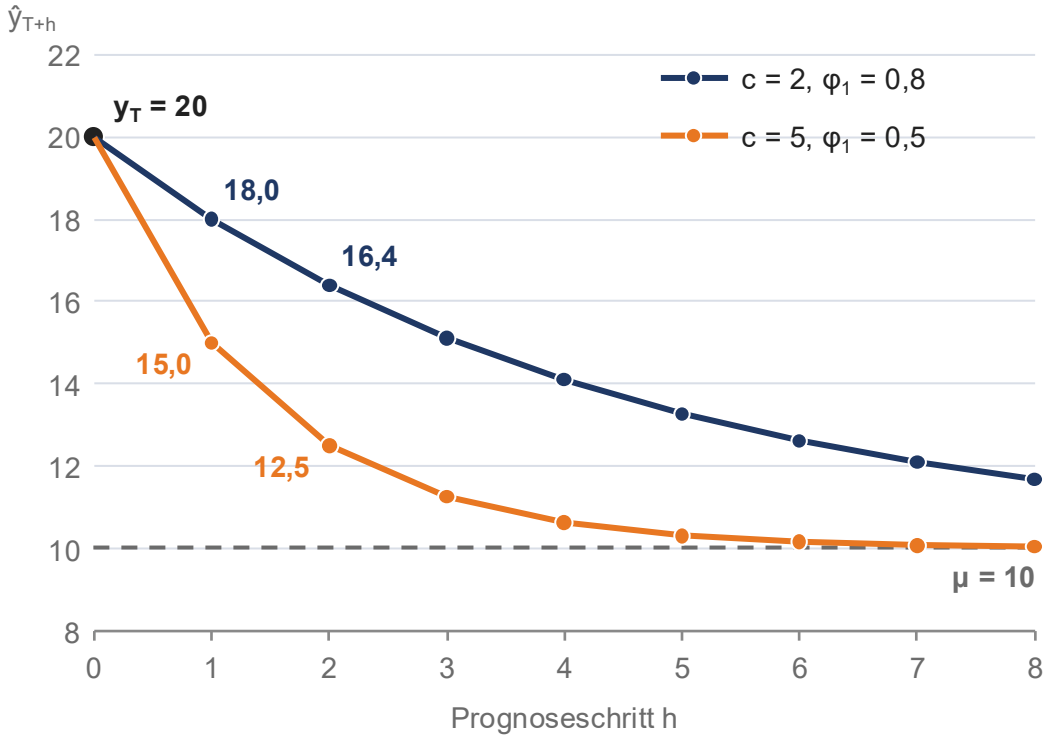
- Unbekannte Zukunft durch Prognose ersetzen, Fehler durch 0
- Abstand zu μ schrumpft je Schritt um den Faktor φ_1 = 0,8
- Ohne Saisonterme landet jede AR-Prognose beim Mittelwert

Selbst rechnen

c = 5, φ_1 = 0,5, y_T = 20

Berechnen Sie $\hat{y}_{T+1}$, $\hat{y}_{T+2}$ und den Grenzwert μ.

Was ändert ein kleineres φ_1?



ARIMA und SARIMA

Erst differenzieren, dann ARMA ➔ für Muster mit Periode m dasselbe noch einmal saisonal

$$\mathrm{SARIMA}(p,d,q)(P,D,Q)_m$$

So liest man die Formel

p, d, q: AR-Ordnung, Differenzen, MA-Ordnung

P, D, Q: dasselbe für Lags t − m, t − 2m, …

m: Periode der Saison (bei uns 24)

Wahl: kleinstes **AICc** (AutoARIMA)

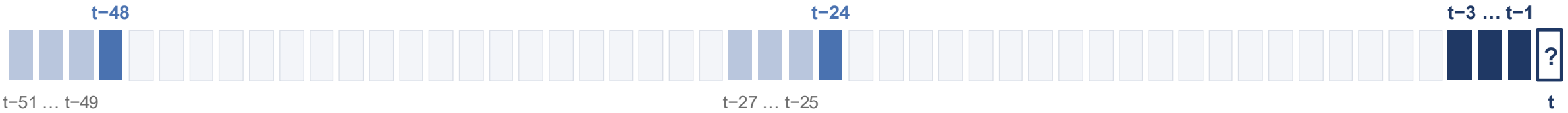
typisch: p, q, P, Q ∈ {0, 1, 2}; d, D ∈ {0, 1}

Unser Modell in Lab A3

SARIMA(3,0,0)(2,1,0)$_{24}$

- D = 1: Differenz y_t − y_{t−24}
- p = 3: letzte drei Stunden
- P = 2: gleiche Stunde vor 24 und 48 h
- Ordnung einmal per AutoARIMA bestimmt, dann fest
- Was fehlt? → die **Wochensaison** (m = 168)

Welche Stunden fließen in die Prognose ein? Modell auf der saisonal differenzierten Reihe $z_t = y_t - y_{t-24}$



Zurück auf das Original: $\hat{y}_t = y_{t-24} + \hat{z}_t$ · **die Wochensaison (t − 168) liegt weit außerhalb dieses Fensters**

SARIMA in drei Schritten

Tagesmuster abziehen, die Abweichung vorhersagen, Tagesmuster wieder addieren

1 Tagesmuster abziehen

Wie viel mehr oder weniger Last als gestern zur selben Stunde?

$$z_t = y_t - y_{t-24}$$

In der Formel: D = 1, m = 24

2 Abweichung vorhersagen

aus den letzten drei Stunden und derselben Stunde gestern und vorgestern

$$\hat{z}_t = \text{Teil A} + \text{Teil B}$$

In der Formel: p = 3, P = 2

3 Tagesmuster wieder addieren

Die vorhergesagte Abweichung kommt auf den Wert von gestern drauf

$$\hat{y}_t = y_{t-24} + \hat{z}_t$$

Zurück auf die Originalreihe

Beispiel: Mittwoch, 09.10.2019, 12 Uhr · Regelzone Amprion

Gestern 12 Uhr: y_{t-24} = **25.342 MW**

Erwartete Abweichung: $\hat{z}_t$ = **−314 MW**

Prognose **25.028 MW** · Ist 25.306 MW

Die Formel lesen: zwei Klammern, zwei Takte

Die erste Klammer schaut auf die Nachbarstunden, die zweite auf dieselbe Stunde an früheren Tagen

$$\text{SARIMA}(3, 0, 0)(2, 1, 0)_{24}$$

Erste Klammer (3, 0, 0) · Stundentakt

Abstand 1 Stunde

p = 3 — **AR-Ordnung:** die letzten drei Stunden t−1, t−2, t−3 gehen ein

d = 0 — **Differenzen:** keine Differenz zur Vorstunde

q = 0 — **MA-Ordnung:** keine vergangenen Prognosefehler

Zweite Klammer (2, 1, 0) · Tagestakt

Abstand m = 24 Stunden

P = 2 — **Saisonale AR-Ordnung:** dieselbe Stunde gestern (t−24) und vorgestern (t−48)

D = 1 — **Saisonale Differenz:** zuerst $y_t - y_{t-24}$ bilden (Schritt 1)

Q = 0 — **Saisonale MA-Ordnung:** keine saisonalen Fehlerterme

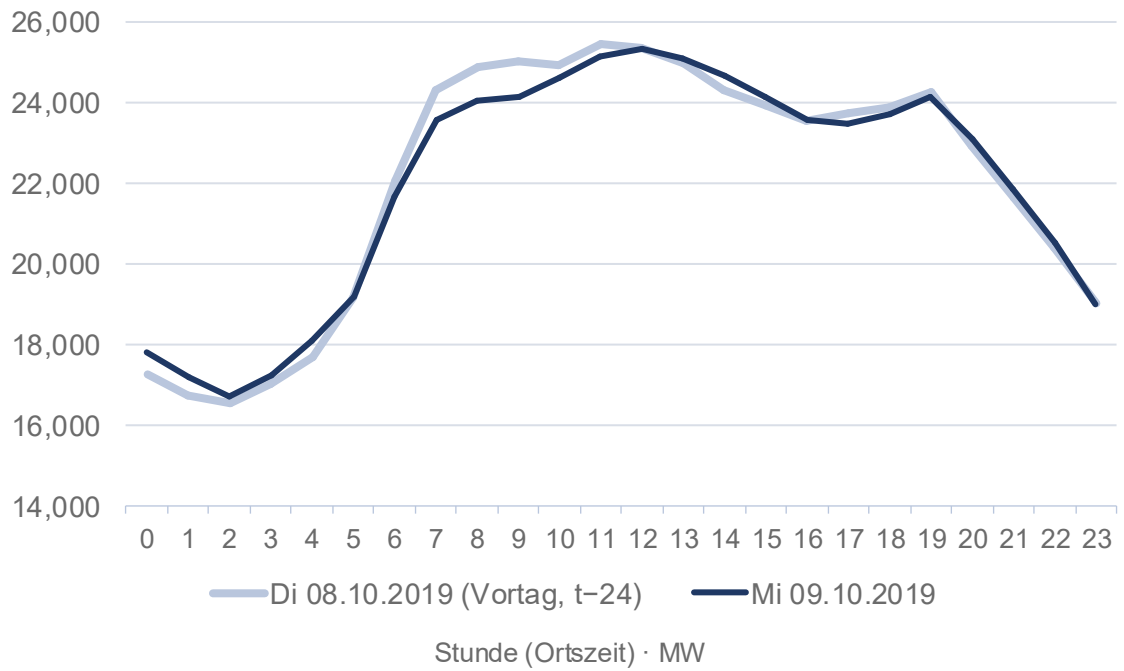
m = 24 — **Periode der Saison:** 24 Stunden = ein Tag. SARIMA kennt genau eine Periode ➔ die Woche (168 h) fehlt.

Woher kommen die Zahlen? AutoARIMA probiert Kombinationen (meist 0 – 2, Differenzen 0 – 1) und nimmt die mit dem kleinsten AICc. Einmal bestimmt, dann für alle 28 Backtest-Tage fest.

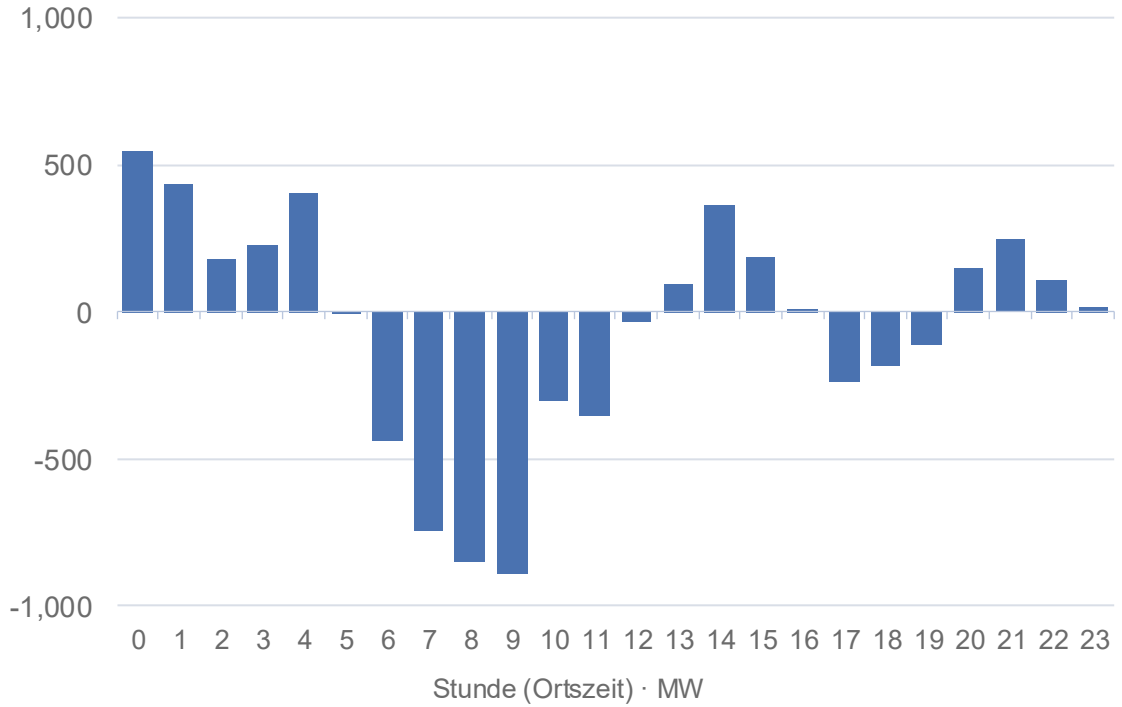
Schritt 1: Saisonal differenzieren

D = 1 mit m = 24: Jede Stunde wird mit derselben Stunde am Vortag verglichen

Last y_t: Dienstag und Mittwoch fast deckungsgleich



Differenz $z_t = y_t - y_{t-24}$: Mittwoch minus Dienstag



8.605 MW Spanne der Last am Mittwoch (16.701 bis 25.306 MW)

296 MW mittlerer Betrag der Differenz (−890 bis +547 MW)

Das Tagesmuster ist herausgerechnet. Übrig bleibt eine kleine Reihe ohne Tagesform ➔ die modelliert Schritt 2.

Schritt 2: Die Abweichung vorhersagen

Zwei Teile: die letzten drei Stunden und was gestern und vorgestern zur selben Stunde unerwartet war

$$\hat{z}_t = \varphi_1 z_{t-1} + \varphi_2 z_{t-2} + \varphi_3 z_{t-3} + \Phi_1 u_{t-24} + \Phi_2 u_{t-48}$$

Teil A · Stundentakt (p = 3)

Wie hat sich die Abweichung z in den letzten drei Stunden entwickelt?

$\varphi_1 = 1{,}554$ | $\varphi_2 = -0{,}746$ | $\varphi_3 = 0{,}164$

Lesart: Die letzte Stunde zählt am meisten. Zusammen schreiben die drei Gewichte die aktuelle Richtung der Abweichung fort.

Das ist ein gewöhnliches AR(3) – wie AR(1), nur mit drei Lags.

Teil B · Tagestakt (P = 2)

Was war gestern und vorgestern zur selben Stunde **unerwartet**? Diese Überraschung heißt u:

$$u_s = z_s - (\varphi_1 z_{s-1} + \varphi_2 z_{s-2} + \varphi_3 z_{s-3})$$

= Abweichung minus das, was Teil A damals vorhergesagt hätte

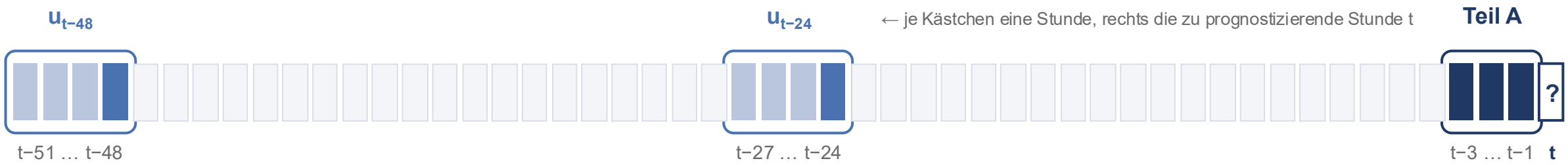
$\Phi_1 = -0{,}449$ | $\Phi_2 = -0{,}259$

Lesart: Negativ – eine Überraschung von gestern wird heute zum Teil wieder zurückgenommen.

Gewichte geschätzt am ersten Backtest-Ursprung (4 Wochen bis 06.10.2019): 5 Parameter, keine Konstante.
Lehrbuchform: $(1 - \varphi_1 B - \varphi_2 B^2 - \varphi_3 B^3)(1 - \Phi_1 B^{24} - \Phi_2 B^{48})\, z_t = \varepsilon_t$, mit B = „eine Stunde zurück“ – ausmultipliziert ergibt sie genau die Gleichung oben.

Welche Stunden fließen in die Prognose ein?

Aus 5 Gewichten werden 11 Stunden, denn jede Überraschung u braucht die drei Stunden davor



Vorgestern, gleiche Stunde

Aus t−48 und den drei Stunden davor entsteht die Überraschung u_{t-48}.

Gewicht $\Phi_2 = -0{,}259$

Gestern, gleiche Stunde

Aus t−24 und den drei Stunden davor entsteht die Überraschung u_{t-24}.

Gewicht $\Phi_1 = -0{,}449$

Die letzten drei Stunden

t−1, t−2, t−3 gehen direkt als Teil A ein.

Gewichte φ_1, φ_2, φ_3

Fachbegriff: Die hellen Felder heißen in der Literatur „Kreuzterme“ (Gewicht $-\varphi_i \cdot \Phi_j$, weil SARIMA multiplikativ ist). Gemeint ist genau das: die Stunden vor t−24 und t−48, die man für die Überraschung u braucht.

Die Wochensaison (t−168) liegt weit außerhalb dieses Fensters.

Durchgerechnet: Mittwoch, 09.10.2019, 12 Uhr

Ein-Schritt-Prognose mit den geschätzten Gewichten · Regelzone Amprion, alle Werte in MW

Schritt	Rechnung	MW
① **Gestern, 12 Uhr**	y_{t-24}	**25 342**
② **Teil A**	1,554·(−352) − 0,746·(−299) + 0,164·(−890)	**−470**
Überraschung gestern	$u_{t-24} = z_{t-24}$ − Teil A damals = 595 − 990	**−395**
Überraschung vorgestern	$u_{t-48} = z_{t-48}$ − Teil A damals = 4.502 − 4.421	**81**
② **Teil B**	−0,449·(−395) − 0,259·81	**+156**
② **Abweichung $\hat{z}_t$**	Teil A + Teil B = −470 + 156	**−314**
③ **Prognose $\hat{y}_t$**	**$y_{t-24} + \hat{z}_t$ = 25.342 − 314**	**25.028**
Ist y_t	Fehler 278 MW = 1,1 %	**25.306**

Die z-Werte dazu

$z_{t-1\ldots t-3}$: −352, −299, −890

z_{t-24}: 595 (Di − Mo)

z_{t-48}: 4 502 (Mo − So!)

Day-ahead ist schwerer

Um Mitternacht sind t−1 … t−3 für 12 Uhr noch unbekannt. Das Modell setzt eigene Prognosen ein – wie bei AR(1).

Backtest-Prognose für diese Stunde: **24.536 MW**, Fehler **770 MW**.

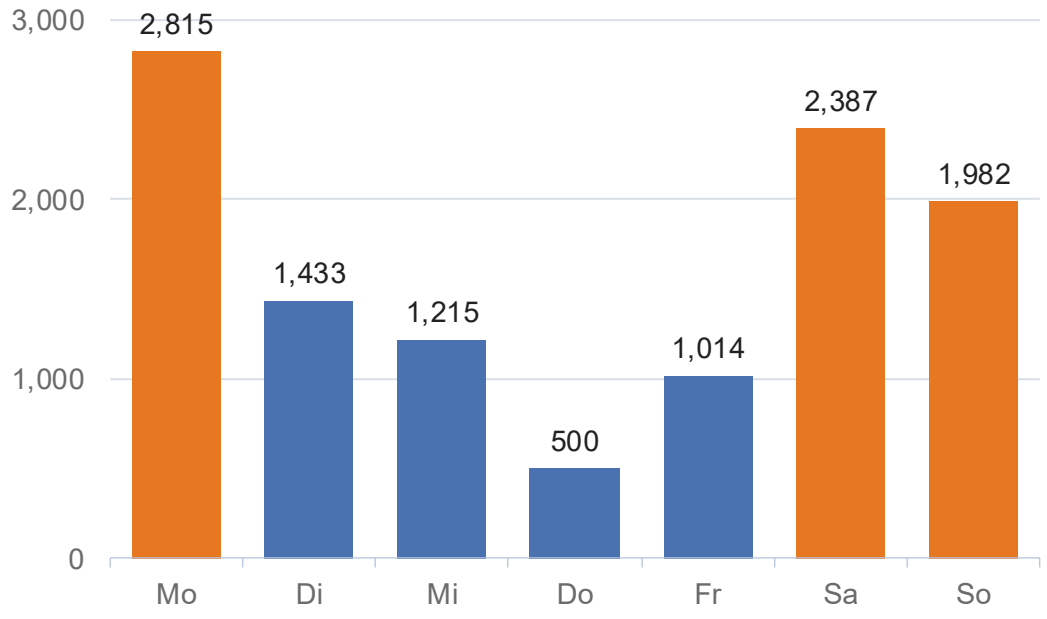
Gerechnet mit ungerundeten Gewichten und z-Werten; Teil A damals = φ-Summe über die drei Stunden vor t−24 bzw. t−48.

Selbst rechnen: Wie ändert sich $\hat{y}_t$, wenn gestern um 12 Uhr eine Überraschung von u_{t-24} = +1.000 MW aufgetreten wäre?

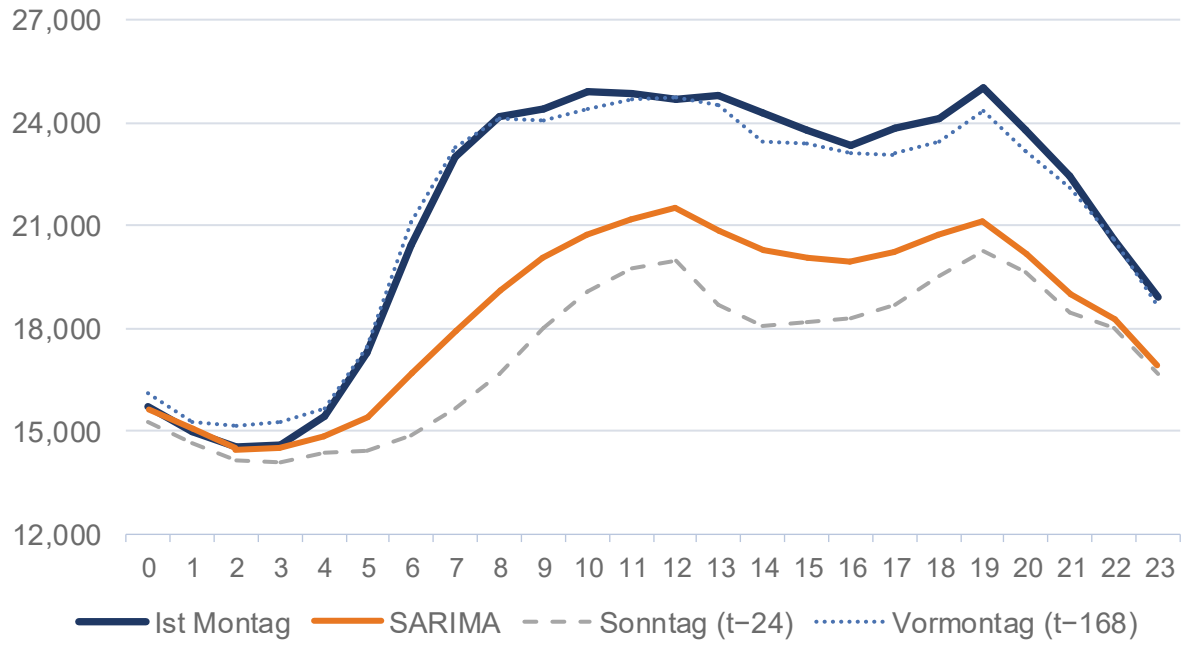
Was fehlt: die Wochensaison

m = 24 kennt nur „gestern“ – nach dem Wochenende ist gestern das falsche Vorbild

Backtest-MAE je Wochentag (28 Tage), MW



Montag, 14.10.2019: Prognose um Mitternacht



Mo 12 Uhr Ist 24.654 · SARIMA 21.521 (−3.133) · Vormontag 24.747 (+93)

Gesamt SARIMA 1.620 MW MAE ➔ die naive Vorwoche schafft 927 MW

Ausweg: zwei Saisons gleichzeitig (24 h und 168 h) → MSTL, nächste Folie. m = 168 in SARIMA wäre möglich, ist aber langsam und instabil.

Zwei Saisons gleichzeitig: MSTL

Erst beide Saisons abziehen, dann den Rest mit ETS prognostizieren

1 Zerlegen

MSTL(24, 168) auf 8 Wochen:

Trend + S_{24} + S_{168} + R

→

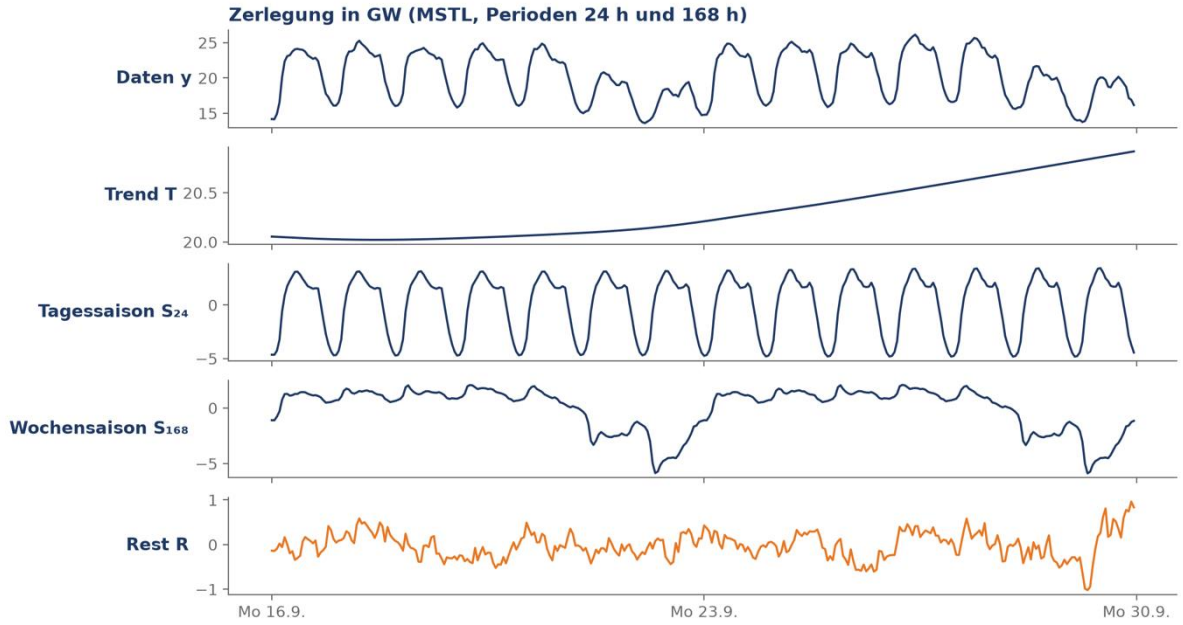
2 Rest prognostizieren

Trend + Rest mit ETS ohne Saison fortschreiben

→

3 Saisons addieren

letzte Tages- und Wochenperiode für die Zukunft wiederholen



Prinzip wie bei ARIMA:

bekannte Struktur entfernen, den Rest modellieren, Struktur wieder addieren

Alternative: Fourier-Terme je Periode (harmonische Regression, Prophet)

Prophet: Zeitreihe als Regression

Trend + Saison + Feiertage + Kovariaten ➔ ohne die eigene Vergangenheit

$$y(t) = g(t) + s(t) + h(t) + \mathbf{w}^\top \mathbf{x}(t) + \varepsilon_t$$

So liest man die Formel

g(t): stückweise linearer Trend mit Knickpunkten

s(t): Saisons als Sinus-/Kosinus-Regression, mehrere Perioden

h(t): Feiertagseffekte ➔ Liste vorgeben

$\mathbf{w}^\top\mathbf{x}(t)$: Kovariaten, z. B. Temperatur

kein y(t−1) im Modell: Regression auf die Zeit

Stärken

mehrere Saisons

Feiertage bequem

gut erklärbar

Schwäche

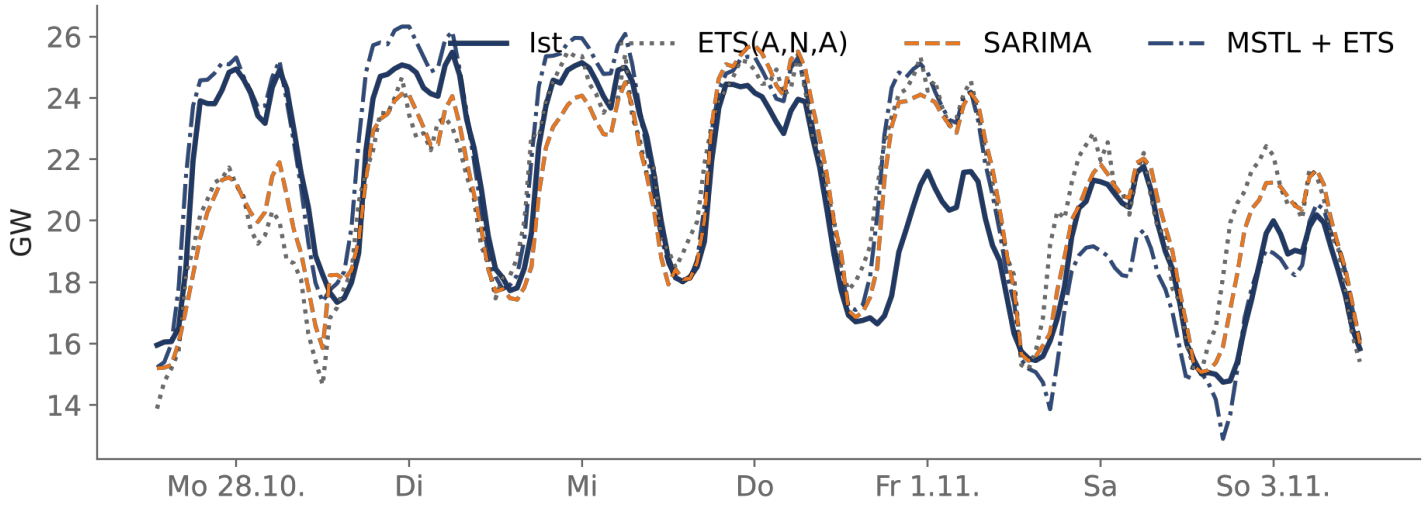
nutzt nicht, was gestern anders war (keine Lags)

bei Stundenlast meist schwächer als Lag-Modelle → Abschnitt 4

Murphy (2023), PML Advanced Topics, Gl. 29.162 · Taylor & Letham (2018), Forecasting at Scale

Ergebnis: Statistik im Backtest

Nicht die Modellklasse entscheidet, sondern ob die Wochensaison im Modell steckt



Modell	MAE (MW)	MASE
Saisonal naiv	927	0,87
ETS(A,N,A)	1.758	1,65
SARIMA	1.620	1,52
MSTL + ETS	673	0,63
AutoETS (Automatik)	4.778	4,49

- ETS und SARIMA kopieren im Kern den Vortag: montags Sonntagsniveau, samstags Freitagsniveau
- MSTL trifft Montag und Wochenende – aber nicht Allerheiligen (Tages-MAE 2.762 MW)
- Backtest 7.10.–3.11.2019 · 28 Ursprünge · Fenster 4 Wochen (MSTL 8 Wochen)

MSTL mit zwei Saisons (673 MW) schlägt die Vorwoche (927 MW) – ETS und SARIMA mit nur m = 24 nicht einmal den Vortag

Zwischenstand: Klassische Modelle

Wie weit kommt man mit Statistik?

1

ETS glättet, ARIMA regressiert

Niveau/Trend/Saison fortschreiben · Lags und Fehler nach Differenzieren

2

Eine Saisonperiode reicht nicht

ETS 1.758 MW, SARIMA 1.620 MW ➔ kaum besser als der Vortag

3

Struktur schlägt Modellklasse

MSTL mit 24 h und 168 h: 673 MW < 927 MW (Vorwoche)

?

Was müsste ein Modell wissen, um Allerheiligen zu treffen?

→ wie h(t) bei Prophet: Feiertage als bekannter Input – kommt bei LightGBM und TFT wieder

Weiter: Was passiert, wenn wir die Zeitreihe als Tabelle behandeln? → **ML & Deep Learning**

4

ML und Deep Learning für Zeitreihen

Leitfrage Was bringen Lags, LSTM und Transformer?



Die Zeitreihe als Tabelle

Jede Stunde wird eine Zeile, die Vergangenheit steht in Spalten daneben

Zielstunde t	Stunde	Wochentag	Feiertag	y_{t-24}	y_{t-168}	Mittel Vortag	…	y_t (Ziel)
Fr 1.11. 08:00	8	Fr	1	24,4	25,4	22,5	…	19,0
Fr 1.11. 12:00	12	Fr	1	24,2	24,5	22,4	…	21,6
Fr 1.11. 18:00	18	Fr	1	24,0	24,4	22,2	…	21,6

Merkmale (9)

- Lags 24, 48, 168, 336 h (aus der ACF)
- Mittel der 24 h bis t − 24 (Niveau des Vortags)
- Kalender in Ortszeit: Stunde, Wochentag, Monat, Feiertag

Neu gegenüber Statistik

Feiertag = 1 steht als Spalte im Modell ➔ Allerheiligen ist kein Rätsel mehr

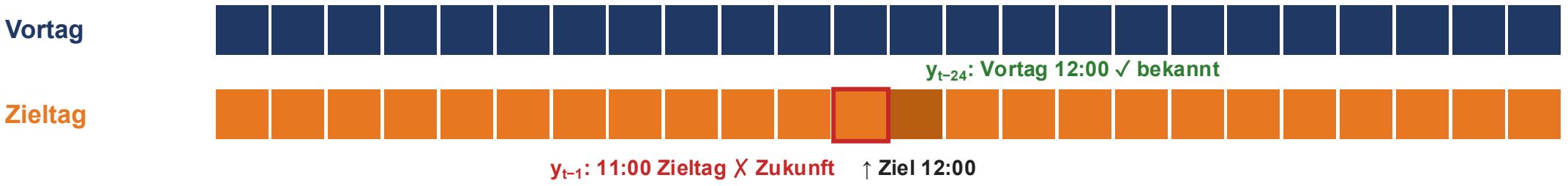
Prophet: Regression auf die Zeit ➔ hier: auf die eigene Vergangenheit (Lags)

≈ 15.000 Trainingszeilen · jede Zeile unabhängig lernbar

Lastwerte in GW, Ortszeit · Ziel in der Prognose unbekannt, hier zur Kontrolle gezeigt · Géron (2022), Kap. 15

Die Leckage-Regel

Jeder Lag muss mindestens so groß sein wie der Horizont, sonst lernt das Modell aus der Zukunft



Merkmal	zulässig?	Grund
y_{t-24}, y_{t-168}	ja	spätestens 23:00 des Vortags bekannt
Mittel von y_{t-24} … y_{t-47}	ja	auf y.shift(24) berechnet
y_{t-1}, y_{t-12}	nein	Stunden des Zieltags – zum Ursprung unbekannt
Feiertag, Wochentag	ja	Kalender ist im Voraus bekannt

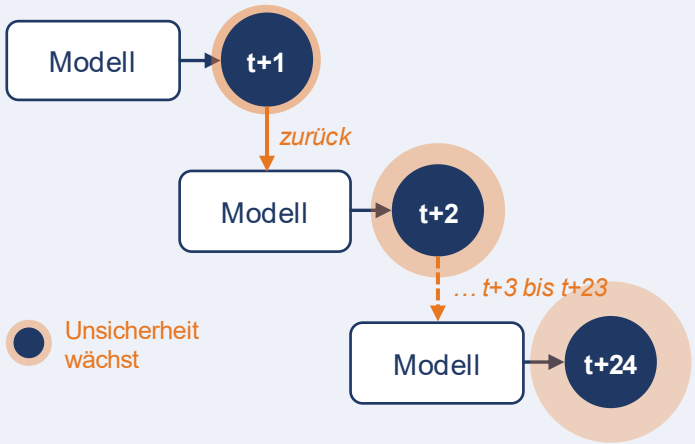
Day-ahead mit Ursprung 23:00 → **alle Lags ≥ 24 h** · dann ist jede Zeile eine echte Prognose, ohne Rekursion

Mehrere Schritte voraus

Rekursiv, direkt oder alles auf einmal ➔ drei Wege zu 24 Stunden Prognose

Rekursiv

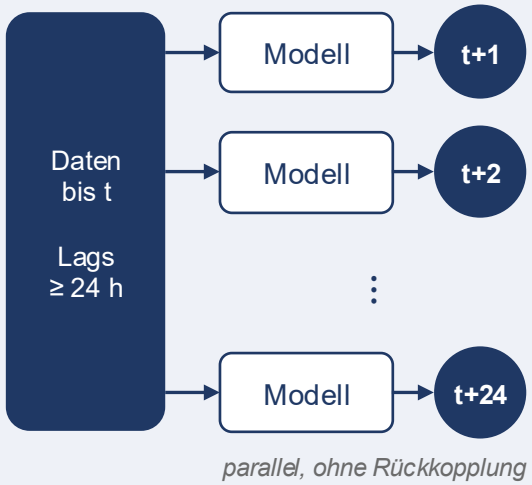
Ein-Schritt-Modell
Prognose zurückfüttern
Fehler pflanzen sich fort



bei uns: **ETS, ARIMA**

Direkt

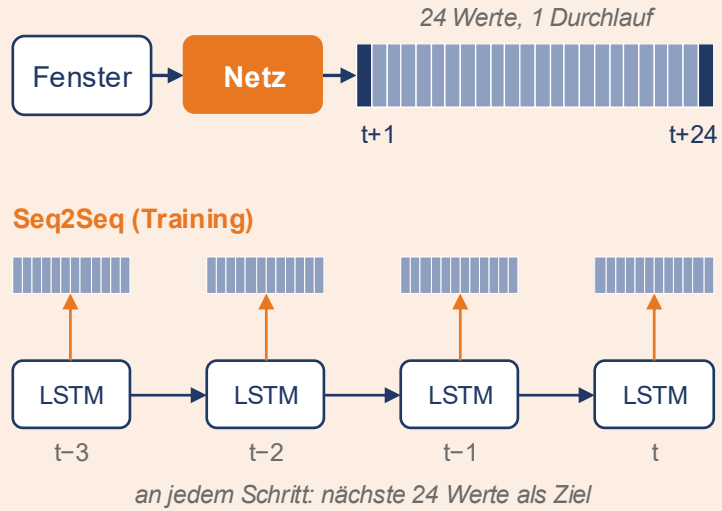
Lags ≥ Horizont
jede Stunde unabhängig
kein Zurückfüttern



bei uns: **LightGBM (A4)**

Alles auf einmal

Netz gibt Vektor aus
24 Werte je Durchlauf
Seq2Seq: an jedem Schritt



bei uns: **LSTM, TFT (A4, A5)**

Géron (2022), Kap. 15: „Forecasting Several Time Steps Ahead“, „Forecasting Using a Sequence-to-Sequence Model“

Gradient Boosting: die Idee

Viele kleine Bäume, jeder korrigiert den Rest der vorherigen

$$F_k(\mathbf{x}) = F_{k-1}(\mathbf{x}) + \eta\, h_k(\mathbf{x})$$

So liest man die Formel

F_k: Prognose nach k Bäumen (F_0 = Mittelwert)

h_k: kleiner Baum, trainiert auf die Residuen $y - F_{k-1}$

η: Lernrate ➔ jeder Baum korrigiert nur einen Teil

Ein Durchlauf pro Baum

1 Rest berechnen
$r = y - F_{k-1}$
Was fehlt der Prognose noch?

→

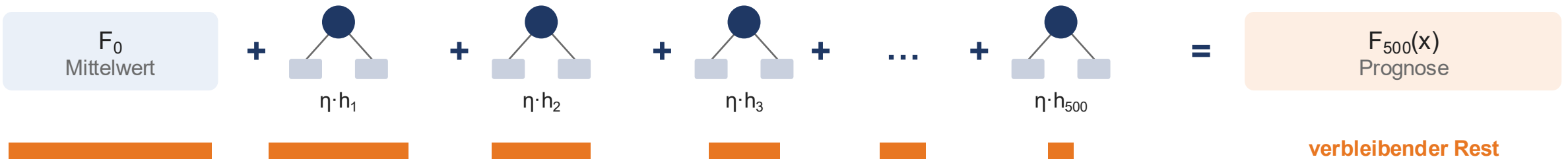
2 Kleinen Baum anpassen
h_k lernt r
wenige Fragen, wenige Blätter

→

3 Gedämpft addieren
$F_k = F_{k-1} + \eta \cdot h_k$
nur ein Teil der Korrektur

↺ nächster Baum: $k \rightarrow k + 1$ · bei uns 500-mal

Das fertige Modell ist eine Summe: Start + viele kleine Korrekturen

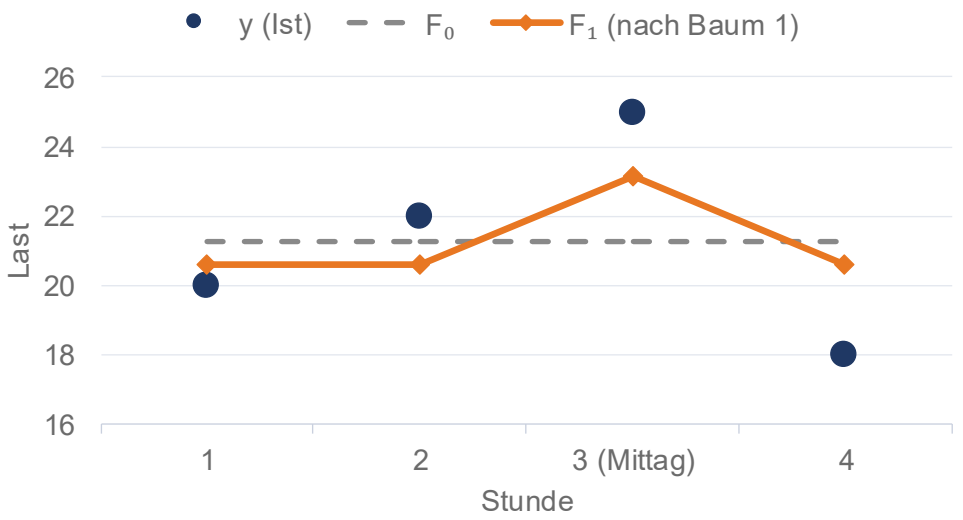


Warum für Zeitreihen? Bäume finden Wechselwirkungen selbst (Feiertag × Stunde) · unser Modell: 500 Bäume, η = 0,05, 31 Blätter

Gradient Boosting durchgerechnet

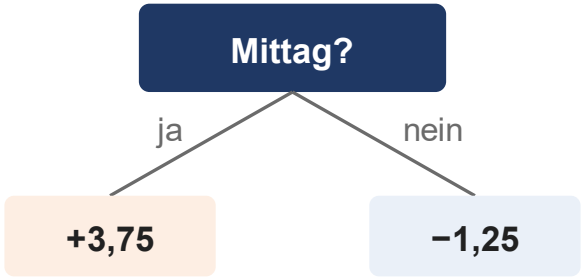
Vier Stunden, ein Baum, η = 0,5 ➔ der Rest schrumpft

Stunde	1	2	3 (Mittag)	4
y	20	22	25	18
F_0	21,25	21,25	21,25	21,25
Residuum	−1,25	+0,75	+3,75	−3,25
h_1 „Mittag?“	−1,25	−1,25	+3,75	−1,25
F_1 (η = 0,5)	20,625	20,625	23,125	20,625
neues Residuum	−0,625	+1,375	+1,875	−2,625



Σ|Residuum|: 9,0 → **6,5** · größter Schritt zur Mittagsstunde

Baum h_1: eine Frage, zwei Blätter



Blattwert = Mittel der Residuen, die dort landen

nein: (−1,25 + 0,75 − 3,25) / 3 = −1,25

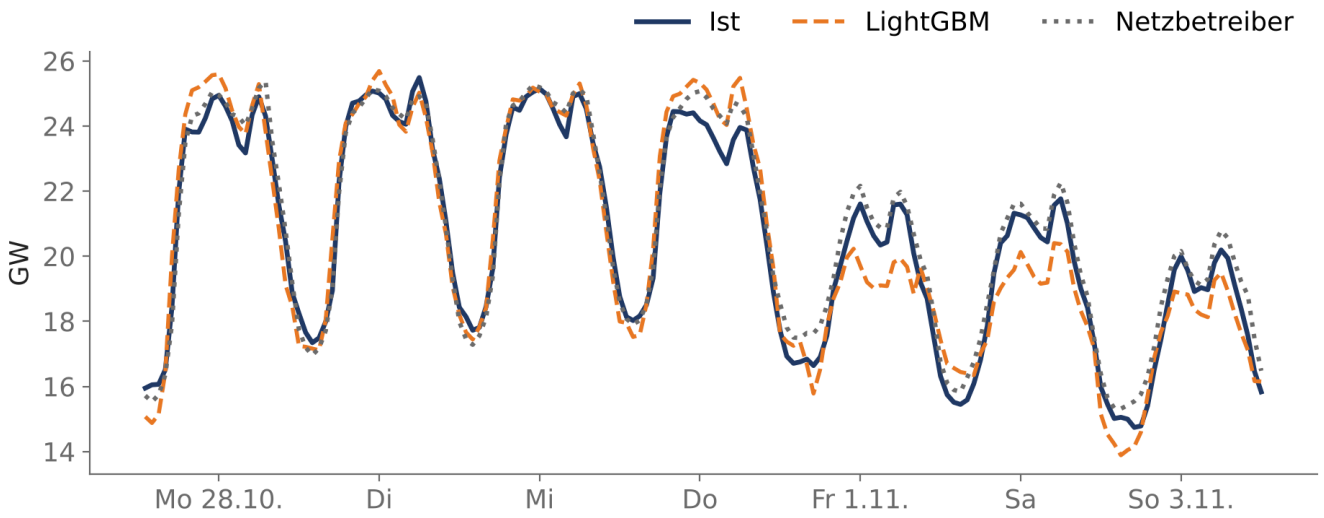
Mittag: 21,25 + 0,5 · 3,75 = **23,125**

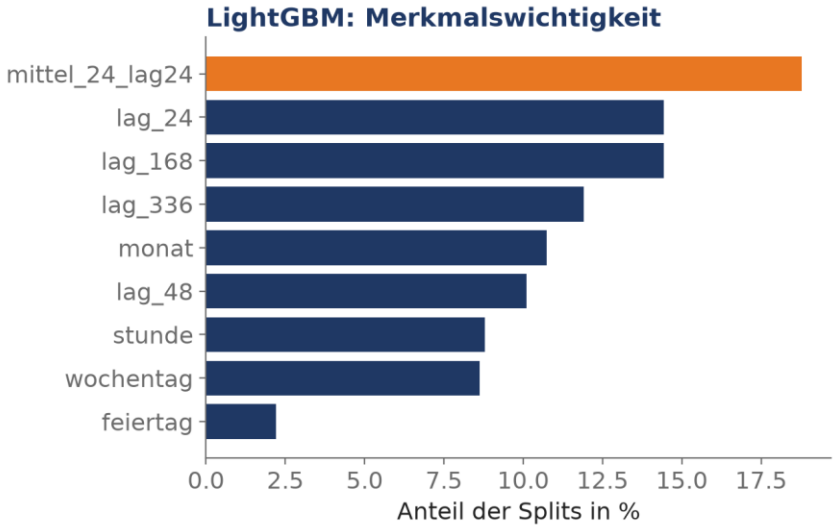
Selbst rechnen

Baum 2 fragt „Abend?“ (Stunde 4). Welche Werte gibt er aus, und wie lautet F_2 für Stunde 1 und 4?

Ergebnis: LightGBM

Lags plus Kalender und LightGBM liegt fast gleichauf mit dem Netzbetreiber





- Mo–Do beide fast auf der Ist-Kurve · Allerheiligen: LightGBM 935 MW Tages-MAE, Netzbetreiber 635 (MSTL 2.762)
- Wichtigkeit = Häufigkeit der Splits, nicht Einfluss – Feiertag wirkt selten, dann aber stark
- Netzbetreiber (gepunktet) = veröffentlichte Day-ahead-Prognose von Amprion, unsere Messlatte

LightGBM 488 MW gegen Netzbetreiber 466 MW – ohne Wetterdaten, trainiert in unter einer Sekunde

Ein LSTM für 24 Stunden

Eine Woche Vergangenheit hinein, 24 Werte heraus ➔ das Netz lernt die Merkmale selbst

Fenster
168 h × 4
Last · sin/cos Stunde · Wochenende

→

LSTM
32 Zustände
läuft über 168 Schritte

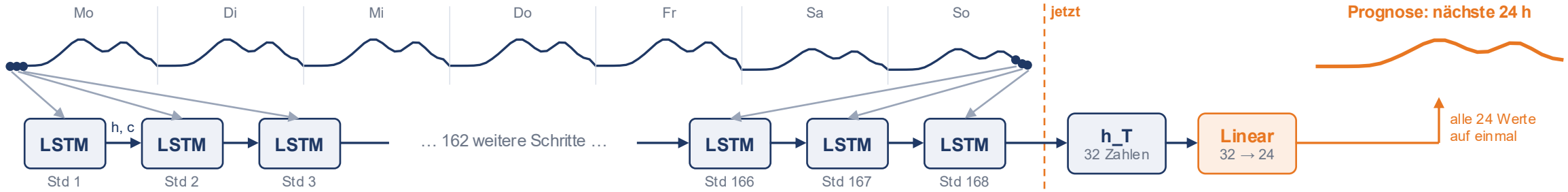
→

h_T
Zusammenfassung der Woche (32 Zahlen)

→

Linear
32 → 24
24 Stunden auf einmal

Abgerollt: dieselbe LSTM-Zelle, 168-mal nacheinander angewendet



Teil	Parameter
LSTM 4·32·(4+32) + 2·4·32	4.864
Kopf 32·24 + 24	792
Summe	5.656

Unterschied zu LightGBM

- keine handgebauten Lags, kein Feiertag – Struktur selbst lernen
- ≈ 5.000 Fenster, 8 Epochen, wenige Sekunden CPU
- Ergebnis: **745 MW** – besser als die Baselines, schlechter als MSTL und LightGBM

Temporal Fusion Transformer

Ein Transformer für Zeitreihen mit getrennten Eingängen für Vergangenheit und bekannte Zukunft

Vergangenheit

Last bis T (168 h)

bekannte Zukunft

Stunde, Wochentag,
Monat, Feiertag (24 h)

→

Variablenauswahl

Gewicht je Eingang
→ Wichtigkeit

→

LSTM Encoder–Decoder

lokale Muster
168 h → 24 h

→

Multi-Head Attention

jeder Zukunftsschritt
schaut auf alle 168 h

→

Quantil-Ausgabe

q10 · q50 · q90
je Stunde

Unser Checkpoint		Herkunft	
Parameter	64.319	Training	2018-01 bis 2019-08
Encoder / Horizont	168 h / 24 h	Validierung	2019-09, Early Stopping

Lim, Arık, Loeff & Pfister (2021), Temporal Fusion Transformers, Int. J. Forecasting 37(4) · pytorch-forecasting 1.8

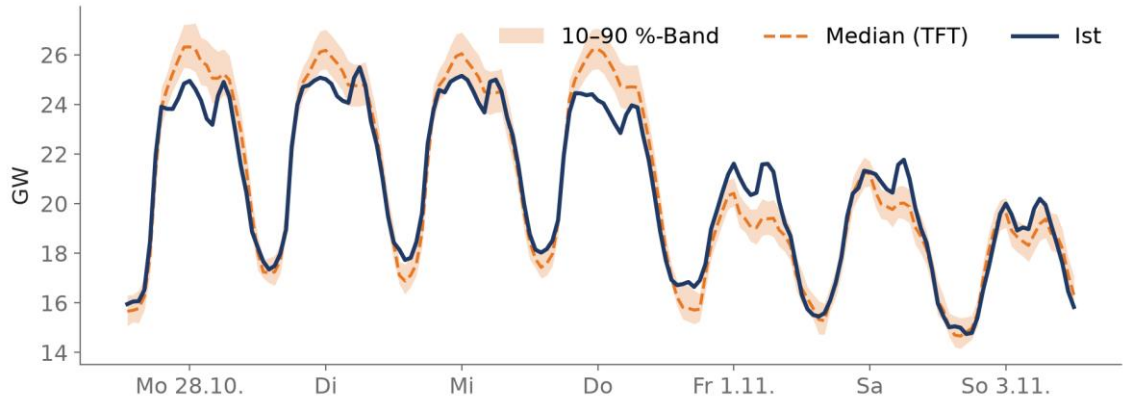
Quantilprognosen und Pinball-Loss

Das Modell soll ein Band liefern, nicht nur eine Zahl ➔ der sog. Pinball-Loss bringt ihm das bei

1 Ziel: ein Band statt einer Zahl

$q_{0,9}$ = Last, die in 90 % der Stunden **nicht überschritten** wird

$q_{0,1}$ bis $q_{0,9}$ = 80 %-Band → so viel Reserve muss bereitstehen



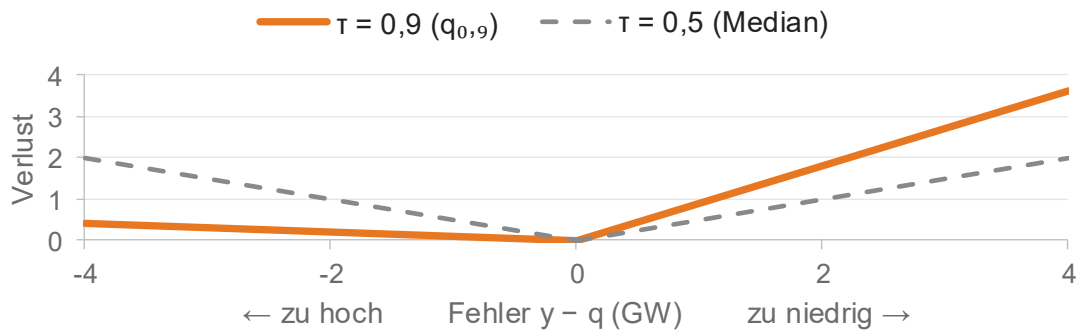
Gute Prognose: ca. 80 % der Ist-Werte (dunkelblau) liegen im Band, je ca. 10 % darüber und darunter

2 Bewertung: Pinball-Loss

$$L_\tau(y,q) = \begin{cases} \tau\,(y-q) & y \geq q \\ (1-\tau)\,(q-y) & y < q \end{cases}$$

y ≥ q Prognose zu niedrig → Fehler × τ

y < q Prognose zu hoch → Fehler × (1 − τ)



Folge: Weil „zu niedrig“ 9× teurer ist, schiebt das Modell $q_{0,9}$ nach oben bis nur noch ca. 10 % der Ist-Werte darüber liegen. Bei τ = 0,5 sind beide Seiten gleich teuer → Median.

Beispiel: Pinball von Hand

Ist-Wert y = 20 GW ➔ zwei Prognosen für das 90 %-Quantil

	Prognose q	Fehler y − q	Pinball τ = 0,9
zu hoch	22	−2	(1 − 0,9) · 2 = **0,2**
zu niedrig	18	+2	0,9 · 2 = **1,8**
zum Vergleich τ = 0,5	22 oder 18	∓2	0,5 · 2 = **1,0**

- Gleicher Abstand, **neunfacher** Verlust für „zu niedrig“
- Das Modell schiebt $q_{0,9}$ nach oben, bis nur 10 % darüber liegen
- Bewertung: Pinball je Quantil + **Abdeckung** des Bands

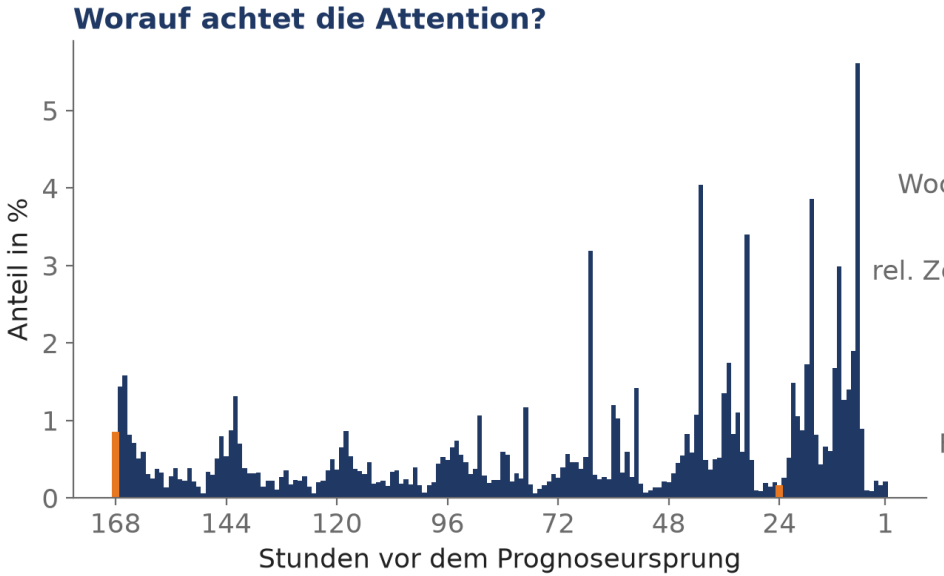
Selbst rechnen

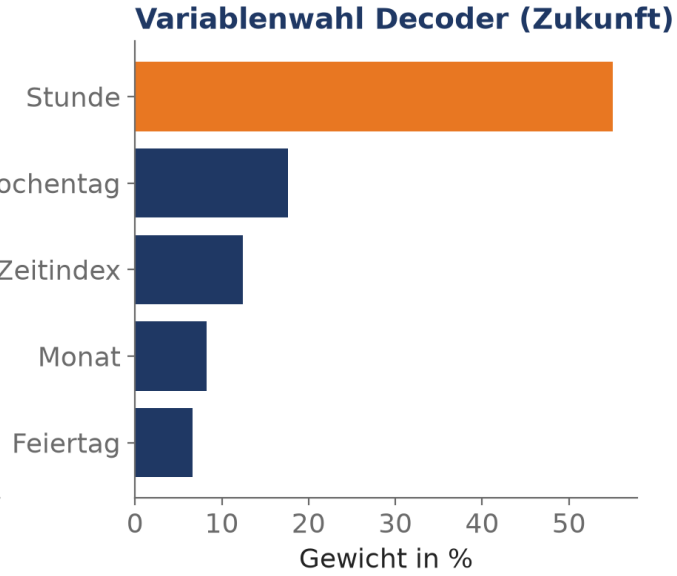
Dieselben Prognosen für **τ = 0,1**.

Welche ist jetzt teurer und warum?

Was der TFT gelernt hat

Die Attention schaut vor allem auf die letzten Tage und noch einmal auf den Tag vor einer Woche





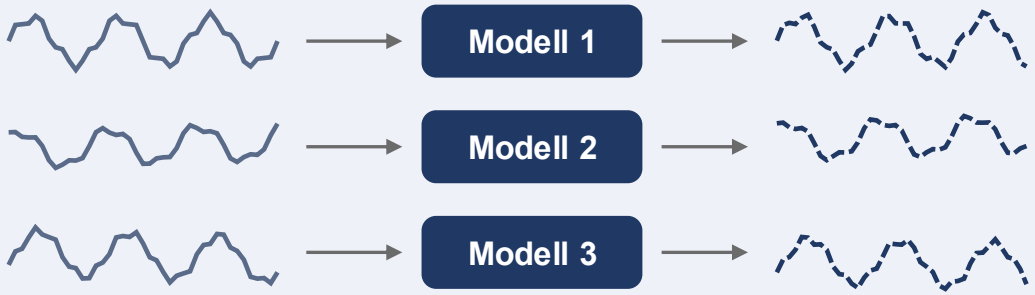
Kennzahl	Wert	Kennzahl	Wert	Deutung
MAE Median	586 MW	Abdeckung 10–90 %	71 %	Band zu schmal (Soll 80 %)
Attention letzte 24 h	29 %	Attention Tag −7	12 %	Ausreißer nach oben = Woche

Global oder lokal?

Neuronale Prognosemodelle spielen ihre Stärke bei vielen verwandten Reihen aus

Lokal: ein Modell je Reihe

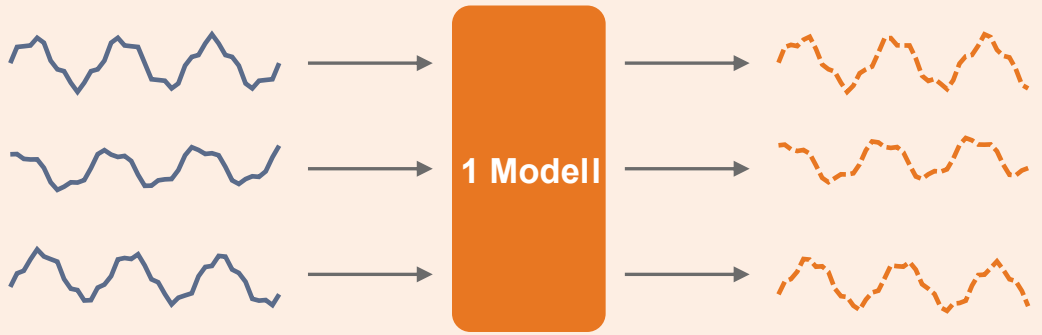
ETS · ARIMA · MSTL · LightGBM



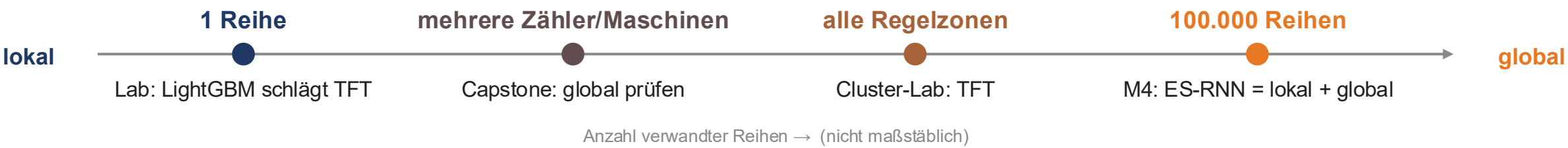
- \+ wenig Daten genügen
- − kein Transfer zwischen Reihen

Global: ein Modell für viele Reihen

DeepAR · N-BEATS · TFT



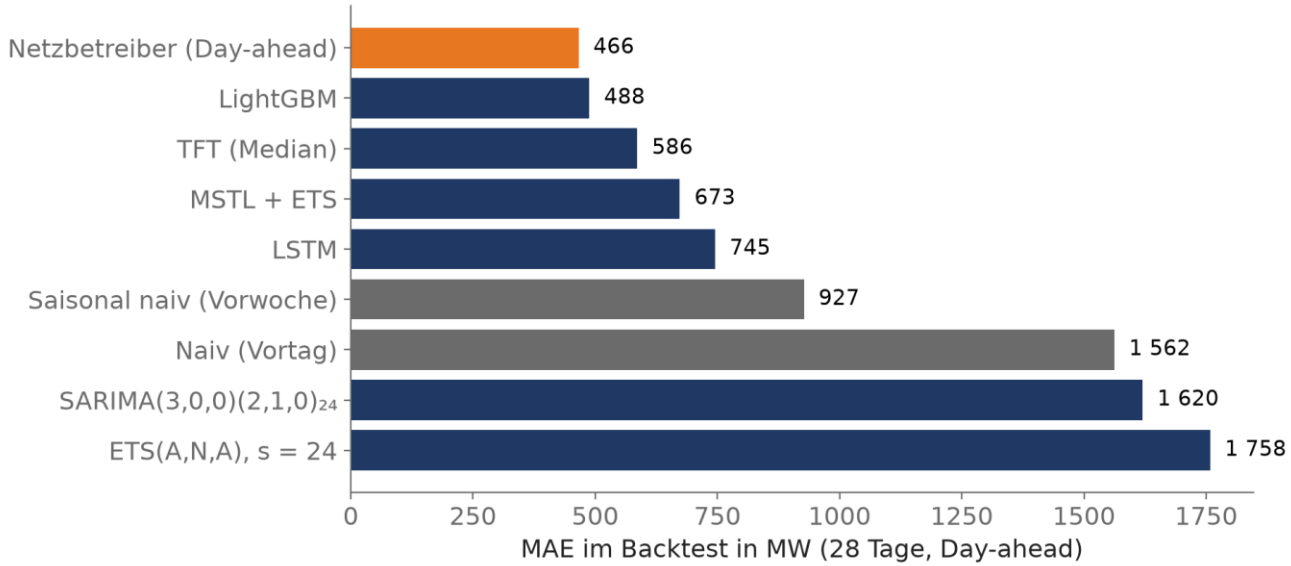
- \+ lernt gemeinsame Muster
- − braucht viele verwandte Reihen



Murphy (2023), PML Advanced Topics, Abschn. 29.12.8 · Smyl (2020), Int. J. Forecasting 36(1)

Alle Modelle im Vergleich

Backtest 7.10.–3.11.2019, 28 Day-ahead-Prognosen – und die Frage, welches Modell man einsetzt



Kriterium	LightGBM	TFT
MAE Backtest	488 MW	586 MW
Training	< 1 s	21 Min.
Unsicherheit	optional	Band (71 %)
Erklärbarkeit	SHAP	Attention

Endtest genau einmal: Dezember 2019

LightGBM **838** · Netzbetreiber **821** · saisonal naiv **1.886** MW

Weihnachtswoche: naiv 3.765 MW – der Backtest ist eine Schätzung, kein Versprechen

Zwischenstand: ML und Deep Learning

Was bringen Lags, LSTM und Transformer?

1

Lags + Kalender schlagen Architektur

LightGBM 488 MW ➔ fast gleichauf mit dem Netzbetreiber (466 MW)

2

Deep Learning braucht viele Reihen

LSTM 745 MW, TFT 586 MW bei einer einzigen Regelzone

3

Quantile und Deutung gibt es nicht umsonst

Pinball-Loss trainiert das Band · Abdeckung prüfen (71 % statt 80 %)

?

Welche zusätzliche Information würde allen Modellen am meisten helfen?

Weiter: Was verraten die Fehler, die übrig bleiben? → **Anomalien und Drift**

5

Anomalien und Drift

Leitfrage Was sagt das Residuum?

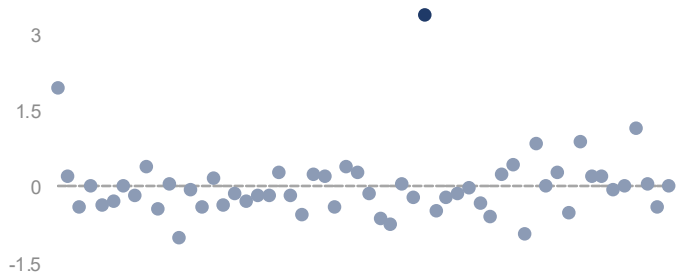


Das Residuum als Diagnose

Gute Residuen sind Rauschen ➔ jedes Muster darin ist Information, die das Modell nicht nutzt

Punktanomalie

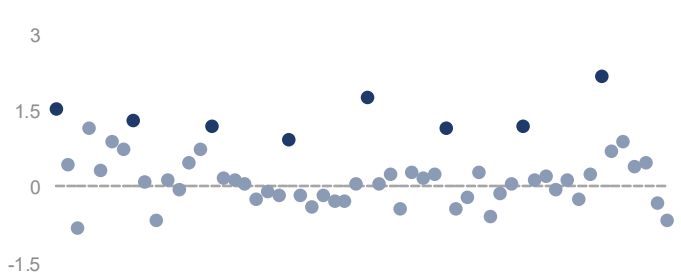
ein Tag weit daneben
Feiertag, Messfehler, Störung
→ Ausnahme erklären



ein einzelner Ausreißer, Rest unauffällig

Fehlendes Merkmal

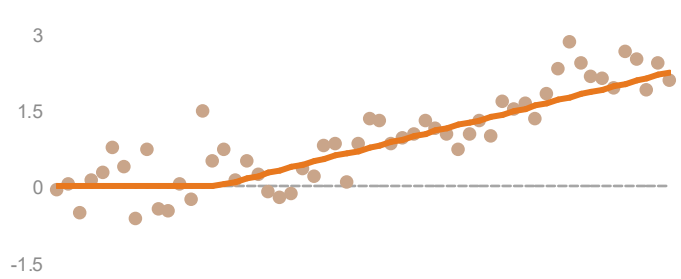
wiederkehrendes Muster
z. B. jeden Montag positiv
→ Merkmal ergänzen



Montage (dunkel) liegen regelmäßig über 0

Drift

Mittelwert verschiebt sich
über Wochen systematisch
→ nachtrainieren



gleitender Mittelwert wandert von 0 weg

$$e_t = y_t - \hat{y}_t$$

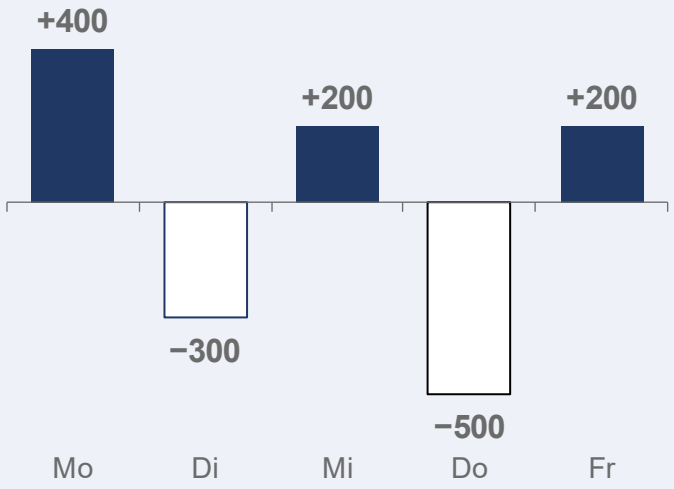
- Gute Residuen: **unkorreliert** und **Mittelwert 0** (FPP3 5.4)
- Unser Material: LightGBM auf Stand 30.9.2019, nicht nachtrainiert – Residuen bis 30.6.2020

Drift erkennen: MAE oder Bias?

Zwei Wochen mit gleichem MAE ➔ erst das Vorzeichen zeigt, welche driftet

Woche A · kein Drift

Tagesfehler e = y − ŷ in MW

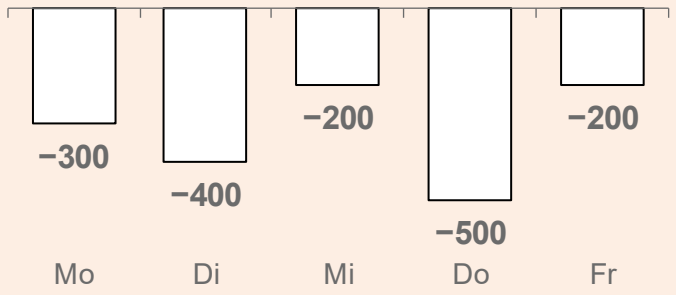


MAE **320** MW · Bias **0** MW

Fehler streuen um 0 → heben sich auf

Woche B · Drift

Tagesfehler e = y − ŷ in MW



MAE **320** MW · Bias **−320** MW

jeden Tag unter 0 → Prognose systematisch zu hoch

So liest man die Formeln

$$\mathrm{MAE} = \frac{1}{n}\sum |e_t|$$

$$\mathrm{Bias} = \frac{1}{n}\sum e_t$$

- **|e|:** Betrag – wie weit daneben?
- **e ohne Betrag:** + und − heben sich auf
- **Bias < 0:** ŷ > y, Prognose zu hoch

Selbst rechnen

Fehler −100, −300, +100, −500 MW

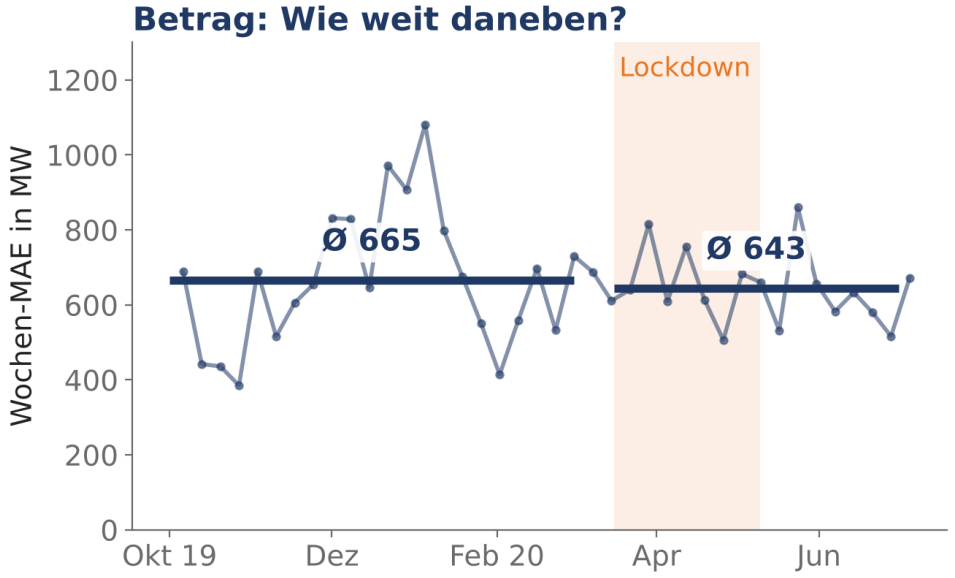
→ MAE = ? Bias = ?

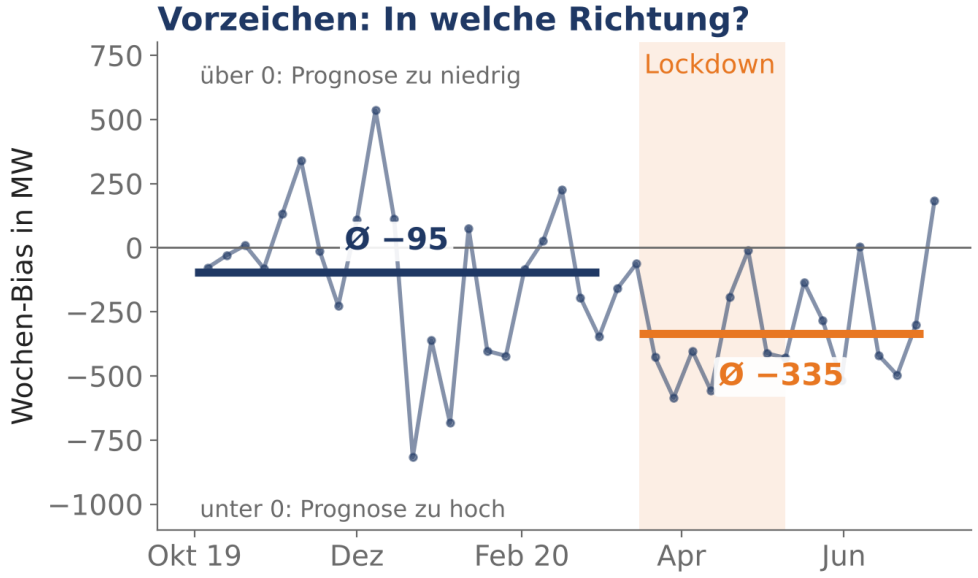
Drift oder Zufall?

Der MAE sagt, **wie weit** die Prognose danebenliegt, der Bias, **in welche Richtung** ➔ Drift ist ein Fehler mit Richtung ➔

Drift: der Lockdown 2020

Unser LightGBM (Stand 9/2019): Der MAE bleibt gleich, der Bias kippt ins Minus





vorher → ab Lockdown	MAE (MW)	Bias (MW)	Stunden zu hoch geschätzt
LightGBM (mit Lags)	665 → 643	**−95 → −335**	**53 % → 71 %**
Netzbetreiber (Profile + Wetter)	645 → 1.072	−338 → −1.032	65 % → 94 %

vorher: 1.10.2019–29.2.2020 · ab Lockdown: 16.3.–30.6.2020 · Residuum e = y − ŷ · Daten: OPSD, Regelzone Amprion

Lags holen das neue Niveau nach 24 h nach ➔ der MAE bleibt ruhig, nur der **Bias** verrät den Drift

Isolation Forest

1 Daten | 2 Ein Schnitt | 3 Isolieren | 4 Viele Bäume | 5 Score | 6 Schwelle



Zielpunkt x: **Normaler Punkt** | **Ausreißer** oder Punkt anklicken

Daten: zwei Merkmale, ein Ausreißer

60 Messpunkte mit den Merkmalen x_1 und x_2 – **ohne Labels**. Ein Punkt liegt abseits.

Idee: Statt „normal" zu modellieren, fragt der Isolation Forest: **Wie viele Zufallsschnitte braucht es, bis ein Punkt allein ist?** Seltene, andersartige Punkte sind schnell isoliert.

Zielpunkt x: normaler Punkt
Koordinaten: $x_1 = 0{,}39 \cdot x_2 = 0{,}46$
n = 60 Punkte · 2 Merkmale

Zielpunkt x unten umschalten oder direkt im Plot anklicken – er wird durch alle Schritte verfolgt.

← → Schritt wechseln · Leertaste = Aktion

Isolation Forest

1 Daten | 2 **Ein Schnitt** | 3 Isolieren | 4 Viele Bäume | 5 Score | 6 Schwelle



Zielpunkt x: **Normaler Punkt** | **Ausreißer** oder Punkt anklicken

Ein Zufallsschnitt

Jeder Schnitt besteht aus zwei Zufallsentscheidungen:

1. **Merkmal q** zufällig wählen (x_1 oder x_2)
2. **Schnittwert p** gleichverteilt zwischen Min und Max von q (oranger Bereich)

Weiter geht es nur mit der Seite, auf der x liegt (hellblau).

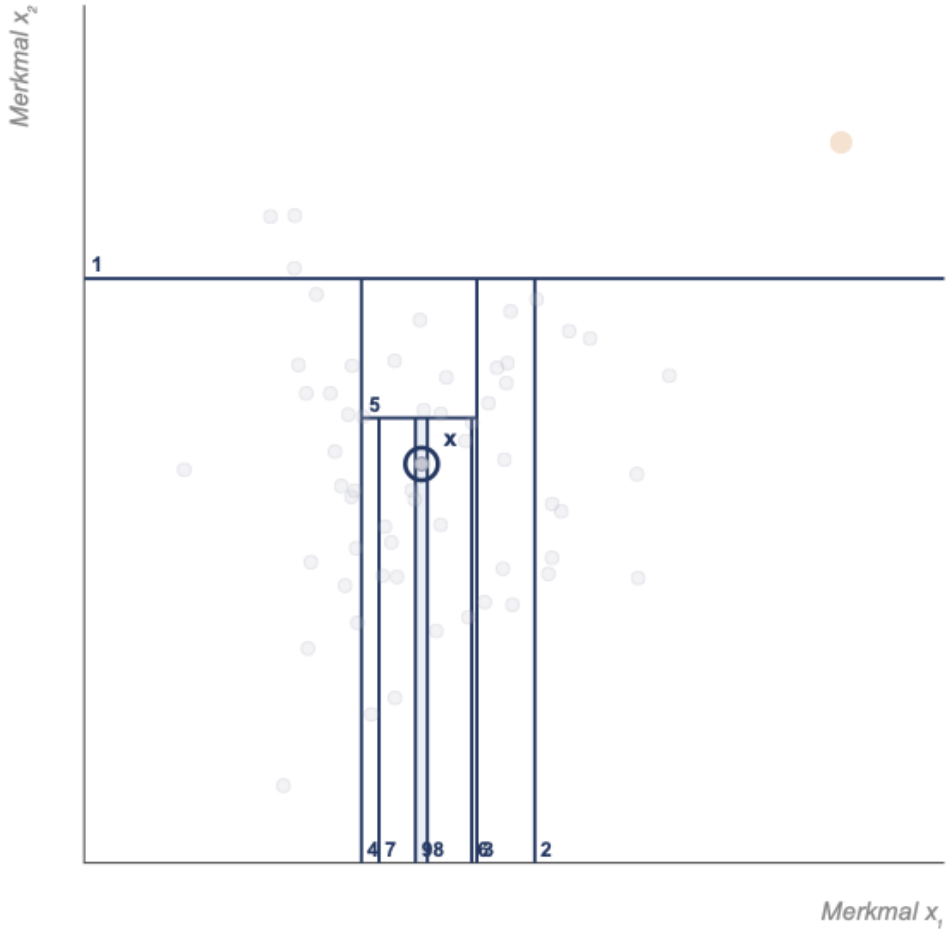
Neu würfeln

Merkmal q: x_2
Bereich: min 0,09 … max 0,84
Schnittwert p: 0,15
Auf der Seite von x: 59 von 60 Punkten

← → Schritt wechseln · Leertaste = Aktion

Isolation Forest

1 Daten | 2 Ein Schnitt | 3 Isolieren | 4 Viele Bäume | 5 Score | 6 Schwelle



Zielpunkt x: **Normaler Punkt** | **Ausreißer** oder Punkt anklicken

Schneiden, bis x allein ist

Schritt 2 wird in der verbleibenden Partition wiederholt. **h(x)** = Anzahl Schnitte, bis x allein ist – die Pfadlänge im Baum.

Nächster Schnitt | Bis zur Isolation | **Neuer Baum**

h(x) = 9 x ist isoliert ✔

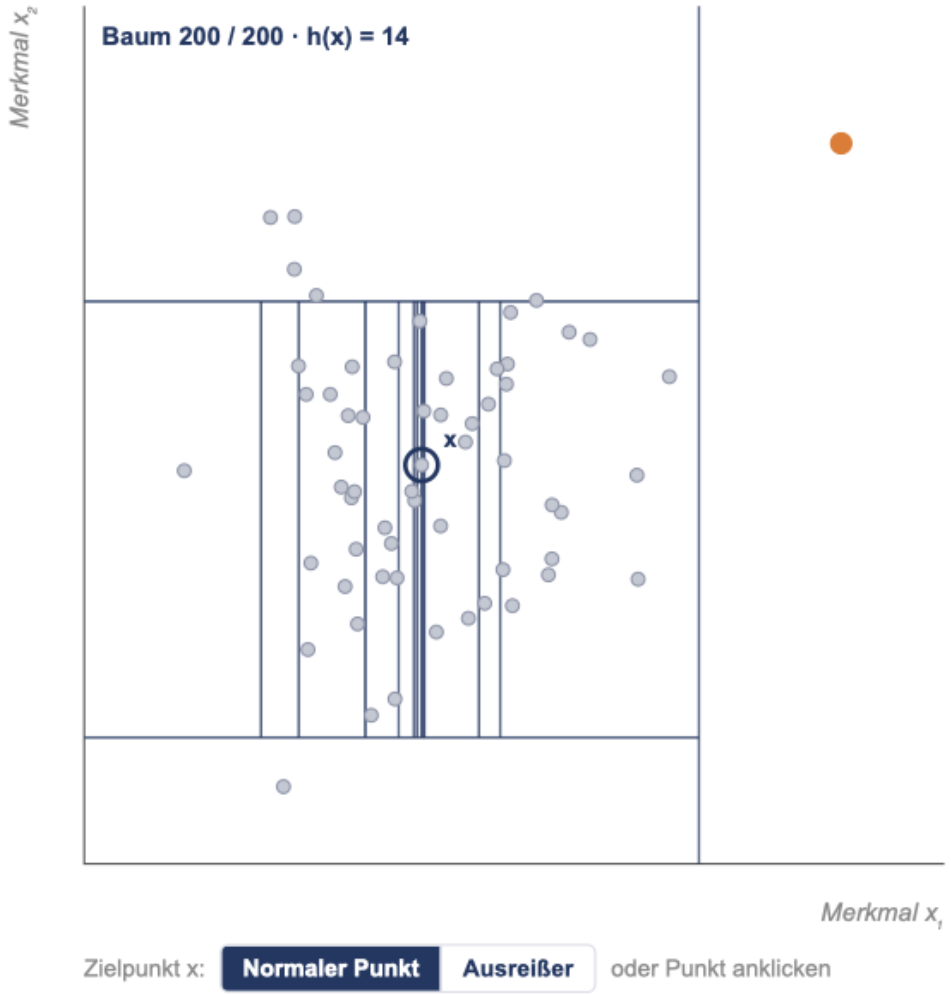
Bisherige Bäume für x: 9

Tipp: denselben Ablauf mit dem Ausreißer und mit dem normalen Punkt zeigen.

← → Schritt wechseln · Leertaste = Aktion

Isolation Forest

1 Daten · 2 Ein Schnitt · 3 Isolieren · 4 Viele Bäume · 5 Score · 6 Schwelle

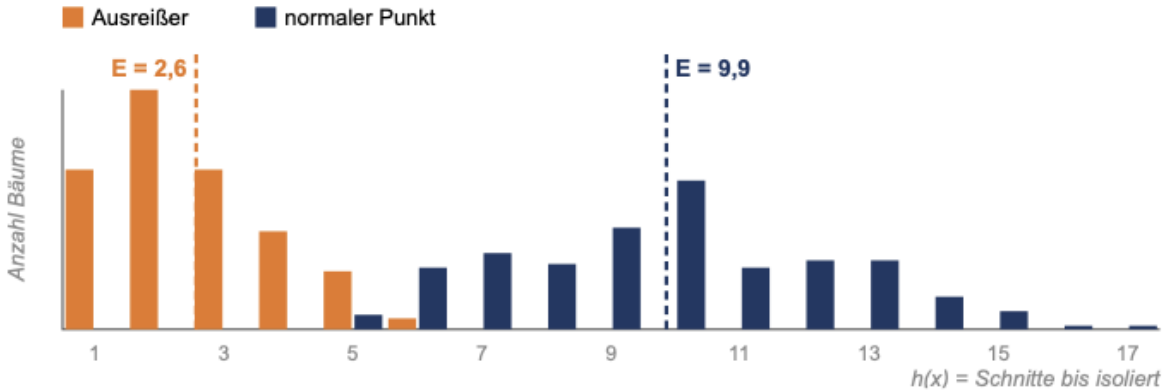


Zielpunkt x: **Normaler Punkt** **Ausreißer** oder Punkt anklicken

Viele Bäume → Mittelwert E[h(x)]

Ein einzelner Baum ist Zufall – h(x) streut von Baum zu Baum. Über 200 Bäume gemittelt wird die Pfadlänge stabil: **Ausreißer links, normale Punkte rechts.**

200 Bäume wachsen lassen **Zurücksetzen**

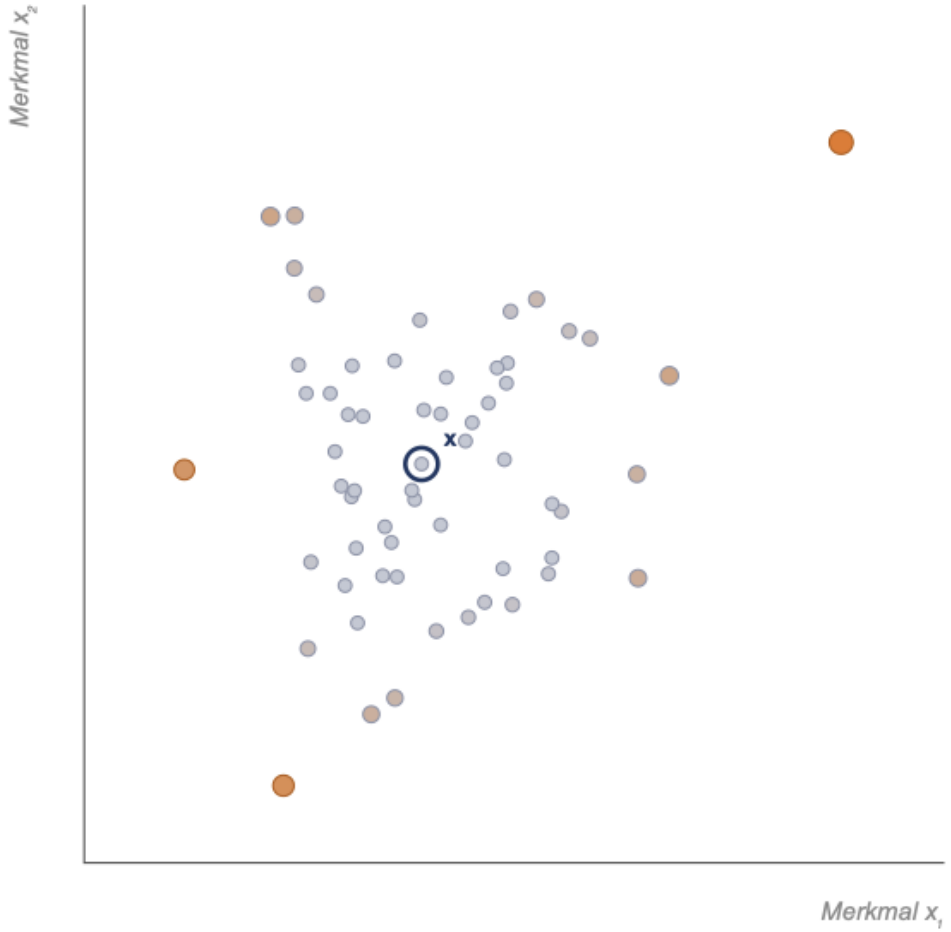


Bäume: 200 / 200
E[h] Ausreißer = 2,57 · **E[h]** normaler Punkt = 9,86

← → Schritt wechseln · Leertaste = Aktion

Isolation Forest

1 Daten · 2 Ein Schnitt · 3 Isolieren · 4 Viele Bäume · **5 Score** · 6 Schwelle



Zielpunkt x: **Normaler Punkt** | **Ausreißer** oder Punkt anklicken

Normieren → Anomalie-Score

$$s(x, n) = 2^{-E[h(x)] / c(n)}$$

c(n) ist die mittlere Pfadlänge bei n Punkten – sie macht den Score unabhängig von der Datenmenge.

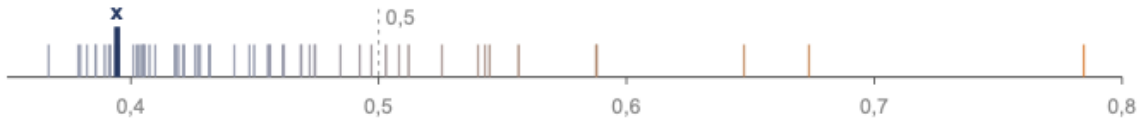
n = 60 (Punkte, aus denen jeder Baum gebaut wird)

c(n) = 2·H(n−1) − 2(n−1)/n

= 2·H(59) − 2·59/60 = 2·4,65 − 1,97 = **7,34** (H(i) ≈ ln i + 0,5772)

E[h(x)] = 9,86 (normaler Punkt, Mittel über 200 Bäume)

s(x, 60) = $2^{-9,86 / 7,34}$ = **0,39**



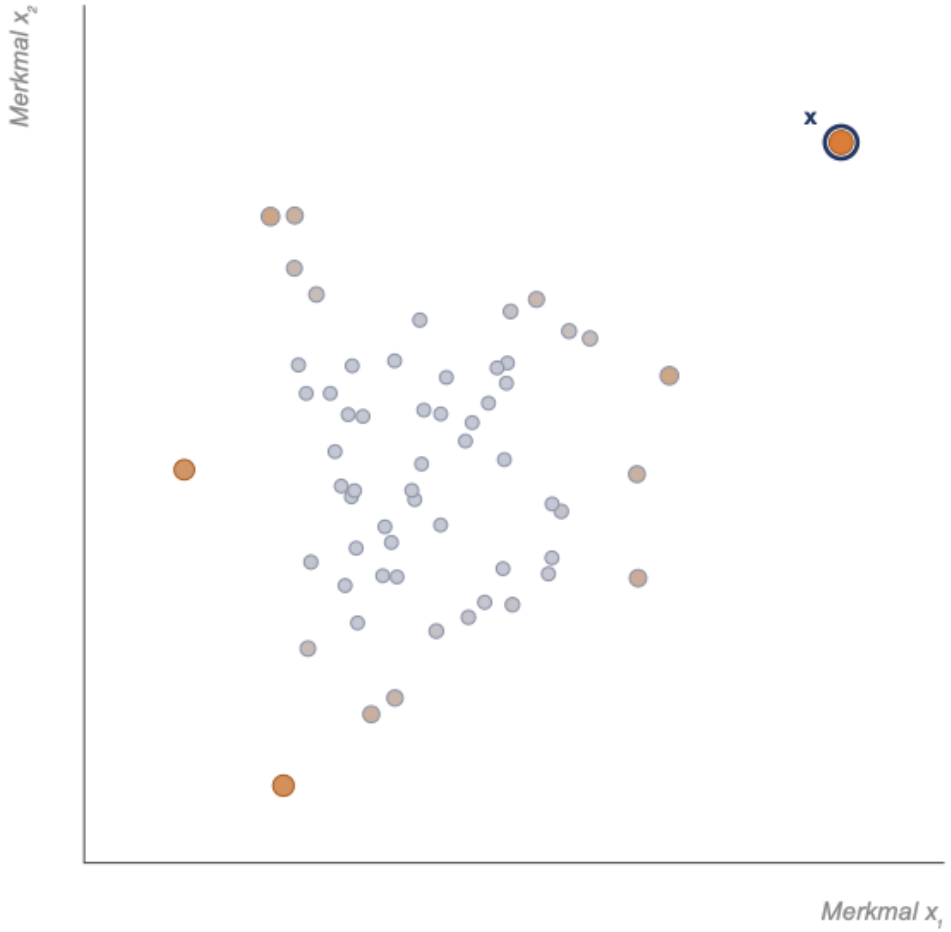
Score s – je weiter rechts, desto auffälliger

s ≈ 1: Anomalie · s < 0,5: normal · Punkt im Plot anklicken

← → Schritt wechseln · Leertaste = Aktion

Isolation Forest

1 Daten · 2 Ein Schnitt · 3 Isolieren · 4 Viele Bäume · **5 Score** · 6 Schwelle



Zielpunkt x: **Normaler Punkt** | **Ausreißer** oder Punkt anklicken

Normieren → Anomalie-Score

$$s(x, n) = 2^{-E[h(x)] / c(n)}$$

c(n) ist die mittlere Pfadlänge bei n Punkten – sie macht den Score unabhängig von der Datenmenge.

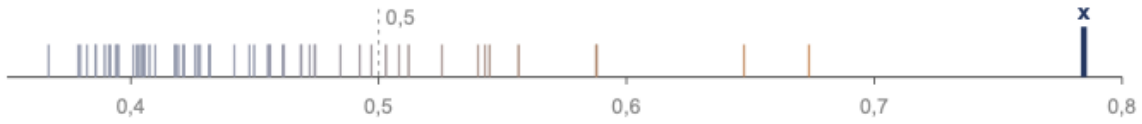
n = 60 (Punkte, aus denen jeder Baum gebaut wird)

c(n) = 2·H(n−1) − 2(n−1)/n

= 2·H(59) − 2·59/60 = 2·4,65 − 1,97 = **7,34** (H(i) ≈ ln i + 0,5772)

E[h(x)] = 2,57 (Ausreißer, Mittel über 200 Bäume)

s(x, 60) = $2^{-2,57 / 7,34}$ = **0,78**



Score s – je weiter rechts, desto auffälliger

s ≈ 1: Anomalie · s < 0,5: normal · Punkt im Plot anklicken

← → Schritt wechseln · Leertaste = Aktion

Isolation Forest

1 Daten · 2 Ein Schnitt · 3 Isolieren · 4 Viele Bäume · 5 Score · 6 Schwelle

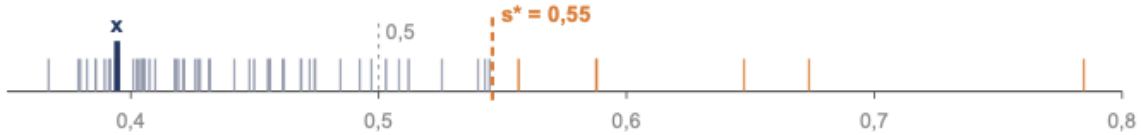


Zielpunkt x: **Normaler Punkt** | **Ausreißer** oder Punkt anklicken

contamination setzt nur die Schwelle

Die Scores bleiben unverändert. **contamination** = erwarteter Anomalie-Anteil legt nur fest, ab welchem s ein Punkt als Anomalie markiert wird.

contamination — **10 %** — **auto**



Score s – je weiter rechts, desto auffälliger

Schwelle s* = 0,55 · **markiert:** 6 von 60 Punkten
x (normaler Punkt, s = 0,39): normal (+1)

```
IsolationForest(n_estimators=200, contamination=0.10).fit_predict(X)
```

← → Schritt wechseln · Leertaste = Aktion

Beispiel: Score von Hand

256 Punkte im Teilbaum – wie viel kürzer muss der Pfad einer Anomalie sein?

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \qquad H(i) \approx \ln i + 0{,}5772$$

n = 256: H(255) ≈ 5,541 + 0,577 = 6,118

c(256) = 12,237 − 1,992 = 10,24

Punkt	E[h(x)]	s = 2^(−E[h]/10,24)	Urteil
A	2	2^(−0,195) = **0,873**	Anomalie
B	10	2^(−0,977) = **0,508**	normal

Selbst rechnen

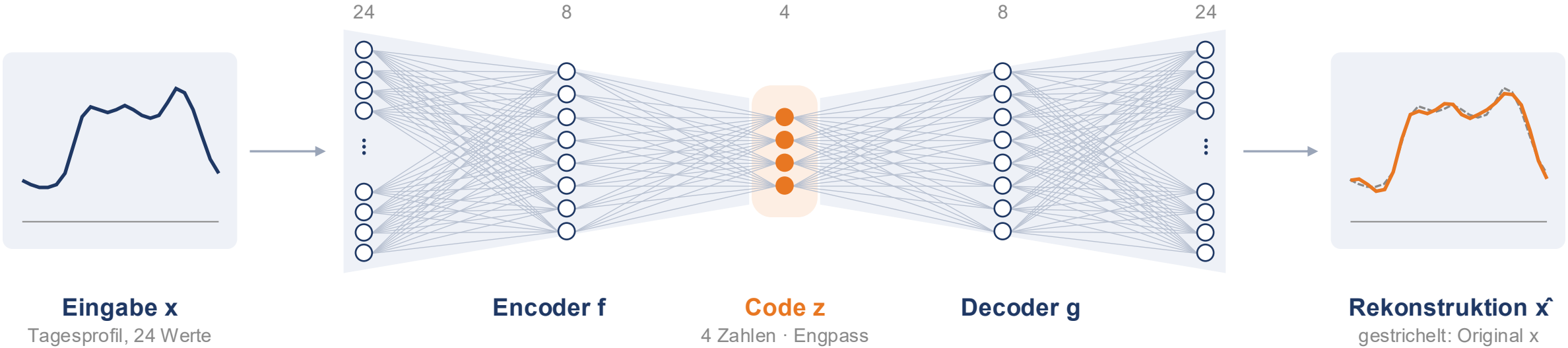
Ein Punkt hat **E[h] = 5**.

Berechnen Sie s.

Bei welchem E[h] ist s genau 0,5?

Autoencoder als Anomaliedetektor

Ein Netz, das normale Tage gut nachbaut, baut ungewöhnliche Tage schlecht nach



Eingabe x
Tagesprofil, 24 Werte

Encoder f

Code z
4 Zahlen · Engpass

Decoder g

Rekonstruktion $\hat{x}$
gestrichelt: Original x

$$\text{Score}(x) = \|x - g(f(x))\|^2$$

- Training nur auf normalen Tagen, ohne Labels
- Feiertag, Lockdown-Tag → passt in keine gelernte Form → großer Fehler
- Schwelle z. B. 99. Perzentil des Fehlers auf normalen Daten

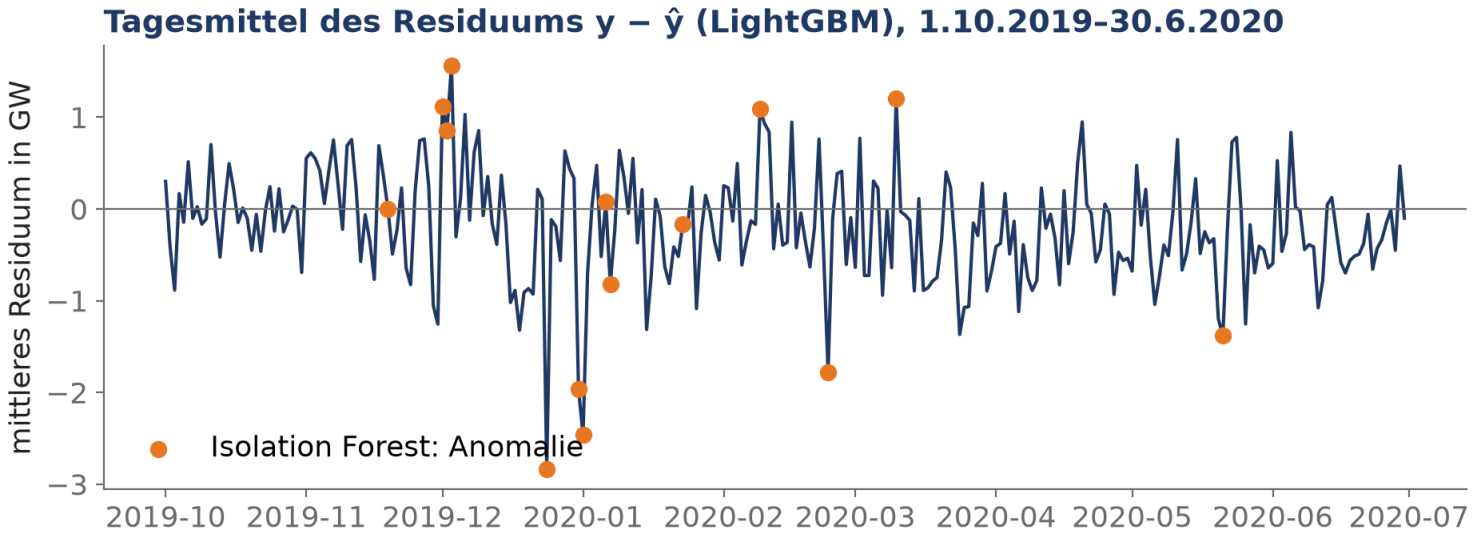
Isolation Forest oder Autoencoder?

IF: schnell, Merkmale selbst festlegen
AE: lernt die Form (Kurve, Bild) selbst
Capstone-Thema 6: AE auf MVTec-Bildern

Murphy (2022), PML Introduction, Abschn. 20.3.1

Ergebnis: Anomalien 2019/20

Isolation Forest auf drei Tagesmerkmalen des Residuums: 14 von 274 Tagen auffällig



Tag	Ø Resid.	Ursache
24.12.2019	−2.839	Heiligabend
1.1.2020	−2.464	Neujahr
31.12.2019	−1.966	Silvester
24.2.2020	−1.785	Rosenmontag
21.5.2020	−1.382	Himmelfahrt
3.12.2019	+1.560	offen

- Fast alles Kalendertage, die das Modell nicht kennt → **Merkmal ergänzen** (Heiligabend, Brückentage, Rosenmontag)
- Der Lockdown ist **keine Punktanomalie**, sondern Drift – zwei Werkzeuge für zwei Fragen

Zwischenstand: Anomalien und Drift

Was sagt das Residuum?

1

Gute Residuen sind Rauschen

unkorreliert, Mittelwert 0 ➔ jedes Muster ist ungenutzte Information

2

Drift sitzt im Vorzeichen

Lockdown: LightGBM-Bias ≈ −300 MW, MAE kaum verändert

3

Kurze Pfade = Anomalie

Isolation Forest: 14 von 274 Tagen, fast alle Kalendertage

?

Drei Wochen Bias −300 MW bei normalem MAE – nachtrainieren?

Weiter: Vom Vorhersagen zum Entscheiden → **Reinforcement Learning kompakt**

6

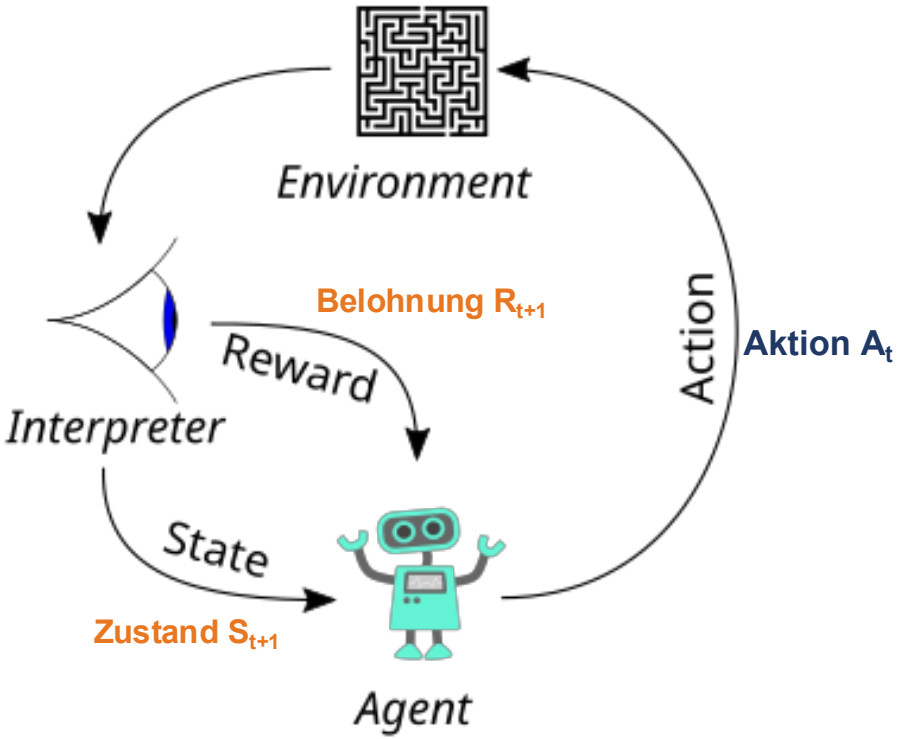
Reinforcement Learning kompakt

Leitfrage Wie lernt ein Agent aus Belohnung?



Vom Vorhersagen zum Entscheiden

Der Agent handelt, die Umwelt antwortet mit neuem Zustand und Belohnung ➔ daraus lernt er



Quelle: https://de.wikipedia.org/wiki/Datei:Reinforcement_learning_diagram.svg

Was ist anders als bisher?

- kein Label, nur eine **Belohnung** (oft verzögert)
- die Aktion **verändert die Daten**, die der Agent sieht
- Ziel: **Summe künftiger Belohnungen**, nicht die nächste

Beispiele

- Batteriespeicher: laden bei niedrigem, entladen bei hohem Preis
- Regelung, Lagerhaltung, Robotik, AlphaGo
- RLHF: PPO trainiert Sprachmodelle → Moderne NLP & LLM-Anpassung

Markov-Entscheidungsprozess

Fünf Zutaten beschreiben jedes RL-Problem ➔ die Gegenwart enthält alles, was zählt

$$(\mathcal{S},\ \mathcal{A},\ p(s', r \mid s, a),\ \gamma) \qquad \pi(a \mid s)$$

So liest man die Formel

S, A: Zustände und Aktionen

p(s′, r | s, a): Wahrscheinlichkeit für neuen Zustand und Belohnung

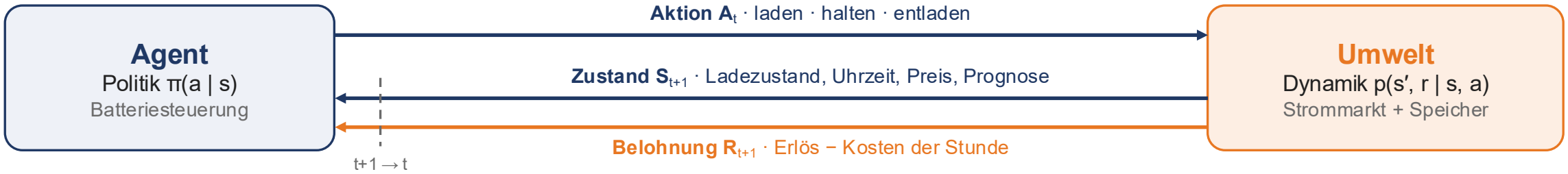
γ ∈ [0, 1): Diskontfaktor ➔ wie stark Zukunft zählt

π(a | s): Politik (was der Agent in s tut)

Markov: nur der aktuelle Zustand zählt, nicht die Vorgeschichte

Batteriespeicher	
Zustand	Ladezustand, Uhrzeit, Preis, Prognose
Aktion	laden · halten · entladen
Belohnung	Erlös minus Kosten der Stunde
Übergang	Ladezustand fest, Preis zufällig
Markov?	nur mit Prognose im Zustand

Agent–Umwelt-Schleife: ein Zeitschritt von t nach t+1



Markov: Der Agent sieht nur S_t. Was er über die Zukunft wissen soll, muss im Zustand stehen (hier: die Preisprognose).

Schaubild in Anlehnung an Sutton, R. S.; Barto, A. G. (2018): Reinforcement Learning: An Introduction, 2. Aufl., MIT Press, Abb. 3.1, S. 48

Return und Diskont

Der Agent maximiert die Summe künftiger Belohnungen ➔ je weiter in der Zukunft, desto weniger zählt sie

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Episodisch (CartPole, max. 500 Schritte, +1 je Schritt): γ = 1 erlaubt → G ≤ 500; mit γ = 0,99 → G ≤ 99,3

Fortlaufend (Speicher läuft endlos): γ < 1 nötig, sonst G = ∞

So liest man die Formel

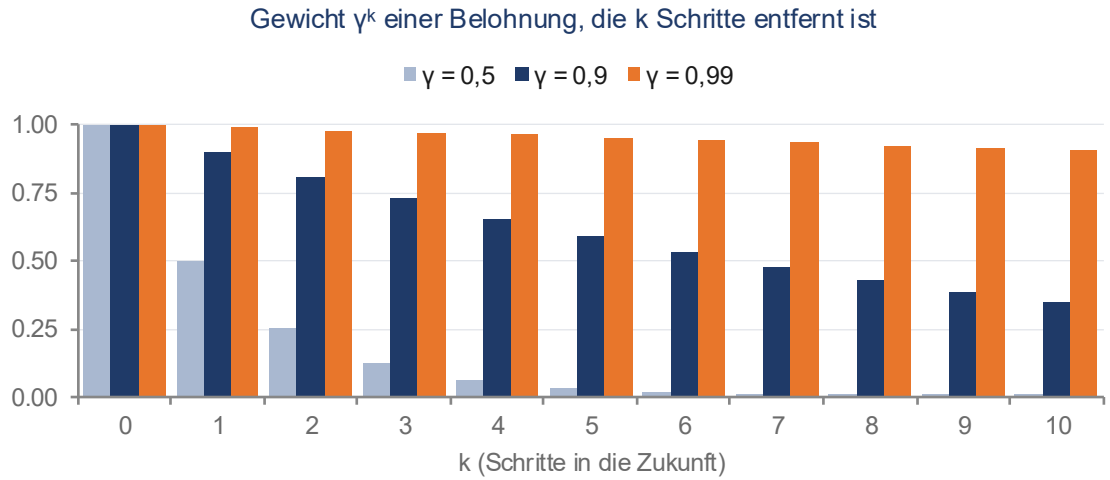
G_t: Return (alle künftigen Belohnungen ab t)

R_{t+k+1}: Belohnung k Schritte nach jetzt

γ^k: Gewicht, 0 ≤ γ ≤ 1 (sinkt mit k)

γ = 0: kurzsichtig · **γ → 1:** weitsichtig

rekursiv: **$G_t = R_{t+1} + \gamma \cdot G_{t+1}$** (jetzt + abgezinster Rest)



Bezug zum MDP

S A P R **γ**

γ ist das 5. Element des MDP-Tupels (S, A, P, R, γ) ➔ Teil der Problemdefinition, nicht des Lernverfahrens. Das Ziel jeder Policy π ist der erwartete Return: **$V^\pi(s) = E_\pi[G_t \mid S_t = s]$**. Weil P nur vom aktuellen Zustand abhängt (Markov), wird aus der Rekursion von G_t die Bellman-Gleichung.

Wertfunktionen und Bellman

Wie gut ist es, in Zustand s die Aktion a zu wählen – und danach optimal weiterzumachen?

DEFINITION

$$q_\pi(s,a) = \mathbb{E}_\pi\big[\, G_t \;\big|\; S_t = s,\ A_t = a \,\big] \qquad \text{mit} \quad G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

BELLMAN-OPTIMALITÄT · Wert = jetzt + Diskont × bester Wert danach

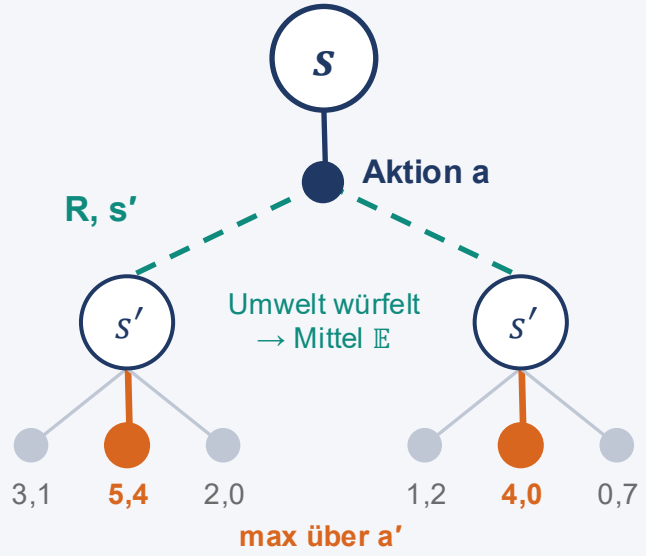
$$\underbrace{q_*(s,a)}_{\text{Wert von a in s}} = \mathbb{E}\Big[\, \underbrace{R_{t+1}}_{\text{jetzt}} + \underbrace{\gamma}_{\text{Diskont}} \underbrace{\max_{a'} q_*(S_{t+1}, a')}_{\text{bester Wert danach}} \;\Big|\; S_t = s,\ A_t = a \,\Big]$$

q_π(s, a) wie gut ist a, wenn ich danach Politik π folge

q*(s, a) wie gut ist a, wenn ich danach optimal weitermache

q* bekannt → greedy handeln: $\pi^*(s) = \text{argmax}_a\, q^*(s,a)$ – fertig

Ein Schritt vorausschauen



ZWEI WEGE ZU q*

Modell p bekannt → planen (Wertiteration)

Bellman-Gleichung wiederholt auf alle Zustände anwenden

Modell unbekannt → aus Erfahrung lernen (Q-Learning)

𝔼 ersetzen durch beobachtete Schritte (s, a, r, s′), ε-greedy erkunden

Policy Gradient und PPO

Statt Werte zu schätzen, die Politik direkt verbessern – Aktionen mit hohem Return wahrscheinlicher machen

$$\theta \leftarrow \theta + \alpha\, G_t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t)$$

$$L^{\mathrm{CLIP}} = \mathbb{E}\Big[\min\left(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\right)\Big]$$

So liest man die Formel

π_θ(a | s): Netz gibt Aktionswahrscheinlichkeiten aus

REINFORCE: hoher Return → Aktion wahrscheinlicher

r_t = π_neu / π_alt: wie stark ändert sich die Politik?

clip mit ε = 0,2: höchstens ±20 % pro Update – stabil

Wo PPO steckt

Standardverfahren für viele Aufgaben

kontinuierliche Aktionen möglich

RLHF bei Sprachmodellen (InstructGPT)

RL in der Praxis

Mächtig, aber anspruchsvoll – deshalb kommt RL in keinem Capstone-Thema vor

1 Dateneffizienz

CartPole: 49 103 Interaktionen · echte Anlage → Simulator nötig

2 Belohnungsdesign

Der Agent optimiert die Belohnung, nicht die Absicht (reward hacking)

3 Sicherheit

Erkunden heißt Fehler ausprobieren – im Betrieb nicht akzeptabel

4 Reproduzierbarkeit

große Streuung über Seeds – mehrere Läufe nötig

Oft reicht weniger:

Speicher am Strommarkt → **Prognose (heute) + lineare Optimierung** ist Industriestandard; RL lohnt mit gutem Simulator

Zwischenstand: Reinforcement Learning

Wie lernt ein Agent aus Belohnung?

1

Handeln, belohnt werden, lernen

MDP · Ziel ist der Return: jetzt + abgezinster Rest

2

Bellman: jetzt + γ · bester Wert danach

Modell bekannt → planen · Modell unbekannt → Q-Learning aus Erfahrung

3

PPO verbessert die Politik direkt

Grundlage von RLHF (Woche 4) · in der Praxis nur mit Simulator, oft reicht Prognose + Optimierung

?

Welche Entscheidung aus Ihrem Umfeld wäre ein RL-Problem – mit Simulator?

Weiter: Welches Projekt wählt Ihre Gruppe – und welchen Compliance-Aufwand bringt es mit? → **Capstone**

Bis nächste Woche

Nachbereiten, lesen, das Datenkonzept vorbereiten

1 Lab 05 nachbereiten

Notebook komplett durchlaufen lassen

optional: lab05_rl_optional (DQN, CartPole)

2 Lesen

FPP3 Kap. 1 + 9 · Sutton & Barto Kap. 1 + 3

Vertiefung: Lim et al. (2021), TFT

3 Capstone vorbereiten

Datenquellen recherchieren, Zugang klären

Thema 2: MIMIC-IV-Zugang jetzt beantragen

4 Ausblick

Block C: Modelle produktiv machen

Daten-Engineering – das Trägermodell kehrt zurück

A

Anhang: zum Nachlesen

Leitfrage Notation, Herleitungen, Code und Lektüre – zum Selbststudium

ARIMA mit dem Backshift-Operator

Die kompakte Schreibweise aus FPP3 – und unser Lab-Modell darin

$$B\,y_t = y_{t-1}, \qquad (1-B)\,y_t = y_t - y_{t-1}, \qquad (1-B^m)\,y_t = y_t - y_{t-m}$$

$$\underbrace{(1-\phi_1 B - \cdots - \phi_p B^p)}_{\text{AR}}\ \underbrace{(1-B)^d}_{\text{I}}\ y_t = c + \underbrace{(1+\theta_1 B + \cdots + \theta_q B^q)}_{\text{MA}}\ \varepsilon_t$$

$$(1-\phi_1 B - \phi_2 B^2 - \phi_3 B^3)\,(1-\Phi_1 B^{24} - \Phi_2 B^{48})\,(1-B^{24})\,y_t = \varepsilon_t$$

- Unten: unser Lab-Modell SARIMA(3,0,0)$(2,1,0)_{24}$ – ausmultipliziert Lags 1–3, 24–27, 48–51, 72–75
- Quelle: FPP3 Abschnitte 9.2, 9.5, 9.9

Bellman-Gleichung hergeleitet

Aus der Rekursion des Returns folgt die Rekursion der Werte

1. $$G_t = R_{t+1} + \gamma\, G_{t+1}$$

   Rekursion des Returns

2. $$v_\pi(s) = \mathbb{E}_\pi\left[R_{t+1} + \gamma\, G_{t+1} \mid S_t = s\right]$$

   Erwartungswert unter π

3. $$v_\pi(s) = \mathbb{E}_\pi\left[R_{t+1} + \gamma\, v_\pi(S_{t+1}) \mid S_t = s\right]$$

   Markov: E[G_{t+1} | S_{t+1}] = v_π(S_{t+1})

4. $$v_*(s) = \max_a \mathbb{E}\left[R_{t+1} + \gamma\, v_*(S_{t+1}) \mid S_t = s, A_t = a\right]$$

   Optimal: beste Aktion statt Mittel über π

Beispiel: Return von Hand

Vier Schritte mit Belohnungen 1, 1, 1 und 10 – was ist die Zukunft heute wert?

Schritt k	**0**	**1**	**2**	**3**	**G_0**
R_{k+1}	1	1	1	10	
$\gamma = 0{,}9$: $\gamma^k R$	1	0,9	0,81	7,29	**10,00**
$\gamma = 0{,}5$: $\gamma^k R$	1	0,5	0,25	1,25	**3,00**

- Bei $\gamma = 0{,}5$ ist die große Belohnung nur noch **1,25** wert
- Kurzsichtiger Agent nimmt lieber sofort kleine Erlöse mit
- Probe rekursiv: $G_3 = 10$, $G_2 = 1 + 0{,}9 \cdot 10 = 10$, …

Selbst rechnen

Berechnen Sie G_0 für **$\gamma = 0$** und **$\gamma = 1$**.

Was bedeutet das jeweils für den Agenten?

Q-Learning

Schätzung ein Stück in Richtung „Belohnung plus bester Wert danach“ schieben – nach jedem Schritt

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[\underbrace{R_{t+1} + \gamma \max_a Q(S_{t+1}, a)}_{\text{TD-Ziel}} - Q(S_t, A_t)\Big]$$

So liest man die Formel

Q(S_t, A_t): bisherige Schätzung

TD-Ziel: Belohnung + bester geschätzter Wert im nächsten Zustand

Klammer = TD-Fehler: wie sehr hat mich der Schritt überrascht?

α: Lernrate – wie weit schiebe ich

off-policy: Ziel mit max, egal was ich als Nächstes tue

Wie Gradientenabstieg

Schritt auf (Ziel − Schätzung)²

Tabelle: eine Zahl je (s, a)

zu viele Zustände → Netz statt Tabelle: **DQN**

Beispiel: Ein Q-Update von Hand

Batteriespeicher: Q(s, entladen) = 2, Erlös 1, bester Wert im Folgezustand 4, α = 0,5, γ = 0,9

Schritt	Rechnung	Ergebnis
TD-Ziel	1 + 0,9 · 4,0	4,6
TD-Fehler	4,6 − 2,0	+2,6
neue Schätzung	2,0 + 0,5 · 2,6	**3,3**

Selbst rechnen

Dasselbe mit **α = 0,1**.

Und mit α = 0,5, aber **Erlös R = −1**?

- Positiver TD-Fehler: besser als erwartet → Schätzung steigt
- Die Schätzung wandert um α · Überraschung = halb so weit
- Folgezustand s′: Q(laden) = 3,0 · Q(halten) = 4,0 · Q(entladen) = 2,5 → max = 4,0

Erkunden oder ausnutzen?

Wer immer die beste bekannte Aktion wählt, findet nie heraus, ob es eine bessere gibt

$$A_t = \begin{cases} \text{zufällige Aktion} & \text{mit Wahrscheinlichkeit } \varepsilon \\ \arg\max_a Q(S_t, a) & \text{sonst} \end{cases}$$

Ausnutzen

beste bekannte Aktion
Belohnung jetzt
Gefahr: hängen bleiben

Erkunden

zufällige Aktion
Information für später
kostet kurzfristig

In der Demo:

ε fällt exponentiell von **0,9 auf 0,05** (Zeitkonstante 1 000 Schritte)

Alternativen: Boltzmann (Temperatur!), UCB, Thompson Sampling

Deep Q-Network (DQN)

Ein neuronales Netz ersetzt die Q-Tabelle – zwei Tricks machen das Training stabil

$$\mathcal{L}(\theta) = \left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2$$

1 Replay-Puffer

(s, a, r, s′) speichern
zufällige Minibatches
bricht Korrelationen

2 Zielnetz θ^-

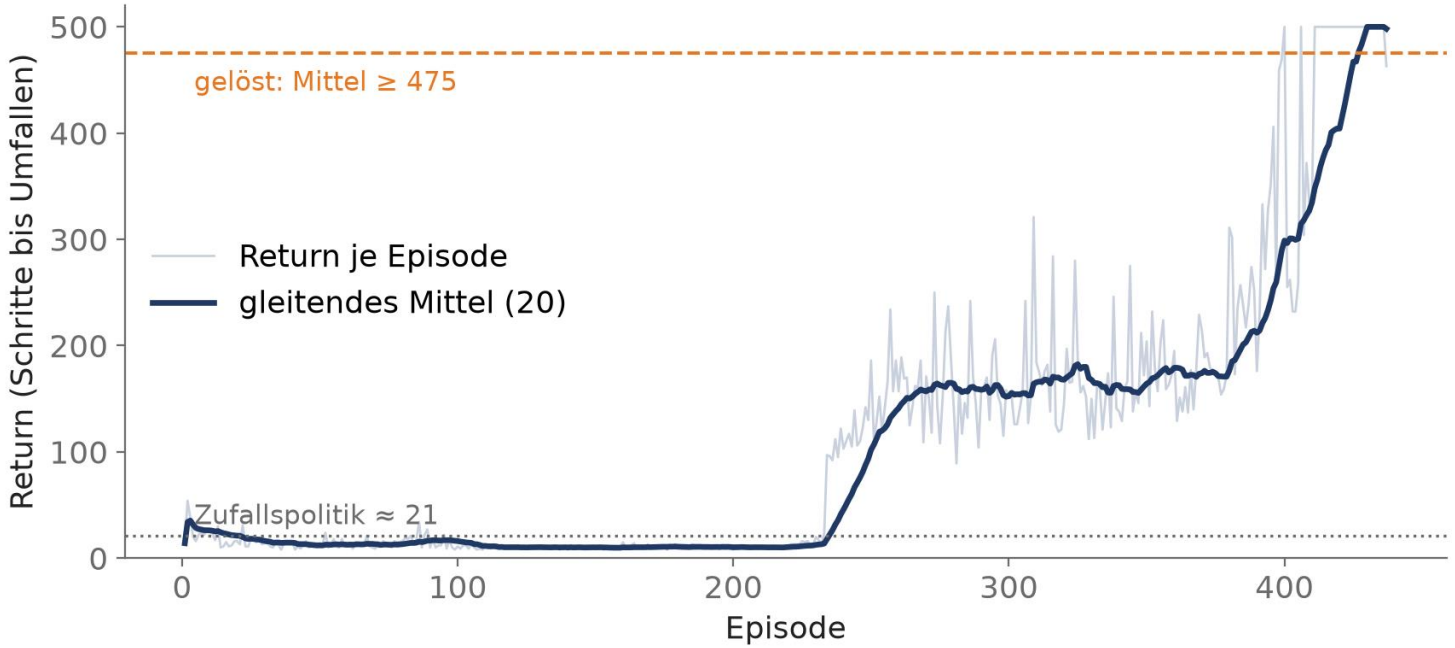
Kopie berechnet das Ziel
alle C Schritte kopiert
oder weich: τ = 0,005

Unser DQN	CartPole
Zustand	4 Messwerte
Aktionen	links · rechts
Netz	4 → 128 → 128 → 2
Parameter	17 410
Zielnetz	weich, τ = 0,005

Mnih et al. (2015), Human-level control through deep reinforcement learning, Nature 518 · 49 Atari-Spiele, ein Netz

CartPole: DQN im Notebook

Ein Stab auf einem Wagen – nach wenigen Minuten CPU balanciert das DQN die vollen 500 Schritte



Referenzlauf	
Zufallspolitik	Return ≈ 21
Episoden	437
Schritte	49 103
Rechenzeit	94 s
gelöst (Ø20 ≥ 475)	Episode 426
greedy danach	Return 500

Umgebung: gymnasium CartPole-v1 · Belohnung +1 je Schritt, maximal 500 · Referenzlauf seed 0

Demo: **lab05_rl_optional.ipynb** – läuft auf dem Laptop, als Vertiefung zu Hause für alle

TFT-Bausteine im Detail

Gated Residual Network und interpretierbare Attention – die Gleichungen aus Lim et al. (2021)

$$\mathrm{GRN}(a, c) = \mathrm{LayerNorm}\big(a + \mathrm{GLU}(\eta_1)\big), \quad \eta_1 = W_1\eta_2 + b_1, \quad \eta_2 = \mathrm{ELU}(W_2 a + W_3 c + b_2)$$

$$\mathrm{GLU}(\gamma) = \sigma(W_4\gamma + b_4) \odot (W_5\gamma + b_5)$$

- **GLU-Tor:** σ ≈ 0 schaltet den Zweig ab → nur die Restverbindung a bleibt – Netz überspringt Ungenutztes
- **Variable Selection:** je Variable ein GRN, Softmax-Gewichte v_t = Variablenwichtigkeit
- **Interpretierbare Attention:** alle Köpfe teilen W_V, werden gemittelt → eine deutbare Attention-Matrix
- **Quantile:** eine lineare Schicht je Quantil, Verlust = Σ Pinball über Quantile und Horizonte

Lim, Arık, Loeff & Pfister (2021), Int. J. Forecasting 37(4), 1748–1764, Gl. 2–5 und Abschn. 4.4

Code: Backtest und LightGBM

Die Kernzeilen aus lab05 – gekürzt

```
def backtest(modelle, fenster=672):
    preds = []
    for o in URSPRUENGE:                        # 28 Tage, je 00:00 UTC
        hist = y.loc[:o - pd.Timedelta(hours=1)].iloc[-fenster:]
        sdf = pd.DataFrame({'unique_id': 'a', 'ds': hist.index, 'y': hist.values})
        preds.append(StatsForecast(models=modelle, freq='h').forecast(df=sdf, h=24))
    return pd.concat(preds).set_index('ds')

p_mstl = backtest([MSTL(season_length=[24, 168], trend_forecaster=AutoETS(model='ZZN'))], fenster=8*168)
```

```
def merkmale(y):
    X = kalender(y.index)                       # Stunde, Wochentag, Monat, Feiertag (Ortszeit)
    for l in [24, 48, 168, 336]:
        X[f'lag_{l}'] = y.shift(l)              # alle Lags >= 24 h -> day-ahead gültig
    X['mittel_24_lag24'] = y.shift(24).rolling(24).mean()
    return X

gbm = lgb.LGBMRegressor(n_estimators=500, learning_rate=0.05, num_leaves=31, random_state=0)
gbm.fit(X[maske_train], y[maske_train]);  p_gbm = gbm.predict(X.loc[backtest_index])
```

Code: TFT laden und befragen

Inferenz mit pytorch-forecasting – kein Training im Lab

```
from pytorch_forecasting import TemporalFusionTransformer
tft = TemporalFusionTransformer.load_from_checkpoint(
          DATA / 'models/tft_amprion/tft_amprion.ckpt', map_location='cpu')

for o in URSPRUENGE:
    i0 = d.index[d.ds == o][0]
    fenster = d.iloc[i0 - 168 : i0 + 24]        # 168 h Vergangenheit + 24 h bekannte Zukunft
    roh = tft.predict(fenster, mode='raw', return_x=True)
    q = roh.output.prediction[0].numpy()          # 24 x 3: q10, q50, q90
    deutung = tft.interpret_output(roh.output, reduction='sum')
    # deutung['attention'], deutung['encoder_variables'], deutung['decoder_variables']
```

Versionen fest in pyproject.toml: **pytorch-forecasting 1.8.0, lightning 2.6.6** – andere Versionen laden den Checkpoint ggf. nicht

Code zum Nachschlagen: Übersicht

Je Methode ein lauffähiger Schnipsel: Daten laden, Split, Modell mit Hyperparametern, fit, predict, Fehlermaß

Zeitreihen verstehen
- Zerlegung mit MSTL
- Autokorrelation und Differenzieren

Prognosen bewerten
- Baselines und Fehlermaße
- Backtest mit rollierendem Ursprung

Klassische Modelle
- Exponentielle Glättung und ETS
- SARIMA
- MSTL-Prognose
- Prophet

ML und Deep Learning
- Gradient Boosting mit LightGBM
- Quantilprognose und Pinball-Loss
- LSTM für 24 Stunden
- Temporal Fusion Transformer trainieren

Anomalien
- Isolation Forest
- Autoencoder als Anomaliedetektor

Reinforcement Learning
- Q-Learning mit Tabelle
- Deep Q-Network
- Policy Gradient (REINFORCE)
- PPO mit Stable-Baselines3

So nutzen: Pfade relativ zu einem Lab-Ordner (z. B. lab05/) – dort ein Notebook anlegen und den Block in eine Zelle kopieren. Alle Werte unten auf jeder Folie sind gemessen, Laufzeiten auf einem Laptop mit 2 CPU-Kernen.

Code: Zerlegung mit MSTL

Trend, Tagessaison, Wochensaison und Rest – mit statsmodels · zu „Trend, Saison und Rest“

```
import pandas as pd
from statsmodels.tsa.seasonal import MSTL

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv',
                 parse_dates=['ds'], index_col='ds')

y = df.loc['2019-08-05':'2019-09-29', 'load_mw']        # 8 Wochen

mstl = MSTL(y, periods=(24, 168))                       # Tag + Woche

res = mstl.fit()

anteile = {'Trend':        res.trend.var(),
           'Tagessaison':  res.seasonal['seasonal_24'].var(),
           'Wochensaison': res.seasonal['seasonal_168'].var(),
           'Rest':         res.resid.var()}

for name, var in anteile.items():
    print('{:<13} {:5.1%}'.format(name, var / y.var()))

res.plot()
```

Ausgabe: Tagessaison 65,9 % · Wochensaison 33,0 % · Trend 0,9 % · Rest 0,8 % der Varianz · ≈ 4 s

Code: Autokorrelation und Differenzieren

acf() liefert r_k für alle Lags, diff(m) entfernt die Saison der Periode m · zu „Autokorrelation“

```
import pandas as pd
from statsmodels.tsa.stattools import acf
from statsmodels.graphics.tsaplots import plot_acf

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv',
                 parse_dates=['ds'], index_col='ds')

train_y = df.loc[:'2019-09-30', 'load_mw']                # nur Training

r = acf(train_y, nlags=336)                               # r_0 ... r_336

for k in [1, 24, 168, 336]:
    print('r_{:<3} = {:.3f}'.format(k, r[k]))

plot_acf(train_y, lags=340)

z = train_y.diff(168).dropna()                            # saisonale Differenz, m = 168

r_z = acf(z, nlags=168)

print('nach diff(168): r_24 = {:.3f} | r_168 = {:.3f}'.format(r_z[24], r_z[168]))
```

Ausgabe: r_1 = 0,967 · r_24 = 0,761 · r_168 = 0,884 · r_336 = 0,847 · nach diff(168): r_168 = −0,355 · ≈ 2 s

Code: Baselines und Fehlermaße

Naiv und saisonal naiv per shift(), MAE/RMSE/MAPE aus scikit-learn, MASE selbst · zu „Fehlermaße“

```
import numpy as np
import pandas as pd
from sklearn.metrics import (mean_absolute_error, root_mean_squared_error,
                             mean_absolute_percentage_error)

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv',
                 parse_dates=['ds'], index_col='ds')
y = df['load_mw'].astype(float)

train_y = y[:'2019-09-30']
test_y  = y['2019-10-07':'2019-11-03']                    # 28 Tage

pred_naiv  = y.shift(24)[test_y.index]                    # Vortag, gleiche Stunde
pred_snaiv = y.shift(168)[test_y.index]                   # Vorwoche, gleiche Stunde

q = np.mean(np.abs(train_y.values[168:] - train_y.values[:-168]))   # MASE-Nenner

for name, pred_y in [('naiv', pred_naiv), ('saisonal naiv', pred_snaiv)]:
    mae  = mean_absolute_error(test_y, pred_y)
    rmse = root_mean_squared_error(test_y, pred_y)
    mape = mean_absolute_percentage_error(test_y, pred_y) * 100
    print('{:<14} MAE {:5.0f} MW | RMSE {:5.0f} MW | MAPE {:4.2f} % | MASE {:.3f}'
          .format(name, mae, rmse, mape, mae / q))
```

Ausgabe: naiv: MAE 1 562 MW · MASE 1,469 · saisonal naiv: MAE 927 MW · RMSE 1 467 MW · MAPE 4,43 % · MASE 0,872 · ≈ 2 s

Code: Backtest mit rollierendem Ursprung

cross_validation() aus statsforecast – 28 Ursprünge, je 24 h · zu „Backtesting mit rollierendem Ursprung“

```
import pandas as pd
from statsforecast import StatsForecast
from statsforecast.models import SeasonalNaive
from sklearn.metrics import mean_absolute_error

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv', parse_dates=['ds'])

data = df.rename(columns={'load_mw': 'y'}).assign(unique_id='amprion')
data = data.loc[data.ds <= '2019-11-03 23:00', ['unique_id', 'ds', 'y']]

sf = StatsForecast(models=[SeasonalNaive(season_length=168)], freq='h')

cv = sf.cross_validation(df=data,
                         h=24,              # Horizont: 24 Stunden
                         step_size=24,      # Ursprung rückt je 1 Tag weiter
                         n_windows=28)      # 28 Ursprünge: 7.10.-3.11.

print('Ursprünge: {} bis {}'.format(cv.cutoff.min(), cv.cutoff.max()))

test_mae = mean_absolute_error(cv['y'], cv['SeasonalNaive'])

print('Backtest-MAE: {:.0f} MW'.format(test_mae))
```

Ausgabe: Ursprünge: 2019-10-06 23:00 bis 2019-11-02 23:00 · Backtest-MAE: 927 MW · ≈ 2 s

Code: Exponentielle Glättung und ETS

AutoETS mit fester Struktur – oder per Automatik · zu „Die ETS-Familie“

```
import pandas as pd
from statsforecast import StatsForecast
from statsforecast.models import AutoETS
from sklearn.metrics import mean_absolute_error

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv', parse_dates=['ds'])

data = df.rename(columns={'load_mw': 'y'}).assign(unique_id='amprion')
data = data.loc[data.ds <= '2019-11-03 23:00', ['unique_id', 'ds', 'y']]

ets = AutoETS(season_length=24,     # Tagessaison
              model='ANA')          # Error A, Trend N, Season A
                                    # 'ANN' = einfache exp. Glättung, 'ZZZ' = Automatik
sf = StatsForecast(models=[ets], freq='h')

cv = sf.cross_validation(df=data, h=24, step_size=24, n_windows=28,
                         input_size=4 * 168)   # je Ursprung: letzte 4 Wochen

test_mae = mean_absolute_error(cv['y'], cv['AutoETS'])

print('Backtest-MAE ETS(A,N,A): {:.0f} MW'.format(test_mae))
```

Ausgabe: Backtest-MAE ETS(A,N,A): 1 758 MW · ≈ 3 s

Code: SARIMA

ARIMA mit festen Ordnungen $(p, d, q)(P, D, Q)_m$ · zu „ARIMA und SARIMA“

```
import pandas as pd
from statsforecast import StatsForecast
from statsforecast.models import ARIMA
from sklearn.metrics import mean_absolute_error

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv', parse_dates=['ds'])

data = df.rename(columns={'load_mw': 'y'}).assign(unique_id='amprion')
data = data.loc[data.ds <= '2019-11-03 23:00', ['unique_id', 'ds', 'y']]

sarima = ARIMA(order=(3, 0, 0),            # p, d, q   · Stundentakt
               seasonal_order=(2, 1, 0),   # P, D, Q   · Tagestakt
               season_length=24)           # m = 24

sf = StatsForecast(models=[sarima], freq='h')

cv = sf.cross_validation(df=data, h=24, step_size=24, n_windows=28,
                         input_size=4 * 168)

test_mae = mean_absolute_error(cv['y'], cv['ARIMA'])

print('Backtest-MAE SARIMA(3,0,0)(2,1,0)24: {:.0f} MW'.format(test_mae))
```

Ausgabe: Backtest-MAE SARIMA(3,0,0)(2,1,0)24: 1 620 MW · ≈ 20 s

Code: MSTL-Prognose

Zwei Saisons abziehen, Rest mit ETS fortschreiben · zu „Zwei Saisons gleichzeitig: MSTL“

```
import pandas as pd
from statsforecast import StatsForecast
from statsforecast.models import MSTL, AutoETS
from sklearn.metrics import mean_absolute_error

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv', parse_dates=['ds'])

data = df.rename(columns={'load_mw': 'y'}).assign(unique_id='amprion')
data = data.loc[data.ds <= '2019-11-03 23:00', ['unique_id', 'ds', 'y']]

mstl = MSTL(season_length=[24, 168],                    # Tag + Woche abziehen
            trend_forecaster=AutoETS(model='ZZN'))      # Rest + Trend mit ETS

sf = StatsForecast(models=[mstl], freq='h')

cv = sf.cross_validation(df=data, h=24, step_size=24, n_windows=28,
                         input_size=8 * 168)   # 8 Wochen: Wochensaison mehrfach

test_mae = mean_absolute_error(cv['y'], cv['MSTL'])

print('Backtest-MAE MSTL: {:.0f} MW'.format(test_mae))
```

Ausgabe: Backtest-MAE MSTL: 673 MW · ≈ 6 s

Code: Prophet

Regression auf Trend, Fourier-Saisons und Feiertage – ohne Lags · zu „Prophet: Zeitreihe als Regression“

```
# nicht in der Kursumgebung:  uv add prophet   (oder pip install prophet)
import pandas as pd
from prophet import Prophet
from sklearn.metrics import mean_absolute_error

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv', parse_dates=['ds'])
data = df.rename(columns={'load_mw': 'y'})[['ds', 'y']]

train = data[(data.ds >= '2019-07-01') & (data.ds < '2019-10-07')]
test  = data[(data.ds >= '2019-10-07') & (data.ds < '2019-11-04')]

m = Prophet(daily_seasonality=True,      # Fourier-Terme Periode 24 h
            weekly_seasonality=True,     # Periode 168 h
            yearly_seasonality=False,    # < 1 Jahr Training
            changepoint_prior_scale=0.05)
m.add_country_holidays(country_name='DE')

m.fit(train)

pred = m.predict(test[['ds']])           # keine Lags: alle 28 Tage auf einmal

test_mae = mean_absolute_error(test['y'], pred['yhat'])

print('Test-MAE Prophet: {:.0f} MW'.format(test_mae))
```

Ausgabe: Test-MAE Prophet: 925 MW · ≈ 3 s · nicht in der Kursumgebung: uv add prophet

Code: Gradient Boosting mit LightGBM

Merkmalstabelle aus Lags und Kalender, dann ein Regressor wie jeder andere · zu „Gradient Boosting: die Idee“

```
import pandas as pd
import lightgbm as lgb
from sklearn.metrics import mean_absolute_error

SEED = 0

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv',
                 parse_dates=['ds'], index_col='ds')
y = df['load_mw'].astype(float)

FEIERTAGE = pd.to_datetime([                        # NRW/RP/SL/BW/BY
    '2018-01-01', '2018-03-30', '2018-04-02', '2018-05-01', '2018-05-10',
    '2018-05-21', '2018-05-31', '2018-10-03', '2018-11-01', '2018-12-25',
    '2018-12-26', '2019-01-01', '2019-04-19', '2019-04-22', '2019-05-01',
    '2019-05-30', '2019-06-10', '2019-06-20', '2019-10-03', '2019-11-01',
    '2019-12-25', '2019-12-26'])
# Kalender in Ortszeit
lokal = y.index.tz_localize('UTC').tz_convert('Europe/Berlin')
```

```
tag = lokal.tz_localize(None).normalize()
X = pd.DataFrame({'stunde': lokal.hour,
                  'wochentag': lokal.dayofweek,
                  'monat': lokal.month,
                  'feiertag': tag.isin(FEIERTAGE).astype(int)}, index=y.index)
for lag in [24, 48, 168, 336]:  # alle Lags >= 24 h
    X['lag_{}'.format(lag)] = y.shift(lag)
X['mittel_24_lag24'] = y.shift(24).rolling(24).mean()

train_X = X[:'2019-09-30'].dropna()
test_X  = X['2019-10-07':'2019-11-03']
train_y, test_y = y[train_X.index], y[test_X.index]

gbm = lgb.LGBMRegressor(n_estimators=500,
                        learning_rate=0.05,
                        num_leaves=31,
                        random_state=SEED,
                        verbose=-1)

gbm.fit(train_X, train_y)

pred_y = gbm.predict(test_X)

test_mae = mean_absolute_error(test_y, pred_y)

print('Backtest-MAE LightGBM: {:.0f} MW'.format(test_mae))
```

Ausgabe: Backtest-MAE LightGBM: 488 MW · ≈ 3 s

Code: Quantilprognose und Pinball-Loss

Ein LightGBM je Quantil mit objective='quantile' · zu „Quantilprognosen und Pinball-Loss“

```
import pandas as pd
import lightgbm as lgb
import numpy as np
from sklearn.metrics import mean_pinball_loss

SEED = 0

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv',
                 parse_dates=['ds'], index_col='ds')
y = df['load_mw'].astype(float)

FEIERTAGE = pd.to_datetime([                        # NRW/RP/SL/BW/BY
    '2018-01-01', '2018-03-30', '2018-04-02', '2018-05-01', '2018-05-10',
    '2018-05-21', '2018-05-31', '2018-10-03', '2018-11-01', '2018-12-25',
    '2018-12-26', '2019-01-01', '2019-04-19', '2019-04-22', '2019-05-01',
    '2019-05-30', '2019-06-10', '2019-06-20', '2019-10-03', '2019-11-01',
    '2019-12-25', '2019-12-26'])
# Kalender in Ortszeit
lokal = y.index.tz_localize('UTC').tz_convert('Europe/Berlin')
```

```
tag = lokal.tz_localize(None).normalize()
X = pd.DataFrame({'stunde': lokal.hour,
                  'wochentag': lokal.dayofweek,
                  'monat': lokal.month,
                  'feiertag': tag.isin(FEIERTAGE).astype(int)}, index=y.index)
for lag in [24, 48, 168, 336]:  # alle Lags >= 24 h
    X['lag_{}'.format(lag)] = y.shift(lag)
X['mittel_24_lag24'] = y.shift(24).rolling(24).mean()

train_X = X[:'2019-09-30'].dropna()
test_X  = X['2019-10-07':'2019-11-03']
train_y, test_y = y[train_X.index], y[test_X.index]

pred = {}
for tau in [0.1, 0.5, 0.9]:
    gbm = lgb.LGBMRegressor(objective='quantile',     # Pinball-Loss statt MSE
                            alpha=tau,                # Quantilniveau
                            n_estimators=500, learning_rate=0.05,
                            random_state=SEED, verbose=-1)
    gbm.fit(train_X, train_y)
    pred[tau] = gbm.predict(test_X)
    pinball = mean_pinball_loss(test_y, pred[tau], alpha=tau)
    print('q{:.0f}: Pinball {:4.0f} MW'.format(tau * 100, pinball))

abdeckung = np.mean((test_y >= pred[0.1]) & (test_y <= pred[0.9]))

print('Abdeckung 10-90-%-Band: {:.1%} (Soll 80 %)'.format(abdeckung))
```

Ausgabe: Pinball q10 119 · q50 247 · q90 118 MW · Abdeckung 10–90-%-Band: 65,9 % (Soll 80 %) · ≈ 3 s

Code: LSTM für 24 Stunden

Fenster aus 168 h × 4 Kanälen hinein, 24 Werte heraus · zu „Ein LSTM für 24 Stunden“

```
import numpy as np
import pandas as pd
import torch
from sklearn.metrics import mean_absolute_error

SEED = 0
torch.manual_seed(SEED)

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv',
                 parse_dates=['ds'], index_col='ds')
y = df['load_mw'].astype(float)
lokal = y.index.tz_localize('UTC').tz_convert('Europe/Berlin')

# nur Trainingsstatistik
mu, sd = y[:'2019-09-30'].mean(), y[:'2019-09-30'].std()
F = np.stack([(y - mu) / sd,  # Last standardisiert
              np.sin(2 * np.pi * lokal.hour / 24),  # Stunde als Kreis
              np.cos(2 * np.pi * lokal.hour / 24),
              lokal.dayofweek >= 5], axis=1).astype(np.float32)  # Wochenende

def fenster(starts):                        # 168 h Eingabe -> 24 h Ziel
    X = np.stack([F[s - 168:s] for s in starts])
    Y = np.stack([F[s:s + 24, 0] for s in starts])
    return torch.tensor(X), torch.tensor(Y)

ende = y.index.get_loc(pd.Timestamp('2019-09-30 23:00'))
# jede 3. Stunde ein Fenster
train_X, train_y = fenster(np.arange(168, ende - 22, 3))
test_idx = pd.date_range('2019-10-07', '2019-11-03', freq='D')
# 28 Day-ahead-Ursprünge
test_X, test_y = fenster([y.index.get_loc(o) for o in test_idx])
```

```
class LSTMPrognose(torch.nn.Module):
    def __init__(self, n_in=4, hidden=32, horizont=24):
        super().__init__()
        self.lstm = torch.nn.LSTM(n_in, hidden, batch_first=True)
        self.kopf = torch.nn.Linear(hidden, horizont)
    def forward(self, x):
        _, (h, _) = self.lstm(x)  # letzter Zustand = Zusammenfassung der Woche
        return self.kopf(h[-1])

lstm = LSTMPrognose()
opt, loss_fn = torch.optim.Adam(lstm.parameters(), lr=3e-3), torch.nn.MSELoss()

for epoche in range(8):
    lstm.train()
    perm = torch.randperm(len(train_X))
    for i in range(0, len(train_X), 128):
        b = perm[i:i + 128]
        opt.zero_grad()
        loss_fn(lstm(train_X[b]), train_y[b]).backward()
        opt.step()

lstm.eval()
with torch.no_grad():
    pred_y = lstm(test_X).numpy() * sd + mu                    # zurück in MW

test_mae = mean_absolute_error(test_y.numpy() * sd + mu, pred_y)

print('Backtest-MAE LSTM: {:.0f} MW'.format(test_mae))
```

Ausgabe: Backtest-MAE LSTM: 745 MW · 5 656 Parameter, 8 Epochen · ≈ 15 s

Code: Temporal Fusion Transformer trainieren

TimeSeriesDataSet beschreibt die Eingänge, der Trainer übernimmt die Schleife · zu „Temporal Fusion Transformer“

```
import numpy as np
import pandas as pd
import lightning.pytorch as pl
from pytorch_forecasting import TimeSeriesDataSet, TemporalFusionTransformer
from pytorch_forecasting.metrics import QuantileLoss
from sklearn.metrics import mean_absolute_error

SEED = 0
pl.seed_everything(SEED)

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv',
                 parse_dates=['ds'])
df = df[df.ds <= '2019-11-03 23:00'].astype({'load_mw': float})
lokal = df.ds.dt.tz_localize('UTC').dt.tz_convert('Europe/Berlin')
df['stunde'] = lokal.dt.hour.astype(str)            # bekannte Zukunft:
df['wochentag'] = lokal.dt.dayofweek.astype(str)    # Kalender als
df['monat'] = lokal.dt.month.astype(str)            # Kategorien
df['time_idx'], df['serie'] = np.arange(len(df)), 'amprion'
t_test = df.time_idx[df.ds == '2019-10-07 00:00'].item()

train = TimeSeriesDataSet(df[df.ds <= '2019-09-30 23:00'],     # bis 30.9.
            time_idx='time_idx', target='load_mw', group_ids=['serie'],
            max_encoder_length=168,           # 1 Woche Vergangenheit
            max_prediction_length=24,         # 24 h Horizont
            time_varying_known_categoricals=['stunde', 'wochentag', 'monat'],
            time_varying_unknown_reals=['load_mw'],
            add_relative_time_idx=True, add_target_scales=True)
test = TimeSeriesDataSet.from_dataset(train, df, min_prediction_idx=t_test)
```

```
tft = TemporalFusionTransformer.from_dataset(train,
            hidden_size=32, attention_head_size=4, dropout=0.1,
            hidden_continuous_size=16, learning_rate=3e-3,
            loss=QuantileLoss(quantiles=[0.1, 0.5, 0.9]))

trainer = pl.Trainer(max_epochs=3,     # Lab-Checkpoint: bis 20 Epochen
                     limit_train_batches=150, gradient_clip_val=0.1,
                     accelerator='cpu', logger=False,
                     enable_checkpointing=False)

trainer.fit(tft, train.to_dataloader(train=True, batch_size=64, num_workers=0))

test_dl = test.to_dataloader(train=False, batch_size=256, num_workers=0)
pred = tft.predict(test_dl, mode='quantiles', return_y=True)
# pred.output: [Fenster, 24 h, 3 Quantile]  · Index 1 = Median

test_mae = mean_absolute_error(pred.y[0].ravel(), pred.output[..., 1].ravel())

print('Test-MAE TFT (Median): {:.0f} MW'.format(test_mae))
```

Ausgabe: Test-MAE TFT (Median): ≈ 580 MW · 3 Epochen ≈ 2–3 Min. · pytorch-forecasting 1.8.0, lightning 2.6.6

Code: Isolation Forest

Drei Merkmale je Tag aus dem Residuum, dann isolieren · zu „Isolation Forest“

```
import pandas as pd
from sklearn.ensemble import IsolationForest

SEED = 0

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv',
                 parse_dates=['ds'], index_col='ds')
y = df['load_mw'].astype(float)

resid = (y - y.shift(168))['2019-10-01':]          # Residuum der Vorwochen-Prognose

tag = resid.resample('D')
X = pd.DataFrame({'mittel':   tag.mean(),           # drei Merkmale je Tag
                  'streuung': tag.std(),
                  'max_abs':  tag.apply(lambda s: s.abs().max())})

iso = IsolationForest(n_estimators=200,
                      contamination=0.05,           # erwarteter Anteil Anomalien
                      random_state=SEED)

iso.fit(X)

X['anomalie'] = iso.predict(X) == -1                # -1 = Anomalie, +1 = normal
X['score'] = -iso.score_samples(X[['mittel', 'streuung', 'max_abs']])   # größer = auffälliger

print('{} von {} Tagen auffällig'.format(X.anomalie.sum(), len(X)))
print(X[X.anomalie].sort_values('score', ascending=False).head(8).round({'mittel': 0, 'streuung': 0, 'max_abs': 0, 'score': 3}))
```

Ausgabe: 14 von 274 Tagen auffällig · Spitze: 8.1.2020, 26.12., 7.1., 25.12., Allerheiligen, 24.12., 3.10. · ≈ 2 s

Code: Autoencoder als Anomaliedetektor

24 → 8 → 4 → 8 → 24, trainiert auf Tagesprofilen, Schwelle aus dem Training · zu „Autoencoder als Anomaliedetektor“

```
import numpy as np
import pandas as pd
import torch

SEED = 0
torch.manual_seed(SEED)

df = pd.read_csv('../data/opsd/amprion_load_2018_2020H1.csv',
                 parse_dates=['ds'], index_col='ds')
y = df['load_mw'].astype(float)

profile = y.values.reshape(-1, 24)  # eine Zeile = ein Tag (UTC)
tage = y.index[::24]
# nur die Form, nicht das Niveau
X = profile / profile.mean(axis=1, keepdims=True) - 1
X = torch.tensor(X, dtype=torch.float32)

train_X = X[tage <= '2019-09-30']
test_X, test_tage = X[tage >= '2019-10-01'], tage[tage >= '2019-10-01']
```

```
ae = torch.nn.Sequential(
    torch.nn.Linear(24, 8), torch.nn.ReLU(), torch.nn.Linear(8, 4),  # Encoder
    torch.nn.Linear(4, 8), torch.nn.ReLU(), torch.nn.Linear(8, 24))  # Decoder
opt, loss_fn = torch.optim.Adam(ae.parameters(), lr=1e-2), torch.nn.MSELoss()

for epoche in range(500):  # Ziel = Eingabe, keine Labels
    opt.zero_grad()
    loss_fn(ae(train_X), train_X).backward()
    opt.step()

with torch.no_grad():
    fehler_train = ((ae(train_X) - train_X) ** 2).mean(dim=1).numpy()
    fehler_test  = ((ae(test_X) - test_X) ** 2).mean(dim=1).numpy()

schwelle = np.percentile(fehler_train, 99)  # 99. Perzentil auf Trainingstagen

auffaellig = pd.Series(fehler_test, index=test_tage)[fehler_test > schwelle]

print('{} von {} Tagen auffällig'.format(len(auffaellig), len(test_X)))
print(auffaellig.sort_values(ascending=False).head(8).round(4))
```

Ausgabe: 2 von 274 Tagen auffällig: 1.1.2020, 2.2.2020 · ≈ 3 s

Code: Q-Learning mit Tabelle

FrozenLake: 16 Zustände × 4 Aktionen, ε-greedy und TD-Update · zu „Q-Learning“

```
import numpy as np
import gymnasium as gym

SEED = 0
rng = np.random.default_rng(SEED)

# 16 Felder, rutschig
env = gym.make('FrozenLake-v1', map_name='4x4', is_slippery=True)

Q = np.zeros((env.observation_space.n, env.action_space.n))  # Tabelle 16 x 4

alpha, gamma, eps = 0.1, 0.99, 0.1      # Lernrate, Diskont, Erkundungsrate

def waehle(s, eps):                     # epsilon-greedy, Gleichstand zufällig
    if rng.random() < eps:
        return env.action_space.sample()
    return int(rng.choice(np.flatnonzero(Q[s] == Q[s].max())))
```

```
env.action_space.seed(SEED)
for episode in range(20_000):
    s, _ = env.reset(seed=SEED + episode)
    fertig = False
    while not fertig:
        a = waehle(s, eps)
        s2, r, term, trunc, _ = env.step(a)
        ziel = r + gamma * Q[s2].max() * (not term)          # TD-Ziel
        Q[s, a] += alpha * (ziel - Q[s, a])                  # Q-Update
        s, fertig = s2, term or trunc

erfolge = []
for i in range(1000):  # gelernte Politik testen
    s, _ = env.reset(seed=100_000 + i)
    fertig = False
    while not fertig:
        s, r, term, trunc, _ = env.step(waehle(s, eps=0.0))
        fertig = term or trunc
    erfolge.append(r)

print('Erfolgsquote greedy: {:.1%}'.format(np.mean(erfolge)))
# 0 = links, 1 = runter, 2 = rechts, 3 = hoch
print(np.argmax(Q, axis=1).reshape(4, 4))
```

Ausgabe: Erfolgsquote greedy: 72,8 % (bestmöglich ≈ 74 % auf rutschigem Eis) · 20 000 Episoden ≈ 13 s

Code: Deep Q-Network

Q-Netz, Zielnetz und Replay-Puffer auf CartPole · zu „Deep Q-Network (DQN)“

```
import random, collections
import numpy as np
import torch
import gymnasium as gym

SEED = 0
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
torch.set_num_threads(1)

env = gym.make('CartPole-v1')  # Zustand: 4 Zahlen, Aktionen: links/rechts
env.action_space.seed(SEED)

GAMMA, LR, BATCH, TAU = 0.99, 1e-4, 128, 0.005
def netz():
    return torch.nn.Sequential(torch.nn.Linear(4, 128), torch.nn.ReLU(),
                               torch.nn.Linear(128, 128), torch.nn.ReLU(),
                               torch.nn.Linear(128, 2))
q, q_ziel = netz(), netz()                          # Q-Netz und Zielnetz
q_ziel.load_state_dict(q.state_dict())
opt = torch.optim.AdamW(q.parameters(), lr=LR, amsgrad=True)
puffer = collections.deque(maxlen=10_000)           # Replay-Puffer

def waehle(s, eps):
    if random.random() < eps:
        return env.action_space.sample()
    with torch.no_grad():
        return int(q(torch.tensor(s)).argmax())
```

```
schritte = 0
for episode in range(500):
    s, _ = env.reset(seed=SEED + episode)
    fertig = False
    while not fertig:
        eps = 0.05 + 0.85 * np.exp(-schritte / 1000)  # Erkundung nimmt ab
        a = waehle(s, eps)
        s2, r, term, trunc, _ = env.step(a)
        puffer.append((s, a, r, s2, float(term)))
        s, fertig, schritte = s2, term or trunc, schritte + 1
        if len(puffer) < BATCH:
            continue
        S, A, R, S2, T = (torch.tensor(np.array(x), dtype=torch.float32)
                          for x in zip(*random.sample(puffer, BATCH)))
        with torch.no_grad():                                  # TD-Ziel
            ziel = R + GAMMA * (1 - T) * q_ziel(S2).max(dim=1).values
        wert = q(S).gather(1, A.long().unsqueeze(1)).squeeze(1)  # Q(s, a)
        loss = torch.nn.functional.smooth_l1_loss(wert, ziel)
        opt.zero_grad(); loss.backward(); opt.step()
        with torch.no_grad():  # Zielnetz weich nachführen
            for pz, p in zip(q_ziel.parameters(), q.parameters()):
                pz.mul_(1 - TAU).add_(TAU * p)

returns = []
for i in range(20):  # gelernte Politik testen
    s, _ = env.reset(seed=2000 + i)
    ret, fertig = 0.0, False
    while not fertig:
        s, r, term, trunc, _ = env.step(waehle(s, eps=0.0))
        ret, fertig = ret + r, term or trunc
    returns.append(ret)

print('mittlerer Return greedy: {:.1f} (max. 500)'.format(np.mean(returns)))
```

Ausgabe: mittlerer Return greedy: 500,0 (max. 500) · 500 Episoden ≈ 5 Min. auf 1 CPU-Kern

Code: Policy Gradient (REINFORCE)

Netz gibt Aktionswahrscheinlichkeiten aus, hoher Return macht Aktionen wahrscheinlicher · zu „Policy Gradient und PPO“

```
import numpy as np
import torch
import gymnasium as gym

SEED = 0
torch.manual_seed(SEED)

env = gym.make('CartPole-v1')

politik = torch.nn.Sequential(torch.nn.Linear(4, 64), torch.nn.ReLU(),
                              torch.nn.Linear(64, 2))      # Logits: links/rechts
opt = torch.optim.Adam(politik.parameters(), lr=3e-3)
GAMMA = 0.99

for episode in range(600):
    s, _ = env.reset(seed=SEED + episode)
    log_p, belohnungen, fertig = [], [], False
    while not fertig:
        logits = politik(torch.tensor(s))
        verteilung = torch.distributions.Categorical(logits=logits)
        a = verteilung.sample()  # Aktion würfeln: pi(a | s)
        s, r, term, trunc, _ = env.step(a.item())
        log_p.append(verteilung.log_prob(a)); belohnungen.append(r)
        fertig = term or trunc
    G, returns = 0.0, []
    for r in reversed(belohnungen):  # Return G_t je Schritt
        G = r + GAMMA * G
        returns.insert(0, G)
    G_t = torch.tensor(returns)
    G_t = (G_t - G_t.mean()) / (G_t.std() + 1e-8)  # normieren = Baseline

    loss = -(torch.stack(log_p) * G_t).sum()                    # REINFORCE
    opt.zero_grad(); loss.backward(); opt.step()

    if episode % 100 == 0:
        print('Episode {:3d}: Return {:.0f}'.format(episode, sum(belohnungen)))

returns = []
for i in range(20):  # gelernte Politik testen (greedy)
    s, _ = env.reset(seed=2000 + i)
    ret, fertig = 0.0, False
    while not fertig:
        with torch.no_grad():
            a = int(politik(torch.tensor(s)).argmax())
        s, r, term, trunc, _ = env.step(a)
        ret, fertig = ret + r, term or trunc
    returns.append(ret)

print('mittlerer Return greedy: {:.1f} (max. 500)'.format(np.mean(returns)))
```

Ausgabe: Return 14 (Episode 0) … 500 (Episode 500) · mittlerer Return greedy: 500,0 · 600 Episoden ≈ 1,5 Min.

Code: PPO mit Stable-Baselines3

Fertige Implementierung – die Hyperparameter entsprechen den Symbolen der Folie · zu „Policy Gradient und PPO“

```
# nicht in der Kursumgebung:  uv add stable-baselines3   (oder pip install stable-baselines3)
import numpy as np
import gymnasium as gym
from stable_baselines3 import PPO
from stable_baselines3.common.evaluation import evaluate_policy

SEED = 0

env = gym.make('CartPole-v1')

ppo = PPO('MlpPolicy', env,
          learning_rate=3e-4,
          n_steps=2048,          # Schritte sammeln, dann aktualisieren
          clip_range=0.2,        # epsilon: höchstens ±20 % Änderung von pi
          gamma=0.99,
          seed=SEED, verbose=0)

ppo.learn(total_timesteps=100_000)

mittel, streuung = evaluate_policy(ppo, env, n_eval_episodes=20, deterministic=True)

print('mittlerer Return greedy: {:.1f} ± {:.1f} (max. 500)'.format(mittel, streuung))
```

Ausgabe: mittlerer Return greedy: 500,0 ± 0,0 (max. 500) · 100 000 Schritte ≈ 1,5 Min. · nicht in der Kursumgebung: uv add stable-baselines3